BOT AUDIT CODE REVIEW

Generated: 2026-05-30 11:52:01
Source Directory: C:\Users\pc\Desktop\projects\Quad-desk\bot

Contents:

• requirements.txt
• __init__.py
• signal_config.py
• data_feed.py
• derivatives_context.py
• heartbeat.py
• ulis_engine.py
• executor.py
• notifier.py
• main.py
• quant_engine.py
• backtest_hybrid.py
• verify_region.py

■ requirements.txt

```
ccxt>=4.4.45
websockets>=12.0
pandas>=2.0.0
numpy>=1.24.0
google-genai>=1.0.0
python-dotenv>=1.0.0
firebase-admin>=6.4.0
cryptography>=42.0.0
httpx>=0.27.0
```

■ __init__.py

```
# Bot package init
```

■ signal_config.py

```
"""
bot/signal_config.py
====================
Single source of truth for ALL signal thresholds across the Quad-Desk trading bot.

Update thresholds HERE — never in individual files.  Both the live execution engine
(bot/main.py) and backtester (bot/backtest_hybrid.py) should import from this module
so that backtest and live results are always synchronised.

OFI RANGE NOTE (v3, Apr 2026):
    After the Three-Stage OFI Integrity Pipeline (quant_engine.py), OFI output
    is in the range (-1, +1) via tanh normalisation.  All OFI thresholds below
    are expressed in this new normalised scale.  Old scale was ±100.
"""

# ■■ Bayesian / Confidence ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
MIN_BAYESIAN           = 0.62   # Minimum posterior confidence for any entry
MIN_BAYESIAN_STRONG    = 0.72   # Strong-signal threshold (STRONG_LONG / STRONG_SHORT)

# ■■ Z-Score (Student's t-distribution VWAP Z) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# M-02 FIX: _vwap_z_score_t() already shrinks the Gaussian Z by t_scale=0.8165.
# Thresholds must be multiplied by 0.8165 to restore the intended σ boundaries.
# e.g. old 1.5 → effective Gaussian req of 1.83σ → now correctly 1.22 = 1.5×0.8165
Z_OVERSOLD             = -1.22  # Student-t adj: was -1.5
Z_OVERBOUGHT           =  1.22  # Student-t adj: was  1.5
MEAN_REV_Z_THRESHOLD   =  1.22  # Student-t adj: was  1.5 — matches NEUTRAL z_threshold

# ■■ OFI (tanh output, range -1 to +1) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
OFI_STRONG_THRESHOLD   =  0.6   # Strong directional flow (equivalent to old |OFI| > 40)
OFI_MID_THRESHOLD      =  0.3   # Moderate flow (equivalent to old |OFI| > 10)
OFI_WEAK_THRESHOLD     =  0.15  # Weak / noise floor

# ■■ Skewness ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
SKEW_SIGNIFICANT       = 0.3    # |skewness| above this is considered meaningful

# ■■ Risk / Position Sizing ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
MAX_RISK_PCT           = 1.0    # Max risk per trade as % of equity
MAX_DAILY_LOSS_PCT     = 3.0    # Stop trading if daily loss hits this %
MAX_DRAWDOWN_PCT       = 15.0   # Hard session drawdown halt (restart required)
ATR_SL_MULTIPLIER      = 1.5    # SL = entry ± ATR × this
ATR_TP_MULTIPLIER      = 3.0    # TP = entry ± ATR × this (3:2 RR minimum)

# ■■ Strategy Scoring ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
TREND_SCORE_MIN        = 1.5    # Minimum composite score for a trend entry
WALL_PROXIMITY_PCT     = 0.003  # % distance to qualify as LIQUIDITY regime

# ■■ ULIS Gate ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ULIS_CONFIDENCE_STRONG = 0.68   # ALDE confidence threshold for STRONG_LONG/SHORT
ULIS_CASCADE_ABORT     = 0.65   # cascade_risk above this → AVOID verdict

# ■■ Cooldowns ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
POST_TRADE_COOLDOWN_S  = 90     # seconds after any exit before new entry allowed
CASCADE_COOLDOWN_S     = 300    # seconds after SL exit (cascade prevention)
CONSECUTIVE_LOSS_HALT  = 3      # was 2 — reduces false lockouts

# ■■ Bayesian Cold-Start ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
COLD_START_TRADE_COUNT = 30
COLD_START_CONFIDENCE_DISCOUNT = 0.05

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# ■■ REGIME-CONDITIONAL PARAMETER MATRIX (HMM Spec, Apr 2026) ■■■■■■■■■■■■■■■■■■■■
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
#
# Every downstream stage reads from this dict rather than using static constants.
# Replaces fixed thresholds (e.g. z >= 2.2 always) with adaptive ones that fit
# the current market environment as classified by the HMM.
#
# Regime → Strategy mapping:
#   RANGE     → Mean-reversion dominant (quiet market, tight SL, low Z bar)
#   NEUTRAL   → All strategies active  (default / transitional)
#   TREND     → Trend-following dominant (noisy market, wide SL, high Z bar)
#   LIQUIDITY → Wall-proximity sweep   (same defaults as NEUTRAL, no HTF block)
#
REGIME_PARAMS = {
    "RANGE": {
        # Low-volatility: small deviations are statistically meaningful
```

```
        "z_threshold":        1.06,  # M-02 FIX: was 1.3, Student-t adj (1.3×0.8165)
        "atr_multiplier_sl":  1.14,   # Was 1.2 — 5% reduction post-Wilder compensation
        "ofi_bound":          0.08,  # Was 0.10. Needs less order flow to enter
        "min_confidence":     0.62,  # was 0.55 — raised to match LIQUIDITY threshold
        "rr_target":          1.8,   # 1.8:1 minimum R:R
        "be_lock_trigger":    0.8,   # Move SL to break-even after 0.8×ATR profit
        "panic_threshold":    2.5,   # Flash-crash trigger
        "candle_gate_sec":    30,    # Was 45s. Faster entry
        "htf_block":          False, # Counter-HTF is the strategy in RANGE
        "cascade_cooldown_s": 180,   # PHASE-2.5: was 300s, lowered to 180s for RANGE recovery
    },
    "NEUTRAL": {
        # Normal-volatility: balanced thresholds
        "z_threshold":        1.22,  # M-02 FIX: was 1.5, Student-t adj (1.5×0.8165)
        "atr_multiplier_sl":  1.43,
        "ofi_bound":          0.10,  # PHASE-4.1: was 0.12, lowered to 0.10 for more signal pass-through
        "min_confidence":     0.62,  # was 0.55 — raised to match LIQUIDITY threshold
        "rr_target":          2.0,
        "be_lock_trigger":    1.5,  # widened: move SL to BE only after 1.5×ATR profit
        "panic_threshold":    5.0,
        "candle_gate_sec":    45,    # Was 60s
        "htf_block":          True,
        "cascade_cooldown_s": 300,   # STRATEGY-B: 5min (default)
    },
    "TREND": {
        # High-volatility: trend-following
        "z_threshold":        2.0,   # Was 2.5. Triggers on shallower pullbacks
        "atr_multiplier_sl":  2.09,  # Was 2.2 — 5% reduction post-Wilder compensation
        "ofi_bound":          0.20,  # Was 0.25
        "min_confidence":     0.65,  # Was 0.72. Massive increase in trend trades
        "rr_target":          2.5,
        "be_lock_trigger":    2.0,  # widened: let trend breathe — do not lock BE until 2×ATR profit
        "panic_threshold":    8.0,
        "candle_gate_sec":    60,    # Was 90s
        "htf_block":          True,
        "cascade_cooldown_s": 240,   # STRATEGY-B: 4min (trend may still be valid)
    },
"LIQUIDITY": {
        # Wall-proximity sweeps
        "z_threshold":        1.22,  # M-02 FIX: was 1.5, Student-t adj (1.5×0.8165)
        "atr_multiplier_sl":  1.43,
        "ofi_bound":          0.12,
        "min_confidence":     0.62,  # FIX-P10: was 0.55 which == sweep neutralizer floor (no-op); raised to 0.62 so th
        "rr_target":          2.0,
        "be_lock_trigger":    1.5,  # widened: move SL to BE only after 1.5×ATR profit
        "panic_threshold":    5.0,
        "candle_gate_sec":    45,    # Was 60s
        "htf_block":          False,
        "cascade_cooldown_s": 180,   # STRATEGY-B: 3min (sweep may repeat next candle)
    },
}
```

■ data_feed.py

```
import asyncio
import json
import logging
import time
import websockets
import httpx
from collections import deque

logger = logging.getLogger(__name__)


class MarketState:
    def __init__(self, symbol: str):
        self.symbol = symbol.upper()
        self.candles: deque = deque(maxlen=200)
        # Keep all trades received; prune old ones in add_trade
        self.recent_trades: deque = deque(maxlen=50000)
        # Order Book snapshot { price_float: size_float }
        self.bids: dict = {}
        self.asks: dict = {}
        # Running cumulative volume delta
        self.cvd: float = 0.0
        # Binance USDM funding rate — fetched periodically via REST.
        # Positive = longs pay shorts (crowded long → bearish pressure).
        # Negative = shorts pay longs (crowded short → bullish squeeze).
        self.funding_rate: float = 0.0
        self.basis: float = 0.0       # Futures mark price - Spot close
        self.mark_price: float = 0.0  # Latest Futures mark price
        self._reconnect_event: asyncio.Event = asyncio.Event()
        self.candle_close_event: asyncio.Event = asyncio.Event()  # NEW: Event-driven execution trigger
        self._last_trade_ts: float = time.time()   # epoch-seconds of last aggTrade received (P0-1 FIX: was 0.0 → time
        self._cvd_was_reset: bool = False  # flag to suppress CVD delta spike after reconnect
        self._aggtrade_msg_count: int = 0  # P0-1 FIX: throughput counter for monitoring
        self._aggtrade_count_reset_ts: float = time.time()  # last reset for msg/min calculation

    # ------------------------------------------------------------------
    # Candle management
    # ------------------------------------------------------------------
    def add_candle(self, c_data: dict, is_final: bool):
        """Add or update the current live candle. Only keep the last MAX_CANDLES."""
        candle = {
            'time': c_data['t'] / 1000,
            'open': float(c_data['o']),
            'high': float(c_data['h']),
            'low':  float(c_data['l']),
            'close': float(c_data['c']),
            'volume': float(c_data['v'])
        }
        if is_final:
            # Closed candle — always append
            self.candles.append(candle)
            self.candle_close_event.set()  # Trigger zero-latency execution
        else:
            # Live candle — replace if same timestamp, otherwise append
            if self.candles and self.candles[-1]['time'] == candle['time']:
                self.candles[-1] = candle
            else:
                self.candles.append(candle)

    # ------------------------------------------------------------------
    # Trade tape
    # ------------------------------------------------------------------
    def add_trade(self, t_data: dict):
        price = float(t_data['p'])
        size = float(t_data['q'])
        is_buyer_maker = t_data['m']
        # If buyer is maker they were resting → the *seller* aggressed → SELL tape
        side = 'SELL' if is_buyer_maker else 'BUY'

        trade = {
            'price': price,
            'size': size,
            'usd_volume': price * size,
            'side': side,
            'time': t_data['T']  # Binance millisecond timestamp
        }
```

```
        self._last_trade_ts = t_data['T'] / 1000.0
        self._aggtrade_msg_count += 1
        self.recent_trades.append(trade)

        # Update CVD
        delta = size if side == 'BUY' else -size
        self.cvd += delta

        # Prune trades older than 5 minutes using wall-clock time
        now_ms = time.time() * 1000
        while self.recent_trades and (now_ms - self.recent_trades[0]['time']) > 300_000:
            self.recent_trades.popleft()

    # ------------------------------------------------------------------
    # Order book
    # ------------------------------------------------------------------
    def update_depth(self, depth_data: dict):
        """Full snapshot of the top-20 levels."""
        raw_b = depth_data.get('b', depth_data.get('bids', []))
        raw_a = depth_data.get('a', depth_data.get('asks', []))
        self.bids = {float(p): float(q) for p, q in raw_b}
        self.asks = {float(p): float(q) for p, q in raw_a}
        # Remove levels with zero quantity (Binance sends these as deletes)
        self.bids = {p: q for p, q in self.bids.items() if q > 0}
        self.asks = {p: q for p, q in self.asks.items() if q > 0}


class BinanceDataFeed:
    """
    Async WebSocket feed for Binance USDM Futures (Live or Testnet).

    Live endpoint:    wss://fstream.binance.com/stream?streams=...
    Testnet endpoint: wss://stream.binancefuture.com/stream?streams=...

    REST klines use fapi.binance.com/fapi/v1/klines (futures),
    NOT api.binance.com/api/v3/klines (spot).
    Using spot endpoints for a futures bot produces slightly different
    price/volume data and misses funding-rate-driven price divergence.
    """

    def __init__(self, symbol: str = "BTCUSDT", interval: str = "15m", testnet: bool = True):
        self.symbol = symbol.lower()
        self.interval = interval.lower()
        self.state = MarketState(symbol)

        # Binance USDM Futures endpoints (separate from Spot)
        if testnet:
            self.rest_url = "https://testnet.binancefuture.com"
            base_url = "wss://stream.binancefuture.com"
        else:
            # LIVE FUTURES DATA BLOCKED IN EU/US — using Spot Proxy for WS Data
            self.rest_url = "https://fapi.binance.com"  # REST works via API keys/CDN
            base_url = "wss://stream.binance.com:9443"  # Spot WebSocket bypasses geoblock

        streams = (
            f"{self.symbol}@kline_{self.interval}"
            f"/{self.symbol}@aggTrade"      # Spot & Futures both use aggTrade
            f"/{self.symbol}@depth20@100ms"
        )
        self.ws_url = f"{base_url}/stream?streams={streams}"
        self.is_running = False
        self._rest_fetch_lock = asyncio.Lock()
        self._last_funding_fetch: float = 0.0   # epoch-seconds of last funding rate REST call
        self._last_kline_frame_ts: float = 0.0  # epoch-seconds of last kline WS frame received

    # ------------------------------------------------------------------
    # Message routing
    # ------------------------------------------------------------------
    async def _handle_message(self, raw: str):
        try:
            payload = json.loads(raw)
        except json.JSONDecodeError as e:
            logger.warning(f"JSON decode error: {e}")
            return

        if 'stream' not in payload:
            return  # Subscription confirmations etc.
        stream: str = payload['stream']
```

```
    data: dict = payload['data']

    try:
        if '@kline_' in stream:
            self.state.add_candle(data['k'], data['k']['x'])
            self._last_kline_frame_ts = time.time()  # P0-2: track freshness for health monitor
        elif '@aggTrade' in stream:
            # Futures aggTrade uses same fields as Spot trade (p, q, m, T)
            self.state.add_trade(data)
        elif '@depth' in stream:
            self.state.update_depth(data)
    except KeyError as e:
        logger.error(f"Missing key in {stream} payload: {e}")
    except Exception as e:
        logger.error(f"Error processing stream '{stream}': {e}", exc_info=True)

# ------------------------------------------------------------------
# Funding Rate Fetch (periodic, every 60 s)
# ------------------------------------------------------------------
async def _fetch_funding_rate(self):
    """
    Fetch the current funding rate from Binance USDM Futures REST.
    Endpoint: GET /fapi/v1/premiumIndex?symbol=BTCUSDT
    Runs every 60 s; independent signal used by ULIS engine.
    """
    url = f"{self.rest_url}/fapi/v1/premiumIndex"
    params = {"symbol": self.symbol.upper()}
    import os
    api_key = os.environ.get("BINANCE_API_KEY", "")
    headers = {"X-MBX-APIKEY": api_key} if api_key else {}
    try:
        async with httpx.AsyncClient(timeout=6.0) as client:
            resp = await client.get(url, params=params, headers=headers)
            resp.raise_for_status()
            data = resp.json()
        rate = float(data.get("lastFundingRate", 0.0))
        self.state.funding_rate = rate
        # NEW: Track Futures-Spot basis for SL/TP price correction
        mark_price = float(data.get("markPrice", 0.0))
        index_price = float(data.get("indexPrice", 0.0))
        if mark_price > 0 and len(self.state.candles) > 0:
            spot_close = self.state.candles[-1]["close"]
            self.state.basis = mark_price - spot_close   # positive = Futures premium
            self.state.mark_price = mark_price
        else:
            self.state.basis = 0.0
            self.state.mark_price = mark_price
        logger.info(
            f"[DataFeed] Funding rate: {rate:+.6f} ({rate*100:+.4f}%) | "
            f"Basis: {getattr(self.state, 'basis', 0.0):+.2f}"
        )
    except Exception as e:
        logger.warning(f"[DataFeed] Failed to fetch funding rate: {e}")

async def funding_rate_loop(self):
    """
    PHASE-0.3: Standalone funding rate fetch loop.
    Runs independently of the WebSocket connection, polling every 60 s.
    Removes the funding fetch from inside the WS handler (which was
    silently dropped on reconnect without retry).
    """
    await asyncio.sleep(5)
    while self.is_running:
        await self._fetch_funding_rate()
        try:
            await asyncio.sleep(60)
        except asyncio.CancelledError:
            break

# ------------------------------------------------------------------
# REST API Prefetch
# ------------------------------------------------------------------
async def _fetch_historical_candles_rest(self):
    """Fetch 100 recent candles from Binance USDM Futures REST to warm up the Quant Engine."""
    async with self._rest_fetch_lock:
        # Futures klines live under /fapi/v1/klines, NOT /api/v3/klines (spot)
        url = f"{self.rest_url}/fapi/v1/klines"
        params = {
```

```
            "symbol": self.symbol.upper(),
            "interval": self.interval,
            "limit": 100
        }
        import os
        api_key = os.environ.get("BINANCE_API_KEY", "")
        headers = {"X-MBX-APIKEY": api_key} if api_key else {}

        # 3.3 FIX: Snapshot CVD before any mutation.
        # If REST fails (network drop, reconnect), we preserve the existing baseline
        # rather than resetting to 0.0 which would produce a false CVD delta spike.
        prev_cvd = self.state.cvd

        try:
            logger.info(f"[DataFeed] Fetching historical {self.interval} candles from {url}...")
            async with httpx.AsyncClient(timeout=10.0) as client:
                resp = await client.get(url, params=params, headers=headers)
                resp.raise_for_status()
                data = resp.json()

            # Reset CVD to re-anchor perfectly based on REST history (only on success)
            self.state.cvd = 0.0
            parsed_candles = []
            for k in data:
                # Binance REST returns an array of arrays
                # [openTime, open, high, low, close, volume, closeTime, qav, trades, taker_buy_base, taker_buy_quot
                c_data = {
                    't': int(k[0]),
                    'o': float(k[1]),
                    'h': float(k[2]),
                    'l': float(k[3]),
                    'c': float(k[4]),
                    'v': float(k[5]),
                }
                parsed_candles.append({"close": float(k[4]), "high": float(k[2]), "low": float(k[3]), "volume": flo
                self.state.add_candle(c_data, is_final=True)

                # Rebuild CVD analytically
                taker_buy_base = float(k[9])
                vol = float(k[5])
                self.state.cvd += (2.0 * taker_buy_base) - vol

            logger.info(f"[DataFeed] Successfully loaded {len(self.state.candles)} historical candles. CVD rebuilt:
            self.candles_seeded = True
            return parsed_candles
        except Exception as e:
            logger.warning(
                f"[DataFeed] Failed to prefetch historical candles ({e}). "
                f"Preserving existing CVD={prev_cvd:.0f} to avoid false delta spike."
            )
            self.state.cvd = prev_cvd  # restore — do not corrupt the signal



# ---------------------------------------------------------------------
# Connection loop with exponential back-off
# ---------------------------------------------------------------------
async def run(self):
    self.is_running = True
    retry_delay = 1

    # ALWAYS fill history before opening streaming connections to prevent the 51-interval delay
    if not getattr(self, "candles_seeded", False):
        await self._fetch_historical_candles_rest()

    while self.is_running:
        try:
            logger.info(f"[DataFeed] Connecting → {self.ws_url}")
            async with websockets.connect(
                self.ws_url,
                ping_interval=20,
                ping_timeout=30,
                close_timeout=10
            ) as ws:
                retry_delay = 1  # Reset back-off on successful connect
                self.state.cvd = 0.0
                self.state._cvd_was_reset = True  # Signal main loop to suppress next delta
                logger.info("[DataFeed] Connected ✓")
```

```
                # PHASE-0.3: Fetch funding rate immediately on connection (before WS loop)
                asyncio.create_task(self._fetch_funding_rate())
                import asyncio as _asyncio
                while self.is_running and not self.state._reconnect_event.is_set():
                    try:
                        msg = await _asyncio.wait_for(ws.recv(), timeout=120)
                        await self._handle_message(msg)
                    except _asyncio.TimeoutError:
                        logger.warning("[DataFeed] recv() timeout (120s) — connection may be frozen. Reconnecting..
                        break
                    except websockets.exceptions.ConnectionClosedOK:
                        break
                # Clear reconnect signal so the outer loop reconnects immediately
                if self.state._reconnect_event.is_set():
                    self.state._reconnect_event.clear()
                    logger.info('[DataFeed] Reconnect event consumed - reconnecting now.')

        except websockets.exceptions.ConnectionClosedOK:
            logger.info("[DataFeed] Connection closed cleanly.")
        except websockets.exceptions.ConnectionClosed as e:
            logger.warning(f"[DataFeed] Connection dropped: {e}. Reconnecting in {retry_delay}s…")
        except OSError as e:
            logger.error(f"[DataFeed] Network error: {e}. Retrying in {retry_delay}s…")
        except Exception as e:
            logger.error(f"[DataFeed] Unexpected error: {e}", exc_info=True)

        if self.is_running:
            await asyncio.sleep(retry_delay)
            retry_delay = min(retry_delay * 2, 60)

async def feed_health_monitor(self, notifier=None):
    """
    P0-2 FIX: Feed health monitor — runs as an independent async task.
    Checks every 60s whether a kline WebSocket frame was received within
    3× the candle interval. If not (i.e. feed is frozen), sends a
    Telegram alert and forces a reconnect by briefly stopping the feed loop.
    """
    interval_secs = {
        "1m": 60, "3m": 180, "5m": 300, "15m": 900, "1h": 3600
    }.get(self.interval, 900)
    stale_threshold = interval_secs * 3.0

    # Give the feed 60s to warm up before monitoring
    await asyncio.sleep(60)

    # FIX: Track boot time so we can detect "kline never arrived at all"
    _monitor_boot_ts = time.time()

    while self.is_running:
        await asyncio.sleep(60)
        if not self.is_running:
            break

        # FIX: If kline frames have NEVER arrived (e.g. geoblock), force
        # reconnect after 180s instead of skipping forever.  The old code
        # did `continue` here, meaning a fully dead kline stream would
        # never trigger a reconnect.
        if self._last_kline_frame_ts == 0.0:
            secs_since_boot = time.time() - _monitor_boot_ts
            if secs_since_boot > 180:
                alert_msg = (
                    f"■■ [DataFeed] FEED DEAD: zero kline frames received "
                    f"since boot ({secs_since_boot:.0f}s ago). Forcing reconnect."
                )
                logger.error(alert_msg)
                if notifier:
                    try:
                        await notifier.send_message(alert_msg)
                    except Exception:
                        pass
                self.state._reconnect_event.set()
                _monitor_boot_ts = time.time()  # reset to avoid spam
            continue

        age = time.time() - self._last_kline_frame_ts
        if age > stale_threshold:
            alert_msg = (
                f"■■ [DataFeed] FEED STALE: no kline frame received for {age:.0f}s "
```

```
                f"(threshold={stale_threshold:.0f}s). Forcing reconnect."
            )
            logger.error(alert_msg)
            if notifier:
                try:
                    await notifier.send_message(alert_msg)
                except Exception:
                    pass
            self.state._reconnect_event.set()
            logger.info("[DataFeed] Feed health monitor forcing WS reconnect.")

        # PHASE-2.2: aggTrade stream watchdog (count-delta based)
        # The multiplexed WebSocket can stay "connected" via kline frames
        # while the aggTrade sub-stream is silently dead. Checking only
        # _last_trade_ts misses this because the timestamp never advances
        # but ws.recv() never times out (klines keep the connection alive).
        # Solution: snapshot msg count each 60s cycle. If count is identical
        # two cycles in a row the sub-stream is genuinely dead → reconnect.
        # FIX: Initialize _prev_count to 0 (not -1) so the very first
        # measurement cycle can detect a dead stream without needing a
        # warmup cycle.
        _prev_count  = getattr(self, "_aggtrade_watchdog_prev_count", 0)
        _curr_count  = getattr(self.state, "_aggtrade_msg_count", 0)
        now          = time.time()
        elapsed      = now - getattr(self.state, "_aggtrade_count_reset_ts", now)

        if elapsed >= 60.0:
            msgs_per_min = int(_curr_count / max(elapsed, 1) * 60)
            logger.info(f"[DataFeed] aggTrade throughput: {msgs_per_min} msgs/min (Δ={_curr_count - _prev_count})")
            # Reset counters
            self._aggtrade_watchdog_prev_count = _curr_count
            self.state._aggtrade_msg_count     = 0
            self.state._aggtrade_count_reset_ts = now

            if msgs_per_min == 0 and _prev_count >= 0:
                # Zero throughput AND we've had at least one prior measurement
                trade_age = time.time() - getattr(self.state, "_last_trade_ts", 0)
                logger.warning(
                    f"[DataFeed] ■■ aggTrade sub-stream DEAD — 0 msgs/min, "
                    f"last trade {trade_age:.0f}s ago. Forcing WebSocket reconnect."
                )
                if notifier and trade_age > 75:
                    try:
                        await notifier.send_message(
                            f"■■ aggTrade stream dead ({trade_age:.0f}s). "
                            "Forcing reconnect — CVD/tape will be briefly unreliable."
                        )
                    except Exception:
                        pass
                self.state._reconnect_event.set()
                logger.info("[DataFeed] aggTrade watchdog: _reconnect_event set.")



def stop(self):
    logger.info("[DataFeed] Stop requested.")
    self.is_running = False

async def run_user_data_stream(self, api_key: str, on_fill_callback) -> None:
    """
    Binance USDM Futures user data stream for real-time fill detection.
    Subscribes to ORDER_TRADE_UPDATE events to detect position closes immediately.
    Run as a separate asyncio.create_task() alongside run().
    """
    import httpx as _httpx

    if "testnet" in self.rest_url:
        listen_key_url = f"{self.rest_url}/fapi/v1/listenKey"
        ws_base = "wss://stream.binancefuture.com"
    else:
        listen_key_url = "https://fapi.binance.com/fapi/v1/listenKey"
        ws_base = "wss://fstream.binance.com"

    headers = {"X-MBX-APIKEY": api_key}

    while self.is_running:
        try:
            # Create listenKey
```

```
    async with _httpx.AsyncClient(timeout=10.0) as client:
        resp = await client.post(listen_key_url, headers=headers)
        resp.raise_for_status()
        listen_key = resp.json()["listenKey"]

    logger.info(f"[UserDataStream] listenKey created ✓")

    # Keepalive — Binance expires listenKey after 60 min without ping
    async def _keepalive():
        while self.is_running:
            await asyncio.sleep(29 * 60)
            try:
                async with _httpx.AsyncClient(timeout=5.0) as c:
                    await c.put(
                        listen_key_url, headers=headers,
                        params={"listenKey": listen_key}
                    )
                logger.info("[UserDataStream] listenKey keepalive ✓")
            except Exception as e:
                logger.warning(f"[UserDataStream] Keepalive failed: {e}")

    keepalive_task = asyncio.create_task(_keepalive())

    async with websockets.connect(
        f"{ws_base}/ws/{listen_key}",
        ping_interval=20, ping_timeout=30
    ) as ws:
        logger.info("[UserDataStream] Connected ✓")
        while self.is_running:
            try:
                raw = await asyncio.wait_for(ws.recv(), timeout=60)
                event = json.loads(raw)
                if event.get("e") == "ORDER_TRADE_UPDATE":
                    order = event.get("o", {})
                    # Filled + reduceOnly = position exit
                    if order.get("X") == "FILLED" and order.get("R"):
                        pnl = float(order.get("rp", 0.0))
                        logger.info(
                            f"[UserDataStream] Fill: orderId={order.get('i')} "
                            f"PnL=${pnl:.2f}"
                        )
                        await on_fill_callback(pnl)
            except asyncio.TimeoutError:
                logger.warning("[UserDataStream] recv timeout — reconnecting")
                break

    keepalive_task.cancel()

except Exception as e:
    logger.warning(f"[UserDataStream] Error: {e} — retry in 10s")
    await asyncio.sleep(10)
```

■ derivatives_context.py

```
import httpx
import asyncio
import logging
import time

logger = logging.getLogger(__name__)


class DerivativesContext:
    """
    Fetches institutional-grade signals from free public APIs.
    All four sources are completely free — no API key required.
    Run once per candle close, cache for 5 minutes.
    """

    def __init__(self, symbol: str = "BTCUSDT"):
        self.symbol = symbol
        self._cache: dict = {}
        self._cache_ts: dict = {}
        self.CACHE_TTL = 300  # 5 minutes

    async def _fetch(self, key: str, url: str, params: dict = None) -> dict:
        now = time.time()
        if key in self._cache and (now - self._cache_ts.get(key, 0)) < self.CACHE_TTL:
            return self._cache[key]
        try:
            async with httpx.AsyncClient(timeout=8.0) as client:
                resp = await client.get(url, params=params)
                resp.raise_for_status()
                data = resp.json()
            self._cache[key] = data
            self._cache_ts[key] = now
            return data
        except Exception as e:
            logger.warning(f"[Derivatives] {key} fetch failed: {e}")
            return {}

    async def get_open_interest(self) -> dict:
        """
        Binance USDM REST — completely free.
        OI rising + price flat = position building (coiled spring).
        OI collapsing = deleveraging, do not enter new positions.
        """
        current = await self._fetch(
            "oi_current",
            "https://fapi.binance.com/fapi/v1/openInterest",
            {"symbol": self.symbol}
        )
        history = await self._fetch(
            "oi_history",
            "https://fapi.binance.com/futures/data/openInterestHist",
            {"symbol": self.symbol, "period": "15m", "limit": 30}
        )

        current_oi = float(current.get("openInterest", 0))

        oi_change_pct = 0.0
        oi_momentum_1h = 0.0

        if history and len(history) >= 4:
            prev_oi = float(history[-2].get("sumOpenInterest", current_oi))
            oi_change_pct = (current_oi - prev_oi) / max(prev_oi, 1) * 100
            oi_1h_ago = float(history[-4].get("sumOpenInterest", current_oi))
            oi_momentum_1h = (current_oi - oi_1h_ago) / max(oi_1h_ago, 1) * 100

        return {
            "current_oi": current_oi,
            "oi_change_pct_15m": oi_change_pct,
            "oi_momentum_1h": oi_momentum_1h,
            "oi_collapsing": oi_momentum_1h < -2.0,
        }

    async def get_top_trader_positioning(self) -> dict:
        """
        Binance publishes top trader (whale) long/short ratio — free.
        When top traders are >70% long = crowded = CONTRARIAN bearish.
```

```
    When top traders are >70% short = crowded short = squeeze risk.
    This is the single most powerful free contrarian signal available.
    """
    data = await self._fetch(
        "top_trader_ls",
        "https://fapi.binance.com/futures/data/topLongShortPositionRatio",
        {"symbol": self.symbol, "period": "15m", "limit": 10}
    )
    if not data:
        return {"top_long_pct": 50.0, "crowd_signal": "NEUTRAL"}

    latest = data[-1]
    long_pct = float(latest.get("longAccount", 0.5)) * 100
    short_pct = 100 - long_pct

    if long_pct > 70:
        crowd_signal = "CROWDED_LONG"    # contrarian bearish
    elif short_pct > 70:
        crowd_signal = "CROWDED_SHORT"   # contrarian bullish squeeze risk
    else:
        crowd_signal = "NEUTRAL"

    return {
        "top_long_pct": long_pct,
        "top_short_pct": short_pct,
        "crowd_signal": crowd_signal,
    }

async def get_options_context(self) -> dict:
    """
    Deribit options API — completely free, no account needed.
    Put/call ratio > 1.2 = institutions hedging downside = FEAR signal.
    This is what every hedge fund uses for directional bias.
    Your bot has ZERO options awareness right now.
    """
    try:
        data = await self._fetch(
            "deribit_options",
            "https://www.deribit.com/api/v2/public/get_book_summary_by_currency",
            {"currency": "BTC", "kind": "option"}
        )
        instruments = data.get("result", [])
        if not instruments:
            return {"put_call_ratio": 1.0, "options_signal": "NEUTRAL"}

        total_put_oi = sum(
            float(i.get("open_interest", 0))
            for i in instruments if "-P" in i.get("instrument_name", "")
        )
        total_call_oi = sum(
            float(i.get("open_interest", 0))
            for i in instruments if "-C" in i.get("instrument_name", "")
        )

        put_call_ratio = total_put_oi / max(total_call_oi, 1)

        if put_call_ratio > 1.2:
            options_signal = "FEAR"       # institutions buying downside protection
        elif put_call_ratio < 0.8:
            options_signal = "GREED"      # no hedging = complacent = contrarian caution
        else:
            options_signal = "NEUTRAL"

        return {
            "put_call_ratio": round(put_call_ratio, 3),
            "options_signal": options_signal,
        }
    except Exception as e:
        return {"put_call_ratio": 1.0, "options_signal": "NEUTRAL", "error": str(e)}

async def get_taker_flow(self) -> dict:
    """
    Binance taker buy/sell ratio — free.
    Taker imbalance > 0.3 with BUY signal = aggressive buyers confirming.
    Taker imbalance < -0.3 with SELL signal = aggressive sellers confirming.
    """
    data = await self._fetch(
        "taker_flow",
```

```
        "https://fapi.binance.com/futures/data/takerlongshortRatio",
        {"symbol": self.symbol, "period": "5m", "limit": 12}
    )
    if not data:
        return {"taker_imbalance": 0.0}

    latest = data[-1]
    buy_vol = float(latest.get("buyVol", 0))
    sell_vol = float(latest.get("sellVol", 0))
    total = buy_vol + sell_vol

    return {
        "taker_buy_ratio": buy_vol / max(total, 1),
        "taker_sell_ratio": sell_vol / max(total, 1),
        "taker_imbalance": (buy_vol - sell_vol) / max(total, 1),
    }

async def get_full_context(self) -> dict:
    """Fetch all four signals concurrently. Total latency = slowest single call."""
    results = await asyncio.gather(
        self.get_open_interest(),
        self.get_top_trader_positioning(),
        self.get_options_context(),
        self.get_taker_flow(),
        return_exceptions=True
    )
    return {
        "open_interest": results[0] if not isinstance(results[0], Exception) else {},
        "top_traders":   results[1] if not isinstance(results[1], Exception) else {},
        "options":       results[2] if not isinstance(results[2], Exception) else {},
        "taker_flow":    results[3] if not isinstance(results[3], Exception) else {},
    }
```

heartbeat.py

```
"""
bot/heartbeat.py
================
Writes the bot's live status to Firebase Firestore every 10 seconds.
The frontend dashboard listens to this document via onSnapshot to show
a real-time "VERIFIED ONLINE" indicator.

Required environment variable:
    FIREBASE_ADMIN_CREDENTIALS — the full JSON string of a Firebase
    service account key (download from Firebase Console →
    Project Settings → Service Accounts → Generate New Private Key).

If the variable is missing the heartbeat is silently skipped; the bot
continues to trade normally.
"""

import os
import json
import logging
import asyncio
import time as _time
from typing import Optional, Any

logger = logging.getLogger(__name__)

# Module-level Firestore client (initialised once)
_db: Optional[Any] = None

_LOCAL_STATS_PATH = "/tmp/quad_bot_stats.json"
_LOCAL_TRADES_PATH = "/tmp/quad_bot_trades.jsonl"
_write_buffer: list = []
_consecutive_failures: int = 0
_use_local_fallback: bool = False
MAX_BUFFER_SIZE = 500


def _buffered_append(entry: dict) -> None:
    """Append to write buffer with a size cap — drop oldest on overflow."""
    global _write_buffer
    if len(_write_buffer) >= MAX_BUFFER_SIZE:
        _write_buffer.pop(0)
    _write_buffer.append(entry)


def _normalize_pem(raw_key: str) -> str:
    """
    Normalize a PEM private key string from any storage encoding to a valid PEM.
    Includes handling for double-escaped newlines (\\\\n) and re-wrapping long
    base64 strings to 64 chars to avoid InvalidPadding errors in the cryptography lib.
    """
    if not raw_key: return ""

    # Step 1: Broad cleaning of escaping and quoting
    # Handle literal backslash-n, literal double-backslash-n, and true newlines
    key = str(raw_key).replace("\\\\n", "\n").replace("\\n", "\n")
    key = key.replace("\\r\\n", "\n").replace("\r\n", "\n").strip()
    key = key.strip('"').strip("'")

    # Step 2: Extract core body and headers
    lines = [l.strip() for l in key.splitlines() if l.strip()]
    if not lines: return ""

    header, footer = "", ""
    body_parts = []

    for l in lines:
        if "BEGIN" in l:
            header = l
        elif "END" in l:
            footer = l
        else:
            # [STRICTER] Base64 body should ONLY contain: A-Z, a-z, 0-9, +, /, and =
            # Any other characters (like backslashes or spaces) are artifacts.
            clean_part = "".join([c for c in l if c in "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+
            if clean_part:
                body_parts.append(clean_part)
```

```
    if not header or not footer:
        return key

    # Step 3: Flatten body and re-wrap strictly at 64 chars
    full_body = "".join(body_parts).rstrip("=")

    # 1.6 FIX: Validate key body length before padding.
    # Firebase RSA-2048 private key body = ~1700 base64 chars.
    # A body under 200 chars is almost certainly truncated by the secret manager.
    body_len = len(full_body)
    if body_len < 200:
        logger.error(
            f"[Heartbeat] Private key body is only {body_len} chars — "
            "likely truncated by Railway/secret manager character limit. "
            "Firebase RSA keys require ~1700 base64 chars. "
            "Paste the FULL JSON file content as the env var value."
        )
        return ""  # Return empty; caller will skip Firebase init cleanly

    remainder = len(full_body) % 4
    if remainder == 2:
        full_body += "=="
    elif remainder == 3:
        full_body += "="
    elif remainder == 1:
        # remainder=1 is structurally invalid in base64 — key is definitely truncated.
        # Padding with '===' would produce a structurally valid but cryptographically
        # wrong key, hiding the real error. Fail loudly instead.
        logger.error(
            "[Heartbeat] Base64 body has remainder=1 — key is definitely truncated. "
            "Do NOT try to pad it. Store the COMPLETE JSON key as the env var."
        )
        return ""

    wrapped_body = [full_body[i:i+64] for i in range(0, len(full_body), 64)]

    final_pem = "\n".join([header] + wrapped_body + [footer])
    return final_pem



def _validate_pem(key: str) -> bool:
    """
    Quick structural validation of a PEM private key without full cryptographic
    loading. Checks that the key has a BEGIN header, END footer, and at least
    one line of base64-encoded content between them.
    Returns True if the key looks valid, False otherwise.
    """
    lines = [l.strip() for l in key.splitlines() if l.strip()]
    if len(lines) < 3:
        return False
    has_header = any("BEGIN" in l for l in lines)
    has_footer = any("END" in l for l in lines)
    has_body   = len(lines) >= 3  # at least header + 1 body line + footer
    return has_header and has_footer and has_body


def init_firebase() -> Optional[Any]:
    """
    Initialise Firebase Admin SDK.
    Returns a Firestore client on success, None on any failure.

    Credential source priority:
    1. FIREBASE_ADMIN_CREDENTIALS — full JSON string of service account key
    2. FIREBASE_CREDENTIALS       — legacy alias (same format)
    """
    global _db
    cred_json = (
        os.environ.get("FIREBASE_ADMIN_CREDENTIALS", "").strip() or
        os.environ.get("FIREBASE_CREDENTIALS", "").strip()
    )
    if not cred_json:
        logger.warning(
            "[Heartbeat] FIREBASE_ADMIN_CREDENTIALS not set — "
            "bot↔dashboard live sync is disabled."
        )
        return None
    try:
```

```
        import firebase_admin
        from firebase_admin import credentials, firestore as fs

        cred_dict = json.loads(cred_json)

        # ■■ Fix PEM newlines (the root cause of 'InvalidPadding') ■■■■■■■■■■
        if "private_key" in cred_dict:
            raw = cred_dict["private_key"]
            normalized = _normalize_pem(raw)
            if not _validate_pem(normalized):
                logger.error(
                    "[Heartbeat] Firebase private_key PEM is malformed after normalization.\n"
                    "  → Re-export the service account JSON from Firebase Console and paste\n"
                    "    the raw file contents (not the escaped string) into FIREBASE_ADMIN_CREDENTIALS."
                )
                return None
            cred_dict["private_key"] = normalized

        if not firebase_admin._apps:
            cred = credentials.Certificate(cred_dict)
            firebase_admin.initialize_app(cred)

        _db = fs.client()
        logger.info("[Heartbeat] Firebase connected ✓ — dashboard sync active.")
        return _db

    except json.JSONDecodeError:
        logger.error(
            "[Heartbeat] FIREBASE_ADMIN_CREDENTIALS is not valid JSON. "
            "Ensure the entire service account JSON is set as a single-line env var."
        )
    except Exception as e:
        logger.error(
            f"[Heartbeat] Firebase init failed: {e}\n"
            "  → Fix: Re-export the service account key from Firebase Console → "
            "Project Settings → Service Accounts → Generate New Private Key.\n"
            "  → Then set FIREBASE_ADMIN_CREDENTIALS to the full raw JSON content."
        )
    return None

import threading
import queue

class FirestoreLogHandler(logging.Handler):
    """
    A custom logging handler that forwards log records to the 'botLogs'
    Firestore collection asynchronously using a background thread.
    """
    def __init__(self):
        super().__init__()
        self.log_queue = queue.Queue(maxsize=500)
        self.worker_thread = threading.Thread(target=self._log_worker, daemon=True)
        self.worker_thread.start()

    def _log_worker(self):
        while True:
            try:
                record = self.log_queue.get()
                if record is None:
                    break

                # Wait for Firebase to be initialised if it isn't yet (race condition fix)
                if _db is None:
                    import time
                    import queue
                    time.sleep(1)
                    try:
                        self.log_queue.put_nowait(record)
                    except queue.Full:
                        pass
                    self.log_queue.task_done()
                    continue

                from firebase_admin import firestore as fs
                payload = {
                    "level": record.levelname,
                    "message": self.format(record),
                    "timestamp": fs.SERVER_TIMESTAMP,
```

```
                    "ts_ms": int(record.created * 1000)
                }
                _db.collection("botLogs").add(payload)
            except Exception:
                # Silently catch errors so logs do not crash the worker thread
                pass
            finally:
                self.log_queue.task_done()

    def emit(self, record):
        # FIXED: Only forward ERROR+ to Firestore to stay under 20K/day quota.
        # WARNING and below stays in Railway/local logs only.
        # Errors (SL failures, order rejections, Firebase outages) still reach Firestore.
        if record.levelno < logging.ERROR:
            return
        if record.name.startswith("bot.") or record.name == "__main__":
            try:
                self.log_queue.put_nowait(record)
            except queue.Full:
                pass


def get_db() -> Optional[Any]:
    """Return the initialised Firestore client (or None if not available)."""
    return _db


async def run_heartbeat(stats: dict) -> None:
    """
    Continuously writes bot status to Firestore every 60 s.
    `stats` is a shared dict mutated by the main execution loop.
    Gracefully exits if Firebase is not initialised.

    If Firebase was not available at startup (e.g. bad credentials), this
    function will retry initialisation every 5 minutes so a credential fix
    at runtime is picked up automatically without restarting the bot.

    PHASE-6.1: Batches writes to reduce Firestore usage. On 429 (quota exceeded),
    uses exponential backoff and falls back to local JSON when Firebase is
    consistently unavailable.
    """
    global _db, _write_buffer, _consecutive_failures, _use_local_fallback

    from firebase_admin import firestore as fs

    RETRY_INTERVAL   = 300   # seconds between init retries when _db is None
    WRITE_INTERVAL   = 60    # seconds between heartbeat writes
    BATCH_FLUSH_SECS = 300   # flush buffer to local JSON every 5 minutes
    retry_countdown  = 0
    last_batch_flush = 0.0
    backoff = 0

    while True:
        try:
            # ■■ Retry init if Firebase not yet connected ■■■■■■■■■■■■■■■■■■■■
            if _db is None:
                if retry_countdown <= 0:
                    logger.info("[Heartbeat] Retrying Firebase initialisation…")
                    init_firebase()
                    retry_countdown = RETRY_INTERVAL
                else:
                    retry_countdown -= WRITE_INTERVAL
                await asyncio.sleep(WRITE_INTERVAL)
                continue

            doc_ref = _db.collection("botStatus").document("live")
            payload = {
                "isRunning":       True,
                "lastHeartbeat":   fs.SERVER_TIMESTAMP,
                "symbol":          stats.get("symbol",         "UNKNOWN"),
                "mode":            stats.get("mode",           "DRY-RUN"),
                "environment":     stats.get("environment",    "TESTNET"),
                "activePositions": 1 if stats.get("active_position") else 0,
                "totalTrades":     stats.get("total_trades",  0),
                "lastSignal":      stats.get("last_signal",   "WAIT"),
                "exchange":        stats.get("exchange",       "coinbase").lower(),
                "lastUlis":        stats.get("last_ulis",      "—"),
                "sessionPnl":      stats.get("session_pnl",   0.0),
```

```
    "dailyPnl":        stats.get("daily_pnl",     0.0),
    "gateStats":       stats.get("gate_stats",    {}),
}

if _use_local_fallback:
    # BUG-6 FIX: Replace fs.SERVER_TIMESTAMP with numeric timestamp for JSON serialization
    status_doc = dict(payload)
    status_doc["lastHeartbeat"] = _time.time()
    status_doc["timestamp"] = _time.time()
    _buffered_append({"type": "status", **status_doc, "ts": _time.time()})
else:
    try:
        await asyncio.to_thread(doc_ref.set, payload)
        _consecutive_failures = 0
        if backoff > 0:
            logger.info(f"[Heartbeat] Firebase recovered — backoff reset.")
            backoff = 0
    except Exception as e:
        _consecutive_failures += 1
        logger.warning(f"[Heartbeat] Firebase write error: {e}")
        if _consecutive_failures >= 3:
            _use_local_fallback = True
            logger.warning("[Heartbeat] 3 consecutive failures — switching to local JSON fallback.")
            # FIXED: Alert operator that dashboard is dark
            _tg_token = os.environ.get("TELEGRAM_BOT_TOKEN", "")
            _tg_chat  = os.environ.get("TELEGRAM_CHAT_ID", "")
            if _tg_token and _tg_chat:
                try:
                    import httpx as _httpx
                    asyncio.create_task(
                        _httpx.AsyncClient().post(
                            f"https://api.telegram.org/bot{_tg_token}/sendMessage",
                            json={
                                "chat_id": _tg_chat,
                                "text": (
                                    "■■ Quad-Desk: Firebase is DARK\n"
                                    "3 consecutive write failures.\n"
                                    "Trade data → local /tmp fallback only.\n"
                                    "Check Firestore quota or credentials."
                                )
                            },
                            timeout=5.0
                        )
                    )
                except Exception:
                    pass
        else:
            backoff = min(backoff * 2 + 30, 300)
            logger.info(f"[Heartbeat] Exponential backoff: {backoff}s")

# 5.2: Equity curve time-series point (every heartbeat cycle)
equity_doc = {
    "equity":     stats.get("account_equity", 0.0),
    "sessionPnl": stats.get("session_pnl",    0.0),
    "dailyPnl":   stats.get("daily_pnl",      0.0),
    "timestamp":  fs.SERVER_TIMESTAMP,
}
if _use_local_fallback:
    # BUG-6 FIX: Use numeric timestamp for JSON serialization
    equity_doc["timestamp"] = _time.time()
    _buffered_append({"type": "equity", **equity_doc, "ts": _time.time()})
else:
    # P2-2 FIX: Day-bucketed documents instead of equity_ref.add() per minute.
    # Old: 1 new Firestore doc/minute = 86,400 docs after 60 days.
    # New: 1 doc/day with arrayUnion; merge=True creates doc if missing.
    try:
        from datetime import datetime
        _day_key = datetime.utcnow().strftime("%Y-%m-%d")
        _equity_point = {k: v for k, v in equity_doc.items() if k != "timestamp"}
        _equity_point["ts"] = _time.time()
        _equity_day_ref = _db.collection("equityCurve").document(_day_key)
        await asyncio.to_thread(
            _equity_day_ref.set,
            {"points": fs.ArrayUnion([_equity_point]), "date": _day_key},
            True,
        )
    except Exception as e:
        _consecutive_failures += 1
```

```
                    logger.warning(f"[Heartbeat] Equity write error: {e}")

            # Flush buffer to local JSON every BATCH_FLUSH_SECS
            now = _time.time()
            if _use_local_fallback and _write_buffer and (now - last_batch_flush) >= BATCH_FLUSH_SECS:
                _flush_local_buffer()
                last_batch_flush = now

        except asyncio.CancelledError:
            break
        except Exception as e:
            logger.warning(f"[Heartbeat] Firebase sync error (will retry): {e}")

        try:
            await asyncio.sleep(WRITE_INTERVAL)
        except asyncio.CancelledError:
            break


def _flush_local_buffer():
    """Write buffered entries to local JSON files."""
    global _write_buffer
    if not _write_buffer:
        return
    count = len(_write_buffer)
    try:
        with open(_LOCAL_STATS_PATH, "a") as f:
            for entry in _write_buffer:
                f.write(json.dumps(entry) + "\n")
        _write_buffer.clear()
        logger.info(f"[Heartbeat] Flushed {count} entries to {_LOCAL_STATS_PATH}")
    except Exception as e:
        logger.warning(f"[Heartbeat] Local JSON write failed: {e}")



async def write_offline(stats: dict) -> None:
    """
    Writes a final 'bot stopped' document to Firestore on clean shutdown.
    Call this from the main finally block.
    """
    global _write_buffer
    if _use_local_fallback and _write_buffer:
        _flush_local_buffer()

    if _db is None:
        return

    try:
        from firebase_admin import firestore as fs
        doc_ref = _db.collection("botStatus").document("live")
        await asyncio.to_thread(doc_ref.set, {
            "isRunning":       False,
            "lastHeartbeat":   fs.SERVER_TIMESTAMP,
            "symbol":          stats.get("symbol",       "UNKNOWN"),
            "mode":            stats.get("mode",         "DRY-RUN"),
            "environment":     stats.get("environment",  "TESTNET"),
            "activePositions": 0,
            "totalTrades":     stats.get("total_trades", 0),
            "lastSignal":      "BOT_STOPPED",
            "exchange":        stats.get("exchange",      "coinbase").lower(),
            "lastUlis":        "—",
        })
        logger.info("[Heartbeat] Offline status written to Firebase ✓")
    except Exception as e:
        logger.error(f"[Heartbeat] Offline write failed: {e}")
```

■ ulis_engine.py

```
"""
ULIS/ALDE Hybrid Verdict Engine — Python Port
==============================================
This is a direct translation of the computeAIVerdict() function from ULISView.tsx.
It takes the same live market metrics computed by QuantEngine and produces an
8-type verdict that acts as Stage 7 (final confirmation gate) before order execution.

Verdict types:
  STRONG_LONG    — Full ALDE+ULIS bullish alignment (6-condition AND gate)
  LONG           — Moderate bullish bias
  SHORT          — Moderate bearish bias
  STRONG_SHORT   — Full ALDE+ULIS bearish alignment (6-condition AND gate)
  NEUTRAL        — No clear edge (let trade through unchanged)
  BREAKOUT_WATCH — Liquidity vacuum forming (let trade through unchanged)
  AVOID          — Cascade risk critical — veto all trades
  UNWIND         — Pre-cascade — veto all trades
"""

import logging
import math
from typing import Dict, Any, List, Optional, Tuple

logger = logging.getLogger(__name__)

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Helpers (mirror TS clamp / normalize / sigmoid)
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

def _clamp(v: float, lo: float, hi: float) -> float:
    return max(lo, min(hi, v))

def _normalize(v: float, lo: float, hi: float) -> float:
    return _clamp((v - lo) / (hi - lo), 0.0, 1.0) if hi > lo else 0.0

def _sigmoid(x: float) -> float:
    return 1.0 / (1.0 + math.exp(-x))


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Proxy Score Builder
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

def _build_scores_from_metrics(metrics: Dict[str, Any], bids: List, asks: List) -> Dict[str, Any]:
    """
    Derive the ULIS score components directly from QuantEngine metrics.
    Since we don't have the full frontend data pipeline (GLR feeds, ETF flows, etc.)
    we use principled proxies derived from metrics we DO have.

    v2 change: GLR score now uses FUNDING RATE instead of OFI.
    OFI was collinear with bayesianPosterior (both driven by order-flow) causing
    confidence inflation. Funding rate is orthogonal — it captures crowding/positioning
    pressure rather than instantaneous order imbalance.
    """
    ofi          = metrics.get("ofi", 0.0)
    rsi          = metrics.get("rsi", 50.0)
    z            = metrics.get("zScore", 0.0)
    bayes        = metrics.get("bayesianPosterior", 0.5)
    cvd          = metrics.get("cvd", 0.0)
    atr_pct      = metrics.get("atr_pct", 0.005)
    tape         = metrics.get("tapeSpeed", "NORMAL")
    dominant     = metrics.get("tapeDominant", "BALANCED")
    funding_rate = metrics.get("funding_rate", 0.0)   # NEW: independent signal

    # NLF Score proxy: bayesian posterior is the best single-metric bull field indicator
    nlf_score = _clamp(bayes, 0.0, 1.0)

    # GLR Score proxy: RSI momentum + FUNDING RATE (orthogonal to OFI/Bayes) + CVD direction
    # Funding rate interpretation:
    #   Positive (+) = longs paying shorts → crowded long → bearish pressure on price
    #   Negative (-) = shorts paying longs → crowded short → bullish squeeze risk
    # We INVERT and normalise so that negative funding → high (bullish) score.
    rsi_norm     = _normalize(rsi, 20.0, 80.0)
    # Funding range: typical Binance USDM range is ±0.05% per 8h (±0.0005)
    # Invert sign: negative funding is bullish→ maps to high norm value
    funding_norm = _normalize(-funding_rate, -0.0010, 0.0010)
    cvd_norm     = 0.6 if cvd > 0 else 0.4
```

```
    glr_score    = _clamp(rsi_norm * 0.40 + funding_norm * 0.35 + cvd_norm * 0.25, 0.0, 1.0)

    # Reflexivity Score proxy: high ATR% + screaming tape = high reflexivity (instability)
    atr_reflexivity  = _normalize(atr_pct, 0.003, 0.025)
    tape_reflexivity = 0.7 if tape == "SCREAMING" else 0.2
    reflexivity_score = _clamp(atr_reflexivity * 0.6 + tape_reflexivity * 0.4, 0.0, 1.0)

    # Fragility proxy: distance from Z-Score extremes indicates fragility
    z_fragility = _normalize(abs(z), 0.5, 3.0)
    fragility   = _clamp(z_fragility * 0.5 + atr_reflexivity * 0.5, 0.0, 1.0)

    # Visible liquidity proxy from OFI depth
    # OFI is now in (-1, +1) after the Three-Stage Pipeline (tanh output)
    visible_liquidity = _normalize(abs(ofi), 0.0, 0.6)

    # Latent liquidity: inverse of reflexivity (stable market = more resting liquidity)
    latent_liquidity = _clamp(1.0 - reflexivity_score, 0.0, 1.0)

    bayesian_baseline = bayes * 100.0
    glr_components = {
        "stablecoins": bayesian_baseline + (funding_rate * -10000.0),  # funding drag/boost in bps
        "etfFlows":    bayesian_baseline + (cvd / 5000.0),
    }

    return {
        "nlfScore":         nlf_score,
        "glrScore":         glr_score,
        "reflexivityScore": reflexivity_score,
        "fragility":        fragility,
        "visibleLiquidity": visible_liquidity,
        "latentLiquidity":  latent_liquidity,
        "glrComponents":    glr_components,
    }


# ═══════════════════════════════════════════════════════════════════════════════
# Sweep / BOS Counters (derived from candle history)
# ═══════════════════════════════════════════════════════════════════════════════

def _count_sweeps(candles: list, bids_dict: Dict, asks_dict: Dict) -> Tuple[int, List]:
    """
    Count liquidity sweeps (wick pierces a wall then closes back).
    Uses last 10 candles for recency.
    Returns (sweep_count, bos_list).
    """
    sweep_count = 0
    bos_list = []

    if len(candles) < 3 or not bids_dict or not asks_dict:
        return sweep_count, bos_list

    # P1-4 FIX: Filter to significant walls only (size > median * 5x) before computing sweeps.
    # Using top-of-book (best bid/ask) instead of actual walls massively inflates sweep count.
    bid_prices = sorted(bids_dict.keys(), reverse=True)
    ask_prices = sorted(asks_dict.keys())
    if not bid_prices or not ask_prices:
        return sweep_count, bos_list

    # Compute median sizes to identify significant walls
    bid_sizes = [bids_dict[p] for p in bid_prices]
    ask_sizes = [asks_dict[p] for p in ask_prices]
    import statistics
    median_bid = statistics.median(bid_sizes) if bid_sizes else 0
    median_ask = statistics.median(ask_sizes) if ask_sizes else 0
    wall_threshold_bid = median_bid * 5 if median_bid > 0 else 0
    wall_threshold_ask = median_ask * 5 if median_ask > 0 else 0

    # Filter to significant walls only
    significant_bids = {p: q for p, q in bids_dict.items() if q >= wall_threshold_bid}
    significant_asks = {p: q for p, q in asks_dict.items() if q >= wall_threshold_ask}

    sig_bid_prices = sorted(significant_bids.keys(), reverse=True)
    sig_ask_prices = sorted(significant_asks.keys())
    if not sig_bid_prices or not sig_ask_prices:
        return sweep_count, bos_list

    nearest_bid = sig_bid_prices[0]
    nearest_ask = sig_ask_prices[0]
```

```
    recent = list(candles)[-10:]
    for i in range(1, len(recent)):
        c = recent[i]
        p = recent[i - 1]
        if p["high"] > nearest_ask and c["close"] < nearest_ask:
            sweep_count += 1
        if p["low"] < nearest_bid and c["close"] > nearest_bid:
            sweep_count += 1
        # BOS: consecutive higher-lows or lower-highs = structure break
        if c["close"] > p["high"]:
            bos_list.append({"dir": "up", "price": c["close"]})
        elif c["close"] < p["low"]:
            bos_list.append({"dir": "down", "price": c["close"]})

    return sweep_count, bos_list


# ═══════════════════════════════════════════════════════════════════════════
# FVG Counter (Fair Value Gap) from candles
# ═══════════════════════════════════════════════════════════════════════════

def _count_fvgs(candles: list) -> int:
    """Count fair-value gaps in last 15 candles (3-candle gap pattern)."""
    count = 0
    recent = list(candles)[-15:]
    for i in range(2, len(recent)):
        prev2 = recent[i - 2]
        curr = recent[i]
        if curr["low"] > prev2["high"]:   # bullish FVG
            count += 1
        elif curr["high"] < prev2["low"]:  # bearish FVG
            count += 1
    return count


# ═══════════════════════════════════════════════════════════════════════════
# Order Book Helpers
# ═══════════════════════════════════════════════════════════════════════════

def _book_to_levels(book_dict: Dict[float, float]) -> List[Dict[str, float]]:
    """Convert {price: size} dict to list of {price, size} for TS parity."""
    return [{"price": p, "size": s, "classification": "WALL" if s > 0 else "NORMAL"}
            for p, s in book_dict.items()]


# ═══════════════════════════════════════════════════════════════════════════
# Main ULIS Verdict Function
# ═══════════════════════════════════════════════════════════════════════════

MIN_SWEEP_CONFIRMS = 2

def _sweep_micro_confirms(metrics: dict, sweep_direction: str) -> int:
    """FIX NEW-H1: 3rd signal now orthogonal funding rate."""
    confirms = 0
    rsi = metrics.get("rsi", 50.0)
    if sweep_direction == "bull" and rsi < 48:
        confirms += 1
    elif sweep_direction == "bear" and rsi > 52:
        confirms += 1
    ofi = metrics.get("ofi", 0.0)
    if sweep_direction == "bull" and ofi > 0.15:
        confirms += 1
    elif sweep_direction == "bear" and ofi < -0.15:
        confirms += 1
    funding = metrics.get("funding_rate", 0.0)
    if sweep_direction == "bull" and funding < -0.0005:
        confirms += 1
    elif sweep_direction == "bear" and funding > 0.0008:
        confirms += 1
    return confirms

def compute_ulis_verdict(
    metrics: Dict[str, Any],
    candles: list,
    bids_dict: Dict[float, float],
    asks_dict: Dict[float, float],
) -> Dict[str, Any]:
    """
```

```
Run the ALDE+ULIS hybrid verdict engine on live market metrics.

Returns:
    {
        verdict: str,           # STRONG_LONG | LONG | SHORT | STRONG_SHORT | NEUTRAL | AVOID | UNWIND | BREAKOUT_W
        confidence: float,      # 0–1
        cascade_risk: float,    # 0–1 sigmoid cascade index
        liquidity_vector: float,# -1 to +1  (+ = bullish pressure)
        regime_label: str,      # human-readable market state description
        should_trade: bool,     # False → veto the trade entirely
        confidence_boost: float,# positive = boost, negative = reduce
    }
"""
ofi           = metrics.get("ofi", 0.0)
z_score       = metrics.get("zScore", 0.0)
bayes         = metrics.get("bayesianPosterior", 0.5)
cvd           = metrics.get("cvd", 0.0)
price         = metrics.get("price", 0.0)

bids = _book_to_levels(bids_dict)
asks = _book_to_levels(asks_dict)

# ■■ Build proxy scores ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
scores = _build_scores_from_metrics(metrics, bids, asks)

# ■■ Sweep / BOS / FVG counters ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
sweep_count, bos_list = _count_sweeps(candles, bids_dict, asks_dict)
fvg_count = _count_fvgs(candles)

# ■■ Phase 1: Feature Fusion ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# OFI is now in (-1, +1) from tanh pipeline — no need to divide by 40
ofi_signal    = _clamp(ofi, -1.0, 1.0)
bayes_signal  = (bayes - 0.5) * 2.0         # -1 to +1
# leverage_proxy: OFI tanh output, scaled so |1| maps to high leverage
leverage_proxy = _normalize(abs(ofi), 0.0, 0.8)

total_bid = sum(b["size"] for b in bids)
total_ask = sum(a["size"] for a in asks)
total_depth = total_bid + total_ask or 1.0
bid_frac = total_bid / total_depth
ask_frac = total_ask / total_depth

sweep_bull = min(sweep_count / 6.0, 1.0) if sweep_count > 0 and bayes_signal > 0 else 0.0
sweep_bear = min(sweep_count / 6.0, 1.0) if sweep_count > 0 and bayes_signal < 0 else 0.0

# ■■ Phase 2: ALDE Market State Classifier ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
trend_agreement = (
    math.copysign(1, ofi_signal) == math.copysign(1, bayes_signal) and
    abs(ofi_signal) > 0.15 and abs(bayes_signal) > 0.15
)

if trend_agreement:
    cascade_risk = _sigmoid(
        scores["reflexivityScore"] * 1.5 +
        scores["fragility"] * 1.5 +
        leverage_proxy * 2.0 - 4.0
    )
else:
    cascade_risk = _sigmoid(
        scores["reflexivityScore"] * 3.0 +
        scores["fragility"] * 2.0 +
        leverage_proxy * 2.0 - 3.0
    )
if cascade_risk > 0.70 or scores["fragility"] > 0.65:
    market_state = "UNSTABLE"
elif abs(bayes_signal) > 0.25 and trend_agreement:
    market_state = "TRENDING"
else:
    market_state = "RANGE"

# ■■ Phase 3: Liquidity Pressure Scores ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
glr_stables = scores["glrComponents"].get("stablecoins", 55.0)
glr_etf     = scores["glrComponents"].get("etfFlows", 50.0)

ulp = _clamp(
    bid_frac * 0.30 +
    scores["nlfScore"] * scores["latentLiquidity"] * 0.30 +
    sweep_bull * 0.15 +
```

```
    (glr_stables / 100.0) * 0.25,
    0.0, 1.0
)
dlp = _clamp(
    ask_frac * 0.30 +
    (1.0 - scores["nlfScore"]) * scores["latentLiquidity"] * 0.30 +
    sweep_bear * 0.15 +
    (1.0 - glr_etf / 100.0) * 0.25,
    0.0, 1.0
)

# ■■ Phase 4: Net Liquidity Vector ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
liquidity_vector = _clamp(ulp - dlp, -1.0, 1.0)

# ■■ Phase 5: ALDE Confidence Score ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ilih_score = _normalize(total_depth, 1000.0, 80000.0)
alde_conf0 = (
    0.25 * ilih_score +
    0.22 * scores["glrScore"] +
    0.20 * scores["nlfScore"] +   # reduced from 0.25: bayes already priced via nlf_score
    0.20 * (1.0 - scores["reflexivityScore"]) +
    0.13 * (1.0 - scores["fragility"])
)
# 3.2 FIX: Removed bayes_boost — bayesianPosterior already feeds nlf_score (double-counting).
# Only CVD boost remains as it's an orthogonal (independent) signal.
cvd_boost = math.tanh(cvd / 50_000.0) * 0.06 * math.copysign(1, liquidity_vector or 1)
alde_confidence = _clamp(alde_conf0 + cvd_boost, 0.0, 1.0)

# ■■ Phase 6: Verdict AND-Gates ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
is_high_volatility   = scores["fragility"] > 0.55 or scores["reflexivityScore"] > 0.55
is_mean_reversion    = abs(z_score) > 1.8 and not trend_agreement
is_liquidity_vacuum  = scores["visibleLiquidity"] < 0.35 and fvg_count > 1
is_bos_confirmed     = len(bos_list) > 0
cascade_warning      = cascade_risk > 0.60

if cascade_risk > 0.65 and scores["reflexivityScore"] > 0.50:
    verdict = "AVOID"
    regime_label = "Cascade Risk Critical — Liquidity Singularity · STAND ASIDE"
elif cascade_warning and is_high_volatility and cascade_risk > 0.55:
    verdict = "UNWIND"
    regime_label = "Pre-Cascade: Volatility Expansion · Reduce Exposure Now"
elif market_state == "UNSTABLE" and not is_mean_reversion:
    verdict = "NEUTRAL"
    regime_label = "Market Unstable — Cascade Elevated · Wait for Stabilisation"
elif (
    liquidity_vector > 0.28 and
    scores["nlfScore"] > 0.58 and
    scores["glrScore"] > 0.50 and
    cascade_risk < 0.45 and
    alde_confidence > 0.68 and
    bayes > 0.58
):
    verdict = "STRONG_LONG"
    regime_label = "ALDE+ULIS Bullish Confluence · Full Alignment — High Probability Long"
elif liquidity_vector > 0.08 and alde_confidence > 0.52 and cascade_risk < 0.62:
    verdict = "LONG"
    regime_label = "Bullish Bias · Liquidity Vector Positive — Moderate ALDE Alignment"
elif (
    liquidity_vector < -0.28 and
    scores["nlfScore"] < 0.42 and
    scores["glrScore"] < 0.50 and
    cascade_risk < 0.45 and
    alde_confidence > 0.68 and
    bayes < 0.42
):
    verdict = "STRONG_SHORT"
    regime_label = "ALDE+ULIS Bearish Confluence · Full Alignment — High Probability Short"
elif liquidity_vector < -0.08 and alde_confidence > 0.52 and cascade_risk < 0.62:
    verdict = "SHORT"
    regime_label = "Bearish Bias · Liquidity Vector Negative — Moderate ALDE Alignment"
elif is_mean_reversion:
    verdict = "SHORT" if z_score > 0 else "LONG"
    regime_label = "Mean Reversion Play — Z-Score Extreme · Fade the Extension"
elif is_liquidity_vacuum and abs(liquidity_vector) > 0.12:
    verdict = "BREAKOUT_WATCH"
    regime_label = "Liquidity Vacuum · Breakout Conditions Forming"
else:
    verdict = "NEUTRAL"
```

```
    regime_label = "ALDE Equilibrium — Liquidity Vector Near Zero · No Actionable Edge"

# ■■ Determine trade gate ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
should_trade = verdict not in ("AVOID", "UNWIND")

# ■■ Confidence modifier ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
confidence_boost = 0.0
if verdict == "STRONG_LONG" or verdict == "STRONG_SHORT":
    confidence_boost = 0.08
elif verdict == "AVOID" or verdict == "UNWIND":
    confidence_boost = -1.0   # This ensures confidence drops below threshold
elif verdict == "NEUTRAL":
    confidence_boost = 0.0  # FIX-P3: NEUTRAL = no actionable edge → zero boost (was +0.03 which inflated every en
elif verdict == "BREAKOUT_WATCH":
    confidence_boost = -0.02  # Mild caution: breakout vacuum, not a directional veto

logger.info(
    f"[ULIS] verdict={verdict} | vector={liquidity_vector:.3f} | "
    f"cascade={cascade_risk:.2f} | conf={alde_confidence:.2%} | "
    f"state={market_state} | sweeps={sweep_count} | BOS={len(bos_list)}"
)

return {
    "verdict":          verdict,
    "confidence":       round(alde_confidence, 4),
    "cascade_risk":     round(cascade_risk, 4),
    "liquidity_vector": round(liquidity_vector, 4),
    "regime_label":     regime_label,
    "should_trade":     should_trade,
    "confidence_boost": confidence_boost,
    "ulp":              round(ulp, 4),
    "dlp":              round(dlp, 4),
    "market_state":     market_state,
    "sweep_count":      sweep_count,
}
```

■ executor.py

```
import logging
import time
from typing import Dict, Any, Optional
import ccxt.async_support as ccxt
from bot import heartbeat
from bot.notifier import TelegramNotifier

logger = logging.getLogger(__name__)


class TradingExecutor:
    """
    Handles risk-managed execution of trading signals via ccxt.

    Supports Coinbase Advanced Trade (spot) and Binance (legacy).
    In DRY_RUN mode the executor logs what it WOULD do without placing
    any real orders — safe for live observation/testing.

    Coinbase Advanced Trade uses RSA/EC private key authentication.
    Set COINBASE_API_KEY_NAME and COINBASE_PRIVATE_KEY in your .env.
    """

    def __init__(
        self,
        api_key: str,
        api_secret: str,
        testnet: bool = True,
        dry_run: bool = True,
        exchange_id: str = "binance",
        coinbase_key_name: str = "",
        coinbase_private_key: str = "",
        tg_token: str = "",
        tg_chat_id: str = "",
    ):
        self.testnet   = testnet
        self.exchange_id = exchange_id.lower()
        self.is_futures  = "binanceusdm" in self.exchange_id or "future" in self.exchange_id
        self.dry_run   = dry_run or not self._has_credentials(
            exchange_id, api_key, api_secret, coinbase_key_name, coinbase_private_key
        )

        self.notifier = TelegramNotifier(tg_token, tg_chat_id)

        if self.dry_run:
            logger.warning("[Executor] DRY-RUN mode: no real orders will be placed.")

        # ■■ Exchange initialisation ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        if self.exchange_id == "coinbase":
            self.exchange = self._init_coinbase(
                coinbase_key_name, coinbase_private_key
            )
        else:
            # Binance (legacy / fallback)
            self.exchange = self._init_binance(api_key, api_secret, testnet, self.exchange_id)

        self.active_position: Optional[Dict[str, Any]] = None
        self.pending_order: Optional[Dict[str, Any]] = None  # Track unfilled orders
        self.lock_expiry: float = 0.0
        self.last_panic_reason: str = ""
        self.failed_order_ts: float = 0.0  # Cooldown after live order failure (prevents -2015 spam)
        # P0-3 FIX: Dedicated flag set when emergency_flatten fails.
        # A boolean survives the heartbeat poll clearing active_position; a sentinel dict does not.
        self._flatten_failed: bool = False
        # P0-1 FIX: Lock to prevent TOCTOU race between heartbeat poll and main loop
        # both reading/writing active_position in overlapping async yield points.
        import asyncio as _asyncio_for_lock
        self._position_lock = _asyncio_for_lock.Lock()

        # PHASE-0.3: Exchange-specific fee table.
        # Fees were hardcoded as Binance USDM rates in 3 separate places.
        # Coinbase charges up to 1.2% taker — 24× higher than the old assumption.
        _EXCHANGE_FEES = {
            "coinbase":    {"taker": 0.012,  "maker": 0.006},
            "binance":     {"taker": 0.001,  "maker": 0.0002},
            "binanceusdm": {"taker": 0.0005, "maker": 0.0002},
            "kraken":      {"taker": 0.0026, "maker": 0.0016},
```

```
        "okx":         {"taker": 0.001,  "maker": 0.0008},
    }
    _fees = _EXCHANGE_FEES.get(self.exchange_id, {"taker": 0.001, "maker": 0.0002})
    self.TAKER_FEE = _fees["taker"]
    self.MAKER_FEE = _fees["maker"]
    logger.info(f"[Executor] Fee rates for {self.exchange_id}: taker={self.TAKER_FEE:.4%} maker={self.MAKER_FEE:.4%

# ------------------------------------------------------------------
# Static helpers
# ------------------------------------------------------------------
@staticmethod
def _has_credentials(exchange_id, api_key, api_secret, cb_name, cb_key) -> bool:
    if exchange_id.lower() == "coinbase":
        return bool(cb_name and cb_key)
    return bool(api_key and api_secret)

@staticmethod
def _init_coinbase(key_name: str, private_key: str) -> ccxt.Exchange:
    """
    Coinbase Advanced Trade uses EC/RSA private keys, not HMAC secrets.
    ccxt maps: apiKey → key_name, secret → private_key (PEM string).
    The private key may be stored with literal \\n — normalise to real newlines.
    """
    # Normalise escaped newlines from env vars
    normalised_key = private_key.replace("\\n", "\n").strip()

    exchange = ccxt.coinbase({
        "apiKey":          key_name,
        "secret":          normalised_key,
        "enableRateLimit": True,
        "options": {
            "defaultType":     "spot",
            # Hard-veto legacy V2 features that crash CDP keys
            "fetchCurrencies": False,
            "fetchMarkets":    True,  # Still need markets
        },
    })
    # Override the has['fetchCurrencies'] to force CCXT to skip the public currency fetch
    exchange.has['fetchCurrencies'] = False

    logger.info("[Executor] Coinbase Advanced Trade (V3) initialised.")
    return exchange

@staticmethod
def _init_binance(api_key: str, api_secret: str, testnet: bool, exchange_id: str = "binance") -> ccxt.Exchange:
    # ■■ Key Format Check (Ed25519 vs HMAC) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    # If the secret contains BEGIN header, it's an Ed25519/RSA key.
    is_ed25519 = str(api_secret).strip().startswith("-----BEGIN")
    logger.info(f"[Executor] Initialising Binance ({'Ed25519' if is_ed25519 else 'HMAC'})")

    # Determine if we should force Futures mode
    is_futures_id = "usdm" in exchange_id.lower() or "future" in exchange_id.lower()

    # [FIX] Use specialized ccxt.binanceusdm if targeting USDM futures.
    # This prevents CCXT's load_markets from hitting Spot/Margin SAPI endpoints
    # which can trigger -2015 errors on Futures-only API keys.
    exchange_class = ccxt.binanceusdm if is_futures_id else ccxt.binance

    exchange = exchange_class({
        "apiKey":          api_key,
        "secret":          api_secret,
        "enableRateLimit": True,
        "options": {
            "defaultType": "future" if (is_futures_id or testnet) else "spot",
            "adjustForTimeDifference": True,
            "recvWindow": 10000,
        },
    })

    if testnet:
        exchange.set_sandbox_mode(True)

    # Prevent CCXT from trying to load margin info during load_markets
    exchange.has['fetchMarginAllPairs'] = False
    exchange.has['fetchFundingHistory'] = False
    exchange.has['fetchCurrencies'] = False

    active_type = exchange.options.get("defaultType", "spot")
```

```
        exchange_name = getattr(exchange_class, "__name__", type(exchange).__name__)
        logger.info(f"[Executor] Binance ({'testnet' if testnet else 'live'}) initialised as {active_type.upper()} via
        return exchange

    # ------------------------------------------------------------------
    # Symbol translation helpers
    # ------------------------------------------------------------------
    def _get_ccxt_symbol(self, raw_symbol: str) -> str:
        """
        Convert a raw symbol (e.g. BTCUSDT) to ccxt unified format.
        Binance USDM futures : BTCUSDT  → BTC/USDT:USDT
        Spot / other         : BTCUSDT  → BTC/USDT
        Called by main.py to set futures leverage at startup.
        """
        symbol = raw_symbol.strip().upper()

        # Already unified (contains /)
        if "/" in symbol:
            if self.is_futures and ":" not in symbol:
                quote = symbol.split("/")[-1]
                return f"{symbol}:{quote}"
            return symbol

        # Dash format (BTC-USDT)
        if "-" in symbol:
            symbol = symbol.replace("-", "/")
            if self.is_futures and ":" not in symbol:
                quote = symbol.split("/")[-1]
                return f"{symbol}:{quote}"
            return symbol

        # Raw concatenated form (BTCUSDT, ETHUSDT, …)
        for quote in ("USDT", "USDC", "BUSD", "USD", "BTC", "ETH", "BNB"):
            if symbol.endswith(quote):
                base    = symbol[: -len(quote)]
                unified = f"{base}/{quote}"
                if self.is_futures:
                    return f"{unified}:{quote}"
                return unified

        # Fallback — return as-is
        return symbol

    def _to_exchange_symbol(self, symbol: str) -> str:
        """
        Translate a symbol to the exchange's native format for CCXT.
        Coinbase uses BTC/USD (ccxt unified) or BTC-USD (raw API).
        This method converts any format to ccxt unified: BTC/USDC
        """
        if self.exchange_id != "coinbase":
            return symbol

        # Normalize: ensure we return ccxt unified format (BTC/USDC, BTC/USD, etc.)
        symbol = symbol.strip().upper()

        # If already in BTC/USDC format, return as-is
        if "/" in symbol:
            return symbol

        # If in BTC-USDC format, convert to BTC/USDC
        if "-" in symbol:
            return symbol.replace("-", "/")

        # BTCUSDT style → BTC/USDT
        # Common quote currencies in order of descending length (avoid partial match)
        for quote in ("USDT", "USDC", "USD", "BTC", "ETH", "BNB"):
            if symbol.endswith(quote):
                base = symbol[: -len(quote)]
                return f"{base}/{quote}"

        # Fallback — return with / added if possible
        return symbol

    # ------------------------------------------------------------------
    # Lifecycle
    # ------------------------------------------------------------------
    async def initialize(self):
        """Load markets and confirm connectivity.
```

```
    Binance USDM 'markets not loaded' fix:
    - Retries load_markets() up to 3× with back-off on transient network errors.
    - If all retries fail, injects a minimal BTC/USDT:USDT futures market spec so
      amount_to_precision / price_to_precision calls don't crash on the next order.
    """
    if self.dry_run and not self.exchange.apiKey:
        logger.info("[Executor] Skipping market load in keyless dry-run mode.")
        return

    env  = "TESTNET" if self.testnet else "LIVE"
    exch = self.exchange_id.upper()

    # --- Retry loop: up to 3 attempts with exponential back-off ---
    last_exc: Optional[Exception] = None
    for attempt in range(3):
        try:
            await self.exchange.load_markets()
            logger.info(f"[Executor] Connected to {exch} {env} ✓ (attempt {attempt + 1})")

            # --- Boot Position Check (no reconciliation) ---
            # If an open position exists on the exchange, alert via Telegram
            # and continue trading normally. We do NOT reconcile because the
            # SL/TP would be guessed from an ATR proxy and could be dangerously wrong.
            # User commits to closing all trades before starting the bot.
            try:
                positions = await self.exchange.fetch_positions()
                open_found = False
                for pos in positions:
                    qty = float(pos.get("contracts", 0) or pos.get("positionAmt", 0))
                    if abs(qty) > 0.0001:
                        open_found = True
                        side = "buy" if qty > 0 else "sell"
                        entry_p = float(pos.get("entryPrice", 0))
                        warn_msg = (
                            f"■■ Quad-Desk BOOT WARNING\n"
                            f"Open position found: {side.upper()} {abs(qty)} {pos.get('symbol')} @ {entry_p:.2f}\n"
                            f"Bot is starting normally but has NO internal SL/TP for this position.\n"
                            f"Please verify on Binance — this position is NOT tracked by the bot."
                        )
                        logger.warning(f"[Executor] {warn_msg}")
                        if self.notifier:
                            await self.notifier.send_message(warn_msg)
                if not open_found:
                    logger.info("[Executor] Boot clean — no open positions found on exchange.")

            except Exception as e:
                logger.warning(f"[Executor] Could not check positions on boot: {e}")

            # Send startup status notification
            try:
                mode = "DRY-RUN ■" if self.dry_run else "LIVE ■"
                await self.notifier.send_message(
                    f"■ Quad-Desk {mode} STARTED\n"
                    f"Exchange: {self.exchange_id.upper()} | "
                    f"Taker fee: {self.TAKER_FEE:.3%}"
                )
            except Exception:
                pass

            return   # success — exit initialize
        except Exception as e:
            last_exc = e
            wait_s   = 2 ** attempt          # 1s, 2s, 4s
            logger.warning(
                f"[Executor] load_markets attempt {attempt + 1}/3 failed: {e}. "
                f"{'Retrying in ' + str(wait_s) + 's…' if attempt < 2 else 'All retries exhausted.'}"
            )
            if attempt < 2:
                import asyncio as _asyncio
                await _asyncio.sleep(wait_s)

    # All retries failed — inject minimal market fallback to prevent order failures
    logger.warning(
        f"[Executor] Could not load {exch} markets after 3 attempts ({last_exc}). "
        "Injecting minimal market fallback — bot will attempt to continue."
    )
    self._inject_fallback_markets()
def _inject_fallback_markets(self):
```

```
    """Injects minimal market data into CCXT to prevent crashes."""
    if not hasattr(self.exchange, 'markets') or self.exchange.markets is None:
        self.exchange.markets = {}
    if not hasattr(self.exchange, 'symbols') or self.exchange.symbols is None:
        self.exchange.symbols = []

    if self.exchange_id == "coinbase":
        self.exchange.markets['BTC/USDC'] = {
            'id': 'BTC-USDC', 'symbol': 'BTC/USDC', 'base': 'BTC', 'quote': 'USDC',
            'precision': {'amount': 0.00000001, 'price': 0.01},
            'limits': {'amount': {'min': 0.00001, 'max': 1000}, 'price': {'min': 0.01, 'max': 1000000}, 'cost': {'m
            'active': True, 'type': 'spot', 'spot': True, 'margin': False, 'contract': False
        }
        self.exchange.markets['BTC/USD'] = {
            'id': 'BTC-USD', 'symbol': 'BTC/USD', 'base': 'BTC', 'quote': 'USD',
            'precision': {'amount': 0.00000001, 'price': 0.01},
            'limits': {'amount': {'min': 0.00001, 'max': 1000}, 'price': {'min': 0.01, 'max': 1000000}, 'cost': {'m
            'active': True, 'type': 'spot', 'spot': True, 'margin': False, 'contract': False
        }
        if 'BTC/USDC' not in self.exchange.symbols: self.exchange.symbols.append('BTC/USDC')
        if 'BTC/USD' not in self.exchange.symbols: self.exchange.symbols.append('BTC/USD')

    elif self.is_futures:
        self.exchange.markets['BTC/USDT:USDT'] = {
            'id': 'BTCUSDT', 'symbol': 'BTC/USDT:USDT', 'base': 'BTC', 'quote': 'USDT', 'settle': 'USDT',
            'precision': {'amount': 0.001, 'price': 0.1},
            'limits': {'amount': {'min': 0.001, 'max': 1000}, 'price': {'min': 0.1, 'max': 1000000}},
            'active': True, 'type': 'future', 'spot': False, 'margin': False, 'contract': True
        }
        if 'BTC/USDT:USDT' not in self.exchange.symbols: self.exchange.symbols.append('BTC/USDT:USDT')

async def close(self):
    await self.exchange.close()

# ---------------------------------------------------------------------
# Firestore trade logger
# ---------------------------------------------------------------------
def _log_trade(self, symbol: str, side: str, verdict: str,
               entry: float, stop_loss: float, take_profit: float,
               ulis_verdict: str = "",
               metrics: dict = None,
               signal: dict = None):
    """Write a trade record to Firestore `botTrades` collection.

    5.6: Includes full signal attribution chain so post-trade analysis can
    identify which signals (regime, OFI, Z-score, RSI, Bayesian confidence)
    drove each entry decision.
    """
    db = heartbeat.get_db()
    if db is None:
        return
    try:
        from firebase_admin import firestore as fs
        m = metrics or {}
        s = signal or {}
        doc = {
            "symbol":         symbol,
            "side":           side,
            "verdict":        verdict,
            "entry_price":    entry,
            "stop_loss":      stop_loss,
            "take_profit":    take_profit,
            "timestamp":      fs.SERVER_TIMESTAMP,
            "mode":           "DRY-RUN" if self.dry_run else "LIVE",
            "exchange":       self.exchange_id,
            "ulis_verdict":   ulis_verdict,
            "ts_ms":          int(time.time() * 1000),
            # Signal attribution chain (5.6)
            "regime":         m.get("regime",            "UNKNOWN"),
            "strategy_type":  s.get("strategy_type",     "UNKNOWN"),
            "z_score":        round(m.get("zScore",       0.0), 4),
            "rsi":            round(m.get("rsi",          50.0), 2),
            "ofi_tanh":       round(m.get("ofi",          0.0), 4),  # tanh (-1,+1)
            "bayesian":       round(m.get("bayesianPosterior", 0.5), 4),
            "confidence":     round(float(s.get("confidence", 0.0)), 4),
            "atr_pct":        round(m.get("atr_pct",      0.0), 6),
            "skewness":       round(m.get("skewness",     0.0), 4),
        }
```

```
        res = db.collection("botTrades").add(doc)
        doc_id = res[1].id
        logger.info(f"[Executor] Trade logged to Firestore (ID: {doc_id}) ✓")
        return doc_id
    except Exception as e:
        logger.warning(f"[Executor] Failed to log trade to Firestore: {e}")
        return None

def _update_trade_exit(self, doc_id: str, exit_price: float, pnl: float):
    """Update an existing trade record with exit metadata."""
    if not doc_id:
        return
    db = heartbeat.get_db()
    if db is None:
        return
    try:
        from firebase_admin import firestore as fs
        result = "WIN" if pnl > 0 else "LOSS"
        db.collection("botTrades").document(doc_id).update({
            "exit_price": exit_price,
            "pnl":        pnl,
            "result":     result,
            "exit_ts":    fs.SERVER_TIMESTAMP,
            "exit_ts_ms": int(time.time() * 1000)
        })
        logger.info(f"[Executor] Trade {doc_id} exit updated in Firestore ✓")
    except Exception as e:
        logger.warning(f"[Executor] Failed to update trade exit for {doc_id}: {e}")

# ------------------------------------------------------------------
# Balance
# ------------------------------------------------------------------
async def get_usdt_balance(self, account_size: float = 100.0) -> float:
    """Return free stablecoin balance (USDC/USD/USDT) or simulated equity in dry-run."""
    if self.dry_run:
        return account_size
    try:
        bal = await self.exchange.fetch_balance()
        free = bal.get("free", {})
        total = float(free.get("USDT", 0.0)) + float(free.get("USD", 0.0)) + float(free.get("USDC", 0.0))
        if total <= 0:
            return account_size
        return total
    except Exception as e:
        logger.warning(f"[Executor] get_usdt_balance fallback: {e}")
        return account_size

async def get_btc_balance(self) -> float:
    """Return free BTC balance directly from exchange."""
    if self.dry_run: return 0.0
    try:
        bal = await self.exchange.fetch_balance()
        return float(bal.get("free", {}).get("BTC", 0.0))
    except Exception: return 0.0

async def get_total_equity(self, current_price: float, account_size: float = 100.0) -> float:
    """
    Calculate total account value: (Sum of Stables) + (BTC * Price).
    Ensures risk (position size) is based on entire portfolio, not just cash.
    """
    if self.dry_run: return account_size

    try:
        bal = await self.exchange.fetch_balance()
        free = bal.get("free", {})

        cash = float(free.get("USDT", 0.0)) + float(free.get("USD", 0.0)) + float(free.get("USDC", 0.0))
        crypto_val = float(free.get("BTC", 0.0)) * current_price

        total = cash + crypto_val
        if total <= 5.0: # If effectively zero, fall back
            return account_size

        logger.info(f"[Executor] Equity: Cash=${cash:.2f} + BTC_Val=${crypto_val:.2f} → Total=${total:.2f}")
        return total
    except Exception as e:
        err_msg = f"Failed to calculate total equity: {e}"
        logger.error(f"[Executor] {err_msg}")
```

```
        await self.notifier.send_error_alert(err_msg)
        return account_size

# ------------------------------------------------------------------
# Position sizing
# ------------------------------------------------------------------
def calculate_position_size(
    self,
    current_price: float,
    stop_loss: float,
    equity: float,
    max_risk_pct: float,
) -> float:
    """
    Fixed-fractional sizing: risk_usd / |entry − stop|
    risk_usd = equity × (max_risk_pct / 100)
    """
    risk_usd = equity * (max_risk_pct / 100.0)
    distance = abs(current_price - stop_loss)
    if distance <= 0 or current_price <= 0:
        return 0.0
    return risk_usd / distance

# ------------------------------------------------------------------
# Signal execution
# ------------------------------------------------------------------
async def execute_signal(
    self,
    symbol: str,
    current_price: float,
    signal: Dict[str, Any],
    max_risk_pct: float,
    account_size: float = 100.0,
    ulis_verdict: str = "",
):
    verdict     = signal.get("verdict", "WAIT")
    confidence  = float(signal.get("confidence", 0))
    stop_loss   = float(signal.get("stop_loss", 0))
    take_profit = float(signal.get("take_profit", 0))

    # Guard rails
    if "WAIT" in verdict:
        return

    if self.active_position is not None:
        logger.info("[Executor] Already in a position. Skipping new entry.")
        return

    # Check if previous order is still pending (unfilled)
    if self.pending_order is not None:
        # MED-5 FIX: Auto-expire stale pending_order to prevent permanent lock.
        if time.time() > self.pending_order.get("expires_at", 0):
            logger.warning(
                f"[Executor] Pending order {self.pending_order['id']} TTL expired — "
                "clearing stale state. Verify on exchange if order was filled."
            )
            self.pending_order = None
        else:
            logger.info(f"[Executor] Pending order {self.pending_order['id']} still unfilled. Rejecting new signal.
            return

    if stop_loss <= 0 or take_profit <= 0:
        logger.warning("[Executor] Invalid SL/TP. Aborting.")
        return

    side = "buy" if ("BUY" in verdict or "LONG" in verdict) else "sell"
    # AUDIT FIX #6: Use estimated fill price (bid + spread proxy) for SL check.
    # current_price is the bid. BUY orders fill at the ask (~bid + 2bp).
    # Without this, a SL set between bid and ask passes the check but
    # gets immediately hit at fill.
    fill_price_est = current_price * 1.0002 if side == "buy" else current_price * 0.9998
    if side == "buy" and stop_loss >= fill_price_est:
        logger.warning(f"[Executor] SL {stop_loss} ≥ est. fill {fill_price_est:.2f} for BUY. Aborting.")
        return
    if side == "sell" and stop_loss <= fill_price_est:
        logger.warning(f"[Executor] SL {stop_loss} ≤ est. fill {fill_price_est:.2f} for SELL. Aborting.")
        return
    # 1. Calculate Available Equity for accurate risk sizing.
```

```
# Futures: Use only available margin (Cash) to prevent oversized rejected orders.
# Spot: Use Total Equity (Cash + BTC) to size based on full portfolio.
if self.is_futures:
    equity = await self.get_usdt_balance(account_size)
else:
    equity = await self.get_total_equity(current_price, account_size)

if side == "buy":
    usdc_equity = equity
    if usdc_equity < 5:
        err = f"Insufficient USDC (${usdc_equity:.2f}) to open LONG. Min $5 required."
        logger.warning(f"[Executor] {err}")
        await self.notifier.send_error_alert(err)
        return
else:
    if self.is_futures:
        usdc_equity = equity
        if usdc_equity < 5:
            err = f"Insufficient USDC margin (${usdc_equity:.2f}) to open SHORT on Binance Futures. Min $5 requ
            logger.warning(f"[Executor] {err}")
            await self.notifier.send_error_alert(err)
            return
    else:
        # Spot: Need BTC in account to sell
        btc_bal = await self.get_btc_balance()
        if btc_bal <= 0.00001:
            err = f"Aborting SELL signal: No BTC balance available to sell on spot account."
            logger.warning(f"[Executor] {err}")
            await self.notifier.send_error_alert(err)
            return

raw_size = self.calculate_position_size(current_price, stop_loss, equity, max_risk_pct)
if raw_size <= 0.0:
    logger.warning("[Executor] Calculated position size is 0. Aborting.")
    return

# FIX 16: Minimum notional validation with operator alert
MIN_NOTIONAL_USDT = 105.0  # Binance BTCUSDT min $100 + 5% buffer
MIN_QTY_BTC = 0.001        # Binance BTCUSDT minimum lot size

if raw_size * current_price < MIN_NOTIONAL_USDT:
    needed_acct = (MIN_NOTIONAL_USDT * abs(current_price - stop_loss)) / (max_risk_pct / 100.0)
    logger.error(
        f"[Executor] Account ${equity:.0f} too small. "
        f"Risk-correct notional=${raw_size*current_price:.2f} < Binance min ${MIN_NOTIONAL_USDT}. "
        f"Needs ~${needed_acct:.0f} account. ABORTING order. "
        "Set BOT_ACCOUNT_SIZE=1500 in Railway to resolve."
    )
    if self.notifier:
        await self.notifier.send_message(
            f"■■ Account too small for execution. "
            f"Current: ${equity:.0f}. Required: ~${needed_acct:.0f}."
        )
    return  # Abort — do not scale up

# Translate symbol to exchange format (unified CCXT symbol)
if self.exchange_id == "coinbase":
    ex_symbol = self._to_exchange_symbol(symbol)
else:
    ex_symbol = self._get_ccxt_symbol(symbol)

fmt_size = float(self.exchange.amount_to_precision(ex_symbol, raw_size))
if fmt_size < MIN_QTY_BTC:
    logger.warning(f"[Executor] fmt_size {fmt_size} < min qty {MIN_QTY_BTC}. Aborting.")
    return

# For Coinbase, ensure the symbol is in the markets cache
if self.exchange_id == "coinbase":
    if ex_symbol not in self.exchange.markets or self.exchange.markets.get(ex_symbol) is None:
        # Try to add it to markets cache if missing
        logger.warning(f"[Executor] Symbol {ex_symbol} not in markets cache. Attempting to register...")
        if not hasattr(self.exchange, 'markets') or self.exchange.markets is None:
            self.exchange.markets = {}

        # Use default market spec for BTC/USDC or BTC/USD
        if "BTC" in ex_symbol.upper() and "USDC" in ex_symbol.upper():
            self.exchange.markets['BTC/USDC'] = {
                'id': 'BTC-USDC', 'symbol': 'BTC/USDC', 'base': 'BTC', 'quote': 'USDC',
```

```
                'precision': {'amount': 0.00000001, 'price': 0.01},
                'limits': {'amount': {'min': 0.00001, 'max': 1000}, 'price': {'min': 0.01, 'max': 1000000}, 'co
                'active': True, 'type': 'spot', 'spot': True, 'margin': False, 'contract': False
            }
            logger.info("[Executor] Registered BTC/USDC to markets cache.")
        elif "BTC" in ex_symbol.upper() and "USD" in ex_symbol.upper():
            self.exchange.markets['BTC/USD'] = {
                'id': 'BTC-USD', 'symbol': 'BTC/USD', 'base': 'BTC', 'quote': 'USD',
                'precision': {'amount': 0.00000001, 'price': 0.01},
                'limits': {'amount': {'min': 0.00001, 'max': 1000}, 'price': {'min': 0.01, 'max': 1000000}, 'co
                'active': True, 'type': 'spot', 'spot': True, 'margin': False, 'contract': False
            }
            logger.info("[Executor] Registered BTC/USD to markets cache.")

    if self.dry_run:
        # ■■ DRY RUN ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        # CRIT-4 FIX: Apply slippage model so PnL accounting is realistic.
        # Market orders fill at mid-price + slippage. Without this, dry-run
        # win rates and daily loss caps are consistently optimistic.
        SLIPPAGE_PCT = 0.0002  # 0.02% == typical BTC/USDT USDM market order
        fill_price = (
            current_price * (1.0 + SLIPPAGE_PCT) if side == "buy"
            else current_price * (1.0 - SLIPPAGE_PCT)
        )
        cost = raw_size * fill_price
        logger.info(
            f"[DRY-RUN] {verdict} {ex_symbol} "
            f"| qty={raw_size:.6f} (${cost:.2f}) "
            f"| fill~{fill_price:.2f} (slip={SLIPPAGE_PCT:.2%}) "
            f"| SL={stop_loss} TP={take_profit} "
            f"| equity=${equity:.2f} risk={max_risk_pct}% "
            f"| ULIS={ulis_verdict}"
        )
        doc_id = self._log_trade(ex_symbol, side, verdict, fill_price, stop_loss, take_profit, ulis_verdict)
        self.active_position = {
            "symbol":      ex_symbol,
            "side":        side,
            "size":        raw_size,
            "entry_price": fill_price,
            "stop_loss":   stop_loss,
            "take_profit": take_profit,
            "dry_run":     True,
            "trade_doc_id": doc_id,
        }

        # Telegram Notification
        await self.notifier.send_trade_alert(
            symbol=ex_symbol, side=side, price=current_price,
            size=raw_size, sl=stop_loss, tp=take_profit, is_dry=True
        )

        return

    # ■■ LIVE EXECUTION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    # Guard: if the last order failed (e.g. -2015), wait before retrying.
    # Without this, the bot re-attempts the same signal every 15s.
    if time.time() < self.failed_order_ts:
        remaining = int(self.failed_order_ts - time.time())
        logger.info(f"[Executor] Failed-order cooldown active — {remaining}s remaining. Skipping.")
        return

    try:
        # We already have ex_symbol correctly formatted at line ~506 (e.g. BTC/USDT:USDT)
        # Only do the Coinbase USDC -> USD swap if applicable
        if self.exchange_id == "coinbase" and "USDC" in ex_symbol:
            ex_symbol = ex_symbol.replace("USDC", "USD")

        fmt_size = float(self.exchange.amount_to_precision(ex_symbol, raw_size))
        cost = fmt_size * current_price
        if cost > equity:
            fmt_size = float(
                self.exchange.amount_to_precision(ex_symbol, equity * 0.95 / current_price)
            )

        logger.info(f"[Executor] Placing MARKET {side.upper()} {fmt_size} {ex_symbol} @ ~{current_price}")
        import asyncio
        order = None
        for attempt in range(3):
```

```
    try:
        order = await self.exchange.create_market_order(ex_symbol, side, fmt_size)
        break
    except Exception as e:
        if attempt == 2:
            # Engage 90s cooldown — avoids flooding -2015 errors every cycle
            self.failed_order_ts = time.time() + 90.0
            logger.error(f"[Executor] Live order failed: {e}")
            raise e
        logger.warning(f"[Executor] Market order failed: {e}. Retrying {attempt+1}/3...")
        await asyncio.sleep(0.5)
logger.info(f"[Executor] Market order placed: id={order.get('id')} status={order.get('status')}")

# Track this order as pending until it fills or is cancelled
# MED-5 FIX: Add TTL so a stale pending_order can't lock the bot forever.
self.pending_order = {
    "id": order.get('id'),
    "symbol": ex_symbol,
    "side": side,
    "size": fmt_size,
    "entry_price": current_price,
    "status": order.get('status', 'open'),
    "expires_at": time.time() + 60,  # auto-expire after 60s if never confirmed
}

# Telegram Notification (Success)
await self.notifier.send_trade_alert(
    symbol=ex_symbol, side=side, price=current_price,
    size=fmt_size, sl=stop_loss, tp=take_profit, is_dry=False
)

sl_side = "sell" if side == "buy" else "buy"
sl_order_id = None
tp_order_id = None
sl_placed = False   # track independently for recovery logic
tp_placed = False

if self.is_futures:
    # ■■ FUTURES: STOP (stop-limit) + TAKE_PROFIT_MARKET ■■■■■■■■■■
    # Binance USDM futures uses order type "STOP" for stop-limit orders
    # (NOT "STOP_LIMIT" which is a spot-only type). "STOP" requires both
    # a stopPrice (trigger) and a price (limit fill level).
    # ATR buffer (atr * 0.02) gives ~$63 of slippage room on BTC.
    # If "STOP" is rejected by the exchange, we fall back to STOP_MARKET
    # so the bot never enters a position naked without any SL protection.
    atr_for_sl = signal.get("atr_at_entry", 0.0)
    sl_limit_price = float(self.exchange.price_to_precision(
        ex_symbol,
        stop_loss + (atr_for_sl * 0.02) if side == "buy" else stop_loss - (atr_for_sl * 0.02)
    ))
    sl_order = None
    _sl_last_err = None
    # ■■ Attempt 1: STOP (stop-limit, preferred — no slippage) ■■■■■
    for attempt in range(3):
        try:
            sl_order = await self.exchange.create_order(
                symbol=ex_symbol, type="STOP", side=sl_side,
                amount=fmt_size,
                price=sl_limit_price,
                params={
                    "stopPrice":  float(self.exchange.price_to_precision(ex_symbol, stop_loss)),
                    "reduceOnly": True,   # FIXED: reduceOnly works with amount; closePosition does not
                },
            )
            sl_placed = True
            logger.info(f"[Executor] Futures STOP (stop-limit) SL at {stop_loss} (limit={sl_limit_price}) ✓
            break
        except Exception as e:
            _sl_last_err = e
            logger.warning(f"[Executor] Futures STOP attempt {attempt+1}/3 failed: {e}")
            if attempt < 2:
                await asyncio.sleep(1.0)

    # ■■ Fallback: STOP_MARKET if STOP rejected ■■■■■■■■■■■■■■■■■■■■■
    if not sl_placed:
        logger.warning(
            f"[Executor] STOP order rejected after 3 attempts ({_sl_last_err}). "
            f"Falling back to STOP_MARKET — slippage risk accepted over naked position."
```

```
            )
            try:
                sl_order = await self.exchange.create_order(
                    symbol=ex_symbol, type="STOP_MARKET", side=sl_side,
                    amount=fmt_size,
                    params={
                        "stopPrice":     float(self.exchange.price_to_precision(ex_symbol, stop_loss)),
                        "closePosition": True,
                    },
                )
                sl_placed = True
                logger.info(f"[Executor] Futures STOP_MARKET SL (fallback) at {stop_loss} ✓")
            except Exception as e2:
                raise RuntimeError(
                    f"SL placement failed: STOP ({_sl_last_err}) and STOP_MARKET ({e2}) both rejected"
                )
        sl_order_id = sl_order.get("id")

        tp_order = None
        _tp_last_err = None
        for attempt in range(3):
            try:
                tp_order = await self.exchange.create_order(
                    symbol=ex_symbol, type="TAKE_PROFIT_MARKET", side=sl_side,
                    amount=fmt_size,
                    params={
                        "stopPrice": float(self.exchange.price_to_precision(ex_symbol, take_profit)),
                        "closePosition": True,
                        "workingType": "MARK_PRICE",
                    },
                )
                tp_placed = True
                break
            except Exception as e:
                _tp_last_err = e
                logger.warning(f"[Executor] Futures TP attempt {attempt+1}/3 failed: {e}")
                if attempt < 2:
                    await asyncio.sleep(1.0)
        if not tp_placed:
            # SL IS placed — position is not naked, just has no profit target.
            # Alert loudly but do NOT flatten; the SL protects against loss.
            _tp_warn = (
                f"■■ TP PLACEMENT FAILED for {side.upper()} {ex_symbol} @ {current_price}. "
                f"SL={stop_loss} IS active (id={sl_order_id}). "
                f"Position protected but NO take-profit. MONITOR MANUALLY."
            )
            logger.error(f"[Executor] {_tp_warn}")
            await self.notifier.send_error_alert(_tp_warn)
            tp_order_id = None
        else:
            tp_order_id = tp_order.get("id")
            logger.info(f"[Executor] Futures TAKE_PROFIT_LIMIT at {take_profit} (id={tp_order_id}) ✓")

    else:
        # ■■ SPOT: Exchange-specific limit SL/TP orders ■■■■■■■■■■■■■■■■■
        sl_limit = stop_loss * 0.999 if side == "buy" else stop_loss * 1.001
        sl_order = None
        _sl_last_err = None
        for attempt in range(3):
            try:
                if self.exchange_id == "binance":
                    sl_order = await self.exchange.create_order(
                        symbol=ex_symbol, type="STOP_LOSS_LIMIT", side=sl_side,
                        amount=fmt_size,
                        price=float(self.exchange.price_to_precision(ex_symbol, sl_limit)),
                        params={"stopPrice": float(self.exchange.price_to_precision(ex_symbol, stop_loss))},
                    )
                else:
                    # Coinbase Advanced Trade V3 specific stop directions
                    stop_params = {"stop_price": float(self.exchange.price_to_precision(ex_symbol, stop_loss))}
                    if self.exchange_id == "coinbase":
                        direction = "STOP_DIRECTION_STOP_UP" if stop_loss > current_price else "STOP_DIRECTION_
                        # Note: STOP_UP means trigger when price rises ABOVE stop_price (for shorts/longs),
                        # STOP_DOWN means trigger when price drops BELOW stop_price (for longs).
                        # This works correctly for both BUY and SELL positions due to how
                        # Coinbase interprets the direction relative to the order side.
                        stop_params["stop_direction"] = direction
                    sl_order = await self.exchange.create_order(
```

```
                    symbol=ex_symbol, type="limit", side=sl_side,
                    amount=fmt_size,
                    price=float(self.exchange.price_to_precision(ex_symbol, sl_limit)),
                    params=stop_params,
                )
            sl_placed = True
            break
        except Exception as e:
            _sl_last_err = e
            logger.warning(f"[Executor] SL attempt {attempt+1}/3 failed: {e}")
            if attempt < 2:
                await asyncio.sleep(1.0)
    if not sl_placed:
        raise RuntimeError(f"SL placement failed after 3 attempts: {_sl_last_err}")
    sl_order_id = sl_order.get("id")
    logger.info(f"[Executor] SL attached at {stop_loss} (id={sl_order_id}) ✓")

    # Take-profit limit order
    tp_order = None
    _tp_last_err = None
    for attempt in range(3):
        try:
            tp_order = await self.exchange.create_order(
                symbol=ex_symbol,
                type="limit",
                side=sl_side,
                amount=fmt_size,
                price=float(self.exchange.price_to_precision(ex_symbol, take_profit)),
                params={"timeInForce": "GTC"},
            )
            tp_placed = True
            break
        except Exception as e:
            _tp_last_err = e
            logger.warning(f"[Executor] TP attempt {attempt+1}/3 failed: {e}")
            if attempt < 2:
                await asyncio.sleep(1.0)
    if not tp_placed:
        _tp_warn = (
            f"■■ TP PLACEMENT FAILED for {side.upper()} {ex_symbol} @ {current_price}. "
            f"SL={stop_loss} IS active (id={sl_order_id}). "
            f"Position protected but NO take-profit. MONITOR MANUALLY."
        )
        logger.error(f"[Executor] {_tp_warn}")
        await self.notifier.send_error_alert(_tp_warn)
        tp_order_id = None
    else:
        tp_order_id = tp_order.get("id")
        logger.info(f"[Executor] TP attached at {take_profit} (id={tp_order_id}) ✓")

# BUG-1 FIX: Use actual exchange fill price, not signal price.
# Market orders fill at the ask (for buys) — using current_price
# corrupts every downstream calculation (PnL, BE stop, daily limit).
fill_price = float(order.get("average") or order.get("price") or current_price)
logger.info(f"[Executor] Fill price: {fill_price:.2f} (signal was {current_price:.2f}, diff={fill_price-cur

doc_id = self._log_trade(ex_symbol, side, verdict, fill_price, stop_loss, take_profit, ulis_verdict)
self.active_position = {
    "symbol":      ex_symbol,
    "side":        side,
    "size":        fmt_size,
    "entry_price": fill_price,
    "stop_loss":   stop_loss,
    "take_profit": take_profit,
    "order_id":    order.get("id"),
    "sl_order_id": sl_order_id,
    "tp_order_id": tp_order_id,
    "sl_placed":   sl_placed,
    "tp_placed":   tp_placed,
    "dry_run":     False,
    "trade_doc_id": doc_id,
}

# Clear pending order since we now have an active position
self.pending_order = None

sl_tp_status = "SL+TP ✓" if (sl_placed and tp_placed) else ("SL ✓ | TP ✗ MONITOR" if sl_placed else "■■ NA
logger.info(f"[Executor] Position open — protection status: {sl_tp_status}")
```

```
            await self.notifier.send_trade_alert(
                symbol=ex_symbol, side=side, price=current_price,
                size=fmt_size, sl=stop_loss, tp=take_profit, is_dry=False
            )
            return True


        except ccxt.InsufficientFunds as e:
            logger.error(f"[Executor] Insufficient funds: {e}")
        except ccxt.InvalidOrder as e:
            logger.error(f"[Executor] Invalid order: {e}")
        except Exception as e:
            err_msg = f"Live order / SL placement failed: {e}"
            logger.error(f"[Executor] {err_msg}", exc_info=True)

            # ■■ EMERGENCY: SL failed — position is NAKED. Flatten immediately. ■■
            # Note: TP-only failures are handled above and do NOT reach here —
            # the position keeps its SL in that case and is therefore still protected.
            if self.active_position is None and 'order' in locals() and order and order.get('id'):
                critical_msg = (
                    f"■ CRITICAL: SL PLACEMENT FAILED after market fill! "
                    f"FLATTENING NAKED {side.upper()} {ex_symbol} NOW. Error: {e}"
                )
                logger.critical(f"[Executor] {critical_msg}")
                await self.notifier.send_error_alert(critical_msg)

                close_side = "sell" if side == "buy" else "buy"
                try:
                    await self.exchange.create_market_order(ex_symbol, close_side, fmt_size)
                    logger.info("[Executor] ✓ Naked position flattened successfully.")
                    await self.notifier.send_error_alert(
                        f"■ Naked {side.upper()} {ex_symbol} position flattened. No unintended exposure remains."
                    )
                except Exception as flatten_err:
                    manual_msg = (
                        f"■■ CRITICAL FAILURE: Could not flatten naked {side.upper()} {ex_symbol} position! "
                        f"MANUAL INTERVENTION REQUIRED IMMEDIATELY! Flatten error: {flatten_err}"
                    )
                    logger.critical(f"[Executor] {manual_msg}")
                    await self.notifier.send_error_alert(manual_msg)
            else:
                await self.notifier.send_error_alert(err_msg)

        finally:
            # If we don't have an active position after all that,
            # we MUST clear pending_order so the bot isn't stuck "Waiting"
            if self.active_position is None:
                self.pending_order = None

    # ------------------------------------------------------------------
    # Position monitor (called from main loop for dry-run)
    # ------------------------------------------------------------------

    async def move_sl_to_breakeven(self, symbol: str, entry_price: float):
        """Moves the current Stop Loss to the entry price (Breakeven)."""
        if self._dry_run:
            logger.info("[Executor] DRY-RUN: Simulated moving SL to breakeven.")
            return

        logger.info(f"[Executor] Moving Stop Loss to Breakeven @ {entry_price:.2f}")
        try:
            if not self.active_position:
                return

            # Phase 1: Place the NEW Stop Loss FIRST
            # This ensures we NEVER have a naked position if the API fails
            fmt_size = self.exchange.amount_to_precision(symbol, self.active_position["size"])
            close_side = "sell" if self.active_position["side"] == "buy" else "buy"

            new_order = await self.exchange.create_order(
                symbol,
                "STOP_MARKET",
                close_side,
                fmt_size,
                None,
                params={
                    "stopPrice": float(self.exchange.price_to_precision(symbol, entry_price)),
                    "reduceOnly": True
```

```
                }
            )
            logger.info("[Executor] Phase 1 Success: New Breakeven SL safely placed on exchange.")

            # Phase 2: Now that new SL is secure, cancel the OLD Stop Loss
            open_orders = await self.exchange.fetch_open_orders(symbol)
            for order in open_orders:
                is_stop = order.get("type", "").lower() in ("stop", "stop_market", "stopmarket")
                is_not_new = str(order.get("id")) != str(new_order.get("id"))
                if is_stop and is_not_new:
                    await self.exchange.cancel_order(order["id"], symbol)
                    logger.debug(f"[Executor] Phase 2 Success: Cancelled old SL order {order['id']}")

            if hasattr(self, "notifier") and self.notifier:
                await self.notifier.send_message(f"■ **Breakeven Secured**\
Moved Stop Loss to entry price at {entry_price:.2f} for {symbol}.")

        except Exception as e:
            # If Phase 1 fails, Phase 2 never runs. The original Stop Loss remains active.
            # Your capital is still fully protected.
            err_msg = f"Failed to move SL to breakeven on Live Exchange: {e}"
            logger.error(f"[Executor] CRITICAL: {err_msg}")
            if hasattr(self, "notifier") and self.notifier:
                await self.notifier.send_error_alert(f"■■ **Breakeven Move Failed!**\
{err_msg}\
*Note: Original Stop Loss is still active.*")

    async def check_position_exit(self, current_price: float,
                                  candle_high: float = None,
                                  candle_low: float = None) -> tuple:
        """
        Check SL/TP for position exit.

        DRY-RUN: Simulates SL/TP hits from candle high/low for realistic backtesting.
        LIVE MODE: Polls exchange fetch_positions() to detect real fills by the
                   exchange's STOP_MARKET / TAKE_PROFIT_MARKET orders.

        CRIT-1 FIX: Previously both modes ran the same simulation logic. In live mode
        this caused race conditions where internal state was cleared while the exchange
        still had open protective orders, leading to orphaned cancel calls and missed PnL.

        Returns (exited: bool, pnl: float).
        """
        pos = self.active_position
        if pos is None:
            return False, 0.0

        if not self.dry_run:
            return await self._check_live_position_exit(current_price)

        # ■■ DRY-RUN simulation ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        side = pos["side"]
        sl   = pos["stop_loss"]
        tp   = pos["take_profit"]

        # PHASE-0.3: Use exchange-specific fees set in __init__
        TAKER_FEE = self.TAKER_FEE
        MAKER_FEE = self.MAKER_FEE

        size = pos.get("size") or pos.get("qty") or 0.0
        if size <= 0:
            logger.warning("[Executor] Active position has 0 size. Clearing stale state.")
            self.active_position = None
            return False, 0.0

        notional = size * pos["entry_price"]
        entry_fee = notional * TAKER_FEE

        # Use candle high/low for realistic SL/TP simulation if available
        check_high = candle_high if candle_high is not None else current_price
        check_low  = candle_low  if candle_low  is not None else current_price

        async def finalize_exit(exit_type: str, exit_price: float):
            nonlocal notional, entry_fee, side
            exit_notional = size * exit_price
            exit_fee = exit_notional * (MAKER_FEE if exit_type == "TP" else TAKER_FEE)

            raw_pnl = (exit_price - pos["entry_price"]) * size if side == "buy" else (pos["entry_price"] - exit_price)
```

```
        net_pnl = raw_pnl - (entry_fee + exit_fee)
        logger.info(f"[Executor] {exit_type} HIT{' (SHORT)' if side == 'sell' else ''}. Net PnL=${net_pnl:.2f} (Fee

        if hasattr(self, "notifier") and self.notifier:
            await self.notifier.send_close_alert(
                symbol=pos["symbol"], side=side, price=exit_price, type=exit_type, pnl=net_pnl, is_dry=self.dry_run
            )
        self._update_trade_exit(pos.get("trade_doc_id"), exit_price, net_pnl)
        self.active_position = None
        self.pending_order = None
        return True, net_pnl

    if side == "buy":
        if check_low <= sl:
            return await finalize_exit("SL", sl)
        if check_high >= tp:
            return await finalize_exit("TP", tp)
    else:
        if check_high >= sl:
            return await finalize_exit("SL", sl)
        if check_low <= tp:
            return await finalize_exit("TP", tp)

    return False, 0.0

async def _check_live_position_exit(self, current_price: float) -> tuple:
    """
    LIVE MODE: Poll exchange to detect if position was closed by SL/TP orders.

    P0-1 GHOST-POSITION FIX:
    Binance USDM Futures OMITS zero-quantity positions from fetch_positions entirely.
    The old code searched for a matching symbol with qty < 0.0001 — since Binance
    never returns that entry at all when flat, the position was never detected as closed.

    Correct approach: build a set of symbols that have NON-ZERO qty (open_syms).
    If our symbol is ABSENT from open_syms, the position is flat on the exchange.
    Then fetch the actual fill price + realized PnL from trade history.
    """
    pos = self.active_position
    if pos is None:
        return False, 0.0

    symbol = pos.get("symbol", "")
    logger.debug(f"[LiveExit] Polling exchange for {symbol}…")

    try:
        # Fetch ALL positions — Binance only returns non-flat ones
        all_positions = await self.exchange.fetch_positions()

        # Build the set of symbols that are genuinely open (qty > noise floor)
        open_syms = {
            p.get("symbol")
            for p in all_positions
            if abs(float(p.get("contracts", 0) or 0)) > 0.0001
        }

        if symbol not in open_syms:
            # ■■ Position is FLAT on the exchange ■■■■■■■■■■■■■■■■■■■■■■■■■
            # The SL or TP order was filled by Binance; bot state is stale.
            entry = pos["entry_price"]
            size  = pos["size"]
            side  = pos["side"]

            # Fetch actual fill price + exchange-reported realized PnL
            fill_price = current_price  # fallback
            actual_pnl = None
            try:
                recent_trades = await self.exchange.fetch_my_trades(symbol, limit=10)
                # Closing trades have side opposite to our entry side
                closing = [
                    t for t in recent_trades
                    if t.get("side", "").lower() != side.lower()
                ]
                if closing:
                    fill_price = float(closing[-1]["price"])
                    actual_pnl = float(
                        closing[-1].get("info", {}).get("realizedPnl", 0)
                    )
```

```
                    if actual_pnl != 0:
                        logger.info(
                            f"[LiveExit] Exchange-reported fill={fill_price:.2f} "
                            f"realizedPnl=${actual_pnl:.2f}"
                        )
                    else:
                        logger.info(f"[LiveExit] Exchange-reported fill={fill_price:.2f}")
            except Exception as e:
                logger.warning(f"[LiveExit] fill-price fetch failed: {e}")

            # Use exchange PnL if available, otherwise estimate from geometry
            if actual_pnl is not None and actual_pnl != 0:
                net_pnl = actual_pnl
            else:
                raw_pnl = ((fill_price - entry) * size if side == "buy"
                           else (entry - fill_price) * size)
                fees = (entry + fill_price) * size * self.TAKER_FEE
                net_pnl = raw_pnl - fees

            exit_type = "TP" if net_pnl > 0 else "SL"
            logger.info(
                f"[LiveExit] Position flat on exchange — type={exit_type} "
                f"pnl=${net_pnl:.2f} fill={fill_price:.2f}"
            )
            if self.notifier:
                await self.notifier.send_close_alert(
                    symbol=symbol, side=side, price=fill_price,
                    type=exit_type, pnl=net_pnl, is_dry=False
                )
            self._update_trade_exit(pos.get("trade_doc_id"), fill_price, net_pnl)
            self.active_position = None
            self.pending_order = None
            return True, net_pnl

        # Symbol still present in open_syms → position still live
        return False, 0.0

    except Exception as e:
        logger.warning(f"[LiveExit] poll error: {e}")
        return False, 0.0



async def cancel_opposing_orders(self, filled_side: str = "sl"):
    """
    Cancel the opposing order after one side fills.
    In LIVE mode: if SL fills, cancel TP order (and vice versa).
    Prevents naked re-entries from orphaned orders.
    """
    pos = self.active_position
    if pos is None:
        return

    cancel_id = pos.get("tp_order_id") if filled_side == "sl" else pos.get("sl_order_id")
    symbol = pos.get("symbol", "")

    if cancel_id:
        try:
            await self.exchange.cancel_order(cancel_id, symbol)
            label = "TP" if filled_side == "sl" else "SL"
            logger.info(f"[Executor] Cancelled opposing {label} order {cancel_id} ✓")
        except Exception as e:
            # -2011 "Unknown order sent" means Binance already cancelled it
            # (closePosition=True orders are auto-cleaned when position closes).
            err_str = str(e).lower()
            already_gone = (
                "-2011" in err_str
                or "unknown order" in err_str
                or "not found" in err_str
                or "not_found" in err_str
            )
            if already_gone:
                label = "TP" if filled_side == "sl" else "SL"
                logger.info(f"[Executor] Opposing {label} order {cancel_id} already gone (Binance cleaned it) ✓")
            else:
                logger.warning(f"[Executor] Failed to cancel opposing order {cancel_id}: {e}")

def is_system_locked(self) -> bool:
```

```
    """Check if the system is currently under a panic-mode lock."""
    return time.time() < self.lock_expiry

async def engage_panic_mode(self, reason: str, lock_seconds: int = 300):
    """
    Engages the killswitch: blocks new trades and locks the system.
    Does NOT flatten existing positions — let hard SLs handle exits.
    """
    logger.warning(f"[PanicMode] ENGAGED! Reason: {reason}. Locking system for {lock_seconds}s.")
    self.lock_expiry = time.time() + lock_seconds
    self.last_panic_reason = reason

    # Do NOT flatten positions — market orders into a hollowed order book
    # cause massive slippage. Let the hard Stop-Loss handle exits.

    # Notify user via Telegram
    if self.notifier:
        await self.notifier.send_message(f"■ PANIC MODE ENGAGED!\nReason: {reason}\nSystem locked for {lock_seconds

async def emergency_flatten(self, reason: str):
    """Immediately closes the current position with a Market Order."""
    if not self.active_position:
        return

    pos = self.active_position
    ex_symbol = pos["symbol"]
    side = pos["side"]
    size = pos["size"]
    close_side = "sell" if side == "buy" else "buy"

    logger.warning(f"[Executor] EMERGENCY FLATTEN triggered ({reason}) for {size} {ex_symbol}")

    try:
        if not self.dry_run:
            # 1. Cancel all open orders for this symbol first
            try:
                await self.exchange.cancel_all_orders(ex_symbol)
            except Exception:
                pass

            # 2. Market close
            await self.exchange.create_market_order(ex_symbol, close_side, size)

        logger.info(f"[Executor] Flattened {ex_symbol} ■")

        # BUG-2 + HIGH-1 FIX: Fetch actual fill price from trade history.
        # Old code: _last_price never set → fill_price always = entry → PnL always $0.
        # Every panic exit was recorded as flat, bypassing the daily loss limit.
        entry = pos.get("entry_price", 0.0)
        size  = pos.get("size", 0.0)
        side  = pos.get("side", "buy")
        fill_price = entry  # fallback
        est_pnl    = 0.0
        if not self.dry_run:
            try:
                recent = await self.exchange.fetch_my_trades(ex_symbol, limit=5)
                closing = [
                    t for t in recent
                    if float(t.get("info", {}).get("realizedPnl", 0)) != 0
                ]
                if closing:
                    fill_price = float(closing[-1]["price"])
                    est_pnl    = float(closing[-1].get("info", {}).get("realizedPnl", 0))
                    logger.info(f"[Flatten] Exchange PnL: ${est_pnl:.2f} fill={fill_price:.2f}")
                else:
                    # Estimate from entry vs current candle close
                    raw = (fill_price - entry) * size if side == "buy" else (entry - fill_price) * size
                    est_pnl = raw - (entry + fill_price) * size * self.TAKER_FEE
            except Exception as fe:
                logger.warning(f"[Flatten] Could not fetch fill: {fe}")
        # HIGH-1 FIX: pass real fill_price (not 0.0) to Firestore
        self._last_panic_pnl = est_pnl  # BUG-3: expose for main.py daily_pnl update
        self._update_trade_exit(pos.get("trade_doc_id"), fill_price, est_pnl)
        self.active_position = None
    except Exception as e:
        # 3.4 FIX: Critical failure requires immediate human action.
        # Fire Telegram and log at CRITICAL level regardless of any other state.
        msg = (
```

```
    f"■ CRITICAL: FAILED TO FLATTEN POSITION\n"
    f"Symbol: {ex_symbol} | Side: {side.upper()} | Size: {size}\n"
    f"Reason: {reason}\n"
    f"Error: {str(e)}\n"
    f"ACTION REQUIRED: Log into exchange immediately and close this position manually."
)
logger.critical(msg)
if self.notifier:
    try:
        await self.notifier.send_message(msg)
    except Exception as notify_err:
        logger.error(f"[Executor] Also failed to send Telegram alert: {notify_err}")
# P0-3 FIX: Use a dedicated boolean flag, NOT a sentinel dict.
# The 30s heartbeat poll calls _check_live_position_exit which sets
# active_position = None if the symbol is gone — silently wiping the
# sentinel dict and allowing the bot to resume trading unprotected.
# A separate boolean survives the poll clearing active_position.
self._flatten_failed = True
self.active_position = None  # Clear position — halt is enforced via _flatten_failed flag
```

■ notifier.py

```
import logging
import httpx
import asyncio
from typing import Optional

logger = logging.getLogger(__name__)

class TelegramNotifier:
    """
    Handle sending trade alerts and error notifications to Telegram.
    """
    def __init__(self, token: Optional[str], chat_id: Optional[str]):
        self.token = token
        self.chat_id = chat_id
        self.base_url = f"https://api.telegram.org/bot{token}/sendMessage" if token else None

    @property
    def is_active(self) -> bool:
        return bool(self.token and self.chat_id)

    async def send_message(self, text: str):
        if not self.is_active:
            return

        try:
            async with httpx.AsyncClient(timeout=10.0) as client:
                payload = {
                    "chat_id": self.chat_id,
                    "text": text,
                    "parse_mode": "HTML"
                }
                resp = await client.post(self.base_url, json=payload)
                resp.raise_for_status()
        except Exception as e:
            logger.error(f"[Telegram] Failed to send message: {e}")

    async def send_trade_alert(self, symbol: str, side: str, price: float, size: float, sl: float, tp: float, is_dry: b
        """Send a beautiful trade entry alert."""
        mode_str = "■ [DRY-RUN]" if is_dry else "■ [LIVE-TRADE]"
        emoji = "■" if side.lower() == "buy" else "■"

        msg = (
            f"<b>{mode_str} ENTRY</b>\n"
            f"■■■■■■■■■■■■■■■■\n"
            f"{emoji} <b>{side.upper()} {symbol}</b>\n"
            f"■ Price: <code>{price:.2f}</code>\n"
            f"■ Size: <code>{size:.6f}</code>\n"
            f"■ SL: <code>{sl:.2f}</code>\n"
            f"■ TP: <code>{tp:.2f}</code>\n"
            f"■■■■■■■■■■■■■■■■"
        )
        await self.send_message(msg)

    async def send_startup_alert(
        self,
        symbol: str,
        exchange: str,
        mode: str,
        leverage: int,
        interval: str,
        version: str = "v7",
    ):
        """Send a startup notification when the bot comes online."""
        mode_emoji = "■" if mode == "DRY-RUN" else "■"
        msg = (
            f"<b>{mode_emoji} QUAD-DESK BOT ONLINE</b>\n"
            f"■■■■■■■■■■■■■■■■■■\n"
            f"■ <b>Exchange:</b> <code>{exchange.upper()}</code>\n"
            f"■ <b>Symbol:</b>   <code>{symbol}</code>\n"
            f"■ <b>Leverage:</b> <code>{leverage}×</code>\n"
            f"■ <b>Interval:</b> <code>{interval}</code>\n"
            f"■ <b>Mode:</b>     <code>{mode}</code>\n"
            f"■ <b>Engine:</b>   <code>7-Stage Hybrid {version}</code>\n"
            f"■■■■■■■■■■■■■■■■■■\n"
            f"<i>Bot is live and scanning markets.</i>"
        )
```

```
    await self.send_message(msg)

async def send_error_alert(self, error_msg: str):
    """Send an urgent error notification."""
    msg = f"■■ <b>BOT ERROR</b>\n■■■■■■■■■■■■■■■■\n<code>{error_msg}</code>"
    await self.send_message(msg)

async def send_close_alert(self, symbol: str, side: str, price: float, type: str, pnl: float, is_dry: bool = False)
    """Send a beautiful trade exit summary alert."""
    mode_str = "■ [DRY-RUN]" if is_dry else "■ [LIVE-TRADE]"
    emoji = "■" if type == "SL" else "■"
    result = "PROFIT" if pnl >= 0 else "LOSS"

    msg = (
        f"<b>{mode_str} CLOSE</b>\n"
        f"■■■■■■■■■■■■■■■■\n"
        f"{emoji} <b>{type} HIT: {side.upper()} {symbol}</b>\n"
        f"■ Exit Price: <code>{price:.2f}</code>\n"
        f"■ {result}: <code>${pnl:.2f}</code>\n"
        f"■■■■■■■■■■■■■■■■"
    )
    await self.send_message(msg)
```

■ main.py

```
"""
Macro Strategy Execution Bot — Binance USDM Futures Edition (7-Stage Hybrid)
============================================================================
Launch with:
    python -m bot.main

Architecture (7 stages):
  1. Feature Engine      — all signals from live market data (Binance Futures WS)
  2. Regime Detection    — classify: TREND | RANGE | LIQUIDITY | NEUTRAL
  3. Liquidity Sweep     — detect stop-hunts above/below walls
  4. Strategy Layer      — A: Trend, B: Mean Reversion, C: Liquidity Sweep
  5. Bayesian Fusion     — sequential probability update
  6. ULIS/ALDE Gate ★NEW — ALDE+ULIS hybrid verdict: veto or boost signal
  7. Risk Engine         — ATR-based SL/TP, daily-loss guard

Environment variables:
    BOT_EXCHANGE               — binanceusdm (Binance USDM Futures, default)
    BINANCE_API_KEY            — Binance Futures API key
    BINANCE_API_SECRET         — Binance Futures API secret
    BOT_SYMBOL                 — default: BTC/USDT
    BOT_TESTNET                — default: false (live trading)
    BOT_LEVERAGE               — default: 3  (futures leverage, 1–20×)
    BOT_MAX_RISK_PCT           — default: 1.0  (% of equity per trade)
    BOT_MAX_DAILY_LOSS_PCT     — default: 3.0  (% of equity; pauses if hit)
    BOT_ANALYSIS_INTERVAL      — default: 10   (seconds between cycles)
    BOT_CANDLE_INTERVAL        — default: 15m  (kline interval)
    BOT_MIN_CONFIDENCE         — default: 0.65 (Bayesian fusion threshold)
    BOT_ACCOUNT_SIZE           — default: 100  (USDT equity in futures wallet)
    BOT_ULIS_GATE              — default: true (enable Stage 6 ULIS gate)
    BOT_PANIC_DROP_PCT         — default: 3.0  (% flash-crash triggers panic mode)
    BOT_PANIC_LOOKBACK         — default: 5    (candles to look back for crash)
    BOT_PANIC_LOCK_SECONDS     — default: 300  (seconds to lock after panic)

Geographic note (Railway USA ↔ Binance Europe account):
    No issue. Binance USDM Futures API (fapi.binance.com) is globally accessible.
    The adjustForTimeDifference + recvWindow=10000 options in executor.py handle
    cross-continental clock skew automatically.
"""

import asyncio
import logging
import os
import re
import signal
import time
import numpy as np
from collections import deque
from datetime import date
from typing import Dict, Any, Optional, Tuple, List

from dotenv import load_dotenv
load_dotenv()

from bot.data_feed import BinanceDataFeed
from bot.quant_engine import QuantEngine
from bot.executor import TradingExecutor
from bot.ulis_engine import compute_ulis_verdict
from bot.derivatives_context import DerivativesContext
from bot import heartbeat
from bot.signal_config import (
    REGIME_PARAMS, POST_TRADE_COOLDOWN_S, COLD_START_TRADE_COUNT,
    COLD_START_CONFIDENCE_DISCOUNT,
    MAX_RISK_PCT as CFG_MAX_RISK_PCT,
    MAX_DAILY_LOSS_PCT as CFG_MAX_DAILY_LOSS_PCT,
    MAX_DRAWDOWN_PCT as CFG_MAX_DRAWDOWN_PCT,
    MIN_BAYESIAN as CFG_MIN_BAYESIAN
)

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Logging
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s | %(levelname)-8s | %(name)s | %(message)s',
    datefmt='%H:%M:%S',
)
```

```
logger = logging.getLogger("bot.main")

try:
    from bot.heartbeat import FirestoreLogHandler
    logging.getLogger().addHandler(FirestoreLogHandler())
except Exception as e:
    logger.warning(f"[Main] Could not attach FirestoreLogHandler: {e}")


# ═══════════════════════════════════════════════════════════════════════════
# Config from environment
# ═══════════════════════════════════════════════════════════════════════════
EXCHANGE            = os.environ.get("BOT_EXCHANGE",            "binanceusdm").lower()
SYMBOL              = os.environ.get("BOT_SYMBOL",              "BTC-USDC" if EXCHANGE == "coinbase" else "BTC/USDT")
TESTNET             = os.environ.get("BOT_TESTNET",             "false").lower() != "false"
MAX_RISK_PCT        = float(os.environ.get("BOT_MAX_RISK_PCT",        str(CFG_MAX_RISK_PCT)))
MAX_DAILY_LOSS_PCT  = float(os.environ.get("BOT_MAX_DAILY_LOSS_PCT",  str(CFG_MAX_DAILY_LOSS_PCT)))
ANALYSIS_INTERVAL   = int(os.environ.get("BOT_ANALYSIS_INTERVAL",    "15"))
CANDLE_INTERVAL     = os.environ.get("BOT_CANDLE_INTERVAL",      "15m")
MIN_CONFIDENCE      = float(os.environ.get("BOT_MIN_CONFIDENCE",      str(CFG_MIN_BAYESIAN)))
ACCOUNT_SIZE        = float(os.environ.get("BOT_ACCOUNT_SIZE",        "100.0"))
LEVERAGE            = int(os.environ.get("BOT_LEVERAGE",              "3"))    # futures leverage (3× = efficient margi
ULIS_GATE_ENABLED   = os.environ.get("BOT_ULIS_GATE",           "true").lower() != "false"

# Fee rates: Coinbase Spot 1.2% | Binance Spot 0.1% | Binance USDM Futures 0.04%
_EXCHANGE_FEE_RATE  = 0.012 if EXCHANGE == "coinbase" else (0.0004 if EXCHANGE == "binanceusdm" else 0.001)

# Panic mode: trigger if price drops this % within PANIC_LOOKBACK candles
# e.g. 3.0 = 3% crash within 5 candles → engage panic mode
PANIC_DROP_PCT     = float(os.environ.get("BOT_PANIC_DROP_PCT",   "3.0"))
PANIC_LOOKBACK     = int(os.environ.get("BOT_PANIC_LOOKBACK",     "5"))   # candles
PANIC_LOCK_SECONDS = int(os.environ.get("BOT_PANIC_LOCK_SECONDS", "300")) # 5 min default
# Max drawdown: halt ALL trading if cumulative session PnL exceeds this % of ACCOUNT_SIZE
MAX_DRAWDOWN_PCT   = float(os.environ.get("BOT_MAX_DRAWDOWN_PCT",  str(CFG_MAX_DRAWDOWN_PCT)))


# Coinbase credentials
CB_KEY_NAME     = os.environ.get("COINBASE_API_KEY_NAME",  "")
CB_PRIVATE_KEY  = os.environ.get("COINBASE_PRIVATE_KEY",   "")

# Binance credentials
BINANCE_API_KEY        = os.environ.get("BINANCE_API_KEY",             "")
BINANCE_API_SECRET     = os.environ.get("BINANCE_API_SECRET",          "")
BINANCE_ED25519_PRIVKEY = os.environ.get("BINANCE_ED25519_PRIVATE_KEY", "")
# Normalise escaped newlines from .env (stored as literal \n)
if BINANCE_ED25519_PRIVKEY:
    BINANCE_ED25519_PRIVKEY = BINANCE_ED25519_PRIVKEY.replace("\\n", "\n").strip()

# Telegram credentials
TG_BOT_TOKEN   = os.environ.get("TELEGRAM_BOT_TOKEN", "")
TG_CHAT_ID     = os.environ.get("TELEGRAM_CHAT_ID",   "")

# Derive dry-run: explicit override or no credentials at all
ENV_DRY_RUN = str(os.environ.get("BOT_DRY_RUN", "")).lower() == "true"
if EXCHANGE == "coinbase":
    DRY_RUN = ENV_DRY_RUN or not (CB_KEY_NAME and CB_PRIVATE_KEY)
else:
    # For Binance: either HMAC secret OR Ed25519 private key is sufficient
    DRY_RUN = ENV_DRY_RUN or not (BINANCE_API_KEY and (BINANCE_API_SECRET or BINANCE_ED25519_PRIVKEY))

# Data feed always uses Binance public WS; normalise symbol to BTCUSDT style
if EXCHANGE == "coinbase":
    base = SYMBOL.replace("/", "-").split("-")[0]
    FEED_SYMBOL = f"{base}USDT"
else:
    # BTC/USDT  → BTCUSDT  |  BTCUSDT → BTCUSDT
    FEED_SYMBOL = SYMBOL.replace("/", "").split(":")[0]

# ═══════════════════════════════════════════════════════════════════════════
# Shared stats (written to Firestore by heartbeat)
# ═══════════════════════════════════════════════════════════════════════════
BOT_STATS: Dict[str, Any] = {
    "symbol":          SYMBOL,
    "exchange":        EXCHANGE.upper(),
    "mode":            "LIVE" if not DRY_RUN else "DRY-RUN",
    "environment":     "TESTNET" if TESTNET else "MAINNET",
    "active_position": None,
    "total_trades":    0,
```

```
    "last_signal":     "WAIT",
    "last_ulis":       "—",
    "daily_pnl":       0.0,
    "session_pnl":    0.0,  # PHASE-3.1: Initialized for daily drawdown reset
    "daily_loss_halt": False,
    "consecutive_losses": 0,
    "cooldown_until":     0.0,
    "gate_stats": {
        "daily_loss_halt":           0,
        "drawdown_halt":             0,
        "zscore_warmup":            0,
        "post_trade_cooldown":      0,
        "cascade_cooldown":         0,
        "regime_no_edge":           0,
        "htf_counter_trend":        0,
        "funding_blocks_long":      0,
        "funding_blocks_short":     0,
        "ulis_veto":                0,
        "ulis_alignment_fail":      0,
        "confidence_below_threshold": 0,
        "candle_gate_expired":       0,
        "micro_confirms_failed":    0,
        "sweep_confirms_failed":    0,
        "fee_geometry":             0,
"signal_none":              0,
        "total_passed":             0,
        "cvd_divergence_veto":      0,
        "derivatives_veto":         0,
    },
    "equity_peak": ACCOUNT_SIZE,
}

GATE_STATS_LAST_LOG = 0.0  # timestamp of last 30-min gate summary
_CYCLE_ERROR_COUNT = 0  # PHASE-3.1: Consecutive cycle errors for escalation
_LAST_CYCLE_ERROR = ""  # PHASE-3.1: Last error string for escalation

LAST_CASCADE_TIME = 0.0
LAST_CVD: float   = 0.0  # Track previous CVD value so execution loop can compute delta
LAST_CANDLE_TS: float = 0.0  # Track last confirmed 15m candle open-time for candle-close gate
LAST_ANY_TRADE_CLOSE_TIME = 0.0  # Track any trade exit for post-trade cooldown
LAST_TRADE_WAS_SL: bool = False  # Track if last exit was SL for split cooldown logic
# FIX-P5: Sweep deduplication — track the candle-open-time of the last fired sweep.
# The same stale wick fires as a "new" sweep every 15s cycle without this guard.
_LAST_FIRED_SWEEP_CANDLE_TS: float = 0.0


def _gate_stats_summary(reason: str, confidence: float = 0.0) -> None:
    """
    Increment the appropriate gate counter and log a summarised rejection
    reason every 30 minutes.  Call this at every WAIT return in _compute_signal.
    """
    import time
    global GATE_STATS_LAST_LOG

    g = BOT_STATS["gate_stats"]
    if reason in g:
        g[reason] += 1
    else:
        g[reason] = 1
        logger.warning(f"[GateStats] Unknown gate key: {reason}")

    now = time.time()
    if now - GATE_STATS_LAST_LOG >= 1800:
        total = sum(v for k, v in g.items() if k != "total_passed")
        passed = g["total_passed"]
        logger.info(
            "[GateStats] === 30-min Gate Rejection Summary ===\n"
            f"  daily_loss_halt      : {g.get('daily_loss_halt', 0)}\n"
            f"  drawdown_halt       : {g.get('drawdown_halt', 0)}\n"
            f"  zscore_warmup       : {g.get('zscore_warmup', 0)}\n"
            f"  post_trade_cooldown : {g.get('post_trade_cooldown', 0)}\n"
            f"  cascade_cooldown    : {g.get('cascade_cooldown', 0)}\n"
            f"  regime_no_edge      : {g.get('regime_no_edge', 0)}\n"
            f"  htf_counter_trend   : {g.get('htf_counter_trend', 0)}\n"
            f"  funding_blocks      : {g.get('funding_blocks_long', 0) + g.get('funding_blocks_short', 0)}\n"
            f"  ulis_veto           : {g.get('ulis_veto', 0)}\n"
            f"  ulis_alignment_fail : {g.get('ulis_alignment_fail', 0)}\n"
            f"  low_confidence      : {g.get('confidence_below_threshold', 0)}\n"
```

```
            f"  candle_gate_expired : {g.get('candle_gate_expired', 0)}\n"
            f"  micro_confirms_fail : {g.get('micro_confirms_failed', 0)}\n"
            f"  sweep_confirms_fail : {g.get('sweep_confirms_failed', 0)}\n"
            f"  fee_geometry        : {g.get('fee_geometry', 0)}\n"
            f"  signal_none         : {g.get('signal_none', 0)}\n"
            f"  ═══════════════════════════\n"
            f"  total rejections    : {total}\n"
            f"  total passed       : {passed}\n"
            f"  pass rate           : {passed/max(1, passed+total):.1%}\n"
            f"  last_confidence     : {confidence:.2%}"
        )
        GATE_STATS_LAST_LOG = now


# ═══════════════════════════════════════════════════════════════════════════════
# ══ STAGE 1 — FEATURE ENGINE HELPERS ═══════════════════════════════
# ═══════════════════════════════════════════════════════════════════════════════

def _parse_walls(all_walls_str: Optional[str]) -> Tuple[List[float], List[float]]:
    if not all_walls_str:
        return [], []
    buy_walls, sell_walls = [], []
    for part in all_walls_str.split(";"):
        m = re.match(r"\s*(BUY|SELL)@([\d.]+)", part.strip())
        if m:
            (buy_walls if m.group(1) == "BUY" else sell_walls).append(float(m.group(2)))
    buy_walls.sort(reverse=True)
    sell_walls.sort()
    return buy_walls, sell_walls


# ═══════════════════════════════════════════════════════════════════════════════
# ══ STAGE 2 — MARKET REGIME DETECTION ══════════════════════════════
# ═══════════════════════════════════════════════════════════════════════════════


def _htf_trend(candle_history: list) -> str:
    """
    Derive a higher-timeframe (4H) trend direction from the last 16 15m candles.
    16 × 15m = 4 hours.  Returns 'BULL', 'BEAR', or 'NEUTRAL'.

    Blocks counter-trend entries:
    - HTF = BULL  →  block SELL / MEAN_REVERSAL_SHORT
    - HTF = BEAR  →  block BUY  / MEAN_REVERSAL_LONG
    - HTF = NEUTRAL → allow either direction
    """
    if len(candle_history) < 16:
        return "NEUTRAL"
    # The open of the 16th candle ago represents the start of the 4H window
    window_open = float(candle_history[-16]["open"])
    htf_close = float(candle_history[-1]["close"])
    if window_open <= 0:
        return "NEUTRAL"
    change_pct = (htf_close - window_open) / window_open
    # HIGH-3 FIX: 0.2% was too tight (BTC moves that in minutes).
    # 0.5% over 4H is a meaningful, non-noise directional move.
    if change_pct >  0.005:   # +0.5% over 4H = clear uptrend
        return "BULL"
    if change_pct < -0.005:   # -0.5% over 4H = clear downtrend
        return "BEAR"
    return "NEUTRAL"


def _vpoc_confidence_boost(price: float, vpoc: Optional[float], direction: str) -> float:
    """
    Return a confidence addend (+0.0 to +0.06) based on how close the entry
    price is to the Volume Point of Control (VPOC — the session's highest-volume
    price level, acting as a mean-reversion magnet).

    Logic:
    - BUY near/below VPOC → buyer is entering at a proven value area → boost
    - SELL near/above VPOC → seller is fading from a proven resistance → boost
    - Entry far from VPOC on the wrong side → no boost (returns 0.0)
    """
    if vpoc is None or price <= 0:
        return 0.0
    dist_pct = abs(price - vpoc) / price   # distance as fraction of price
    is_long  = direction in ("BUY", "MEAN_REVERSAL_LONG")
```

```
    aligned = (is_long and price <= vpoc * 1.002) or (not is_long and price >= vpoc * 0.998)
    if not aligned:
        return 0.0

    # Graduated boost: 0.06 when at VPOC, falling to 0 at 0.3% away
    if dist_pct <= 0.003:
        return round(0.06 * (1.0 - dist_pct / 0.003), 4)
    return 0.0


async def _derivatives_gate(direction: str, deriv_context: dict) -> tuple[bool, str]:
    """
    Pre-entry veto using free institutional signals.
    Returns (should_trade: bool, reason: str).
    Called inside _compute_signal after strategy selection, before Bayesian.
    """
    is_long = direction in ("BUY", "MEAN_REVERSAL_LONG")

    oi    = deriv_context.get("open_interest", {})
    tt    = deriv_context.get("top_traders", {})
    opts  = deriv_context.get("options", {})
    flow  = deriv_context.get("taker_flow", {})

    crowd        = tt.get("crowd_signal", "NEUTRAL")
    opts_signal  = opts.get("options_signal", "NEUTRAL")
    oi_collapsing = oi.get("oi_collapsing", False)
    taker_imbalance = flow.get("taker_imbalance", 0.0)

    veto_reasons = []

    # VETO 1: Top traders crowded in SAME direction as our trade = no edge
    # 70%+ of whales already positioned same as us = crowded trade, squeeze risk
    if is_long and crowd == "CROWDED_LONG":
        veto_reasons.append("Top traders 70%+ LONG — crowded, no squeeze potential")
    if not is_long and crowd == "CROWDED_SHORT":
        veto_reasons.append("Top traders 70%+ SHORT — crowded, no squeeze potential")

    # VETO 2: Options market in FEAR while we want to buy
    # Institutions paying for downside puts = they expect lower prices
    if is_long and opts_signal == "FEAR":
        veto_reasons.append("Options FEAR — institutions hedging downside aggressively")

    # VETO 3: OI collapsing = deleveraging, not new trend forming
    if oi_collapsing:
        oi_mom = oi.get("oi_momentum_1h", 0.0)
        veto_reasons.append(f"OI collapsing {oi_mom:.1f}% in 1h — deleveraging, no new trend")

    if veto_reasons:
        return False, " | ".join(veto_reasons)

    # BOOST signals (logged but don't force entry)
    boosts = []
    if is_long and crowd == "CROWDED_SHORT":
        boosts.append("SHORT SQUEEZE SETUP — whales crowded short")
    if not is_long and crowd == "CROWDED_LONG":
        boosts.append("LONG SQUEEZE SETUP — whales crowded long")
    if abs(taker_imbalance) > 0.3:
        direction_match = (is_long and taker_imbalance > 0) or (not is_long and taker_imbalance < 0)
        if direction_match:
            boosts.append(f"Taker flow confirms: imbalance={taker_imbalance:.2f}")

    return True, " | ".join(boosts) if boosts else "No institutional conflict"


# ═══════════════════════════════════════════════════════════════════════════
# ═══ HMM REGIME CLASSIFIER (3-state, pure numpy) ═══════════════════════════
# ═══════════════════════════════════════════════════════════════════════════

class _HMMRegimeClassifier:
    """
    3-state Gaussian Hidden Markov Model regime detector.

    States:
        0 = RANGE     — low ATR, low |Z|, quiet tape
        1 = TREND     — moderate/high ATR, directional bias, louder tape
        2 = VOLATILE  — high ATR, extreme |Z|, erratic — maps to NEUTRAL

    Features (per observation):
```

```
    f0 = atr_pct       (0 – 0.02)
    f1 = abs(z_score)  (0 – 4)
    f2 = tape_binary   (0 = NORMAL, 1 = SCREAMING)

Upgrades (Apr 2026):
    1. Forward algorithm → posterior probability vector P(state | obs_1...T)
    2. 70% confidence gate — low-confidence ⇒ NEUTRAL (no action)
    3. 3-candle hysteresis — regime change requires 3 consecutive agreements
"""

# --- Emission means (μ) per state × feature ----------------------
# Features: [atr_pct, |z_score|, tape_binary, atr_pct_rank]
#   f0 = atr_pct       (0 – 0.02)  raw ATR as fraction of price
#   f1 = abs(z_score)  (0 – 4)
#   f2 = tape_binary   (0 = NORMAL, 1 = SCREAMING)
#   f3 = atr_pct_rank  (0 – 1)   percentile rank in 30-day window (P2)
_MU = np.array([
    [0.003, 0.8, 0.05, 0.20],  # RANGE
    [0.007, 1.5, 0.30, 0.55],  # TREND — retained at 0.007 per CPO
    [0.014, 2.5, 0.65, 0.85],  # VOLATILE
], dtype=float)
_SIGMA = np.array([
    [0.0010, 0.40, 0.10, 0.10],  # RANGE — COMPRESSED (was 0.0015,0.6,0.15,0.15)
    [0.0030, 0.80, 0.25, 0.20],  # TREND
    [0.0050, 1.00, 0.30, 0.15],  # VOLATILE
], dtype=float)

# --- Transition matrix (rows = from-state, cols = to-state) -------
# 80% persistence, 10% to each neighbour state
_A = np.array([
    [0.82, 0.12, 0.06],
    [0.10, 0.80, 0.10],
    [0.06, 0.12, 0.82],
], dtype=float)

# --- Initial state distribution ----------------------------------
_PI = np.array([0.50, 0.35, 0.15], dtype=float)

# State labels (index → regime string)
_LABELS = ["RANGE", "TREND", "NEUTRAL"]  # VOLATILE maps to NEUTRAL

# Confidence & hysteresis thresholds
MIN_CONFIDENCE        = 0.60
HYSTERESIS_CANDLES    = 2     # FIXED: 30-min lag (was 3 = 45-min lag)
FAST_TRACK_CONFIDENCE = 0.88  # NEW: 1-candle commit at very high confidence

def __init__(self, window: int = 60, update_every: int = 50):
    self._window    = window
    self._update_n  = update_every
    self._obs_buf   = deque(maxlen=200)  # circular feature history
    self._cycle     = 0           # count calls since last param update
    # Mutable copies so online-update can adjust them
    self._mu    = self._MU.copy()
    self._sigma = self._SIGMA.copy()
    # Hysteresis state
    self._committed_regime = "RANGE"   # currently committed regime
    self._candidate_regime = "RANGE"   # regime the HMM is suggesting
    self._candidate_streak = 0         # consecutive candles suggesting candidate

    self._persist_path = os.environ.get("HMM_STATE_PATH", "/tmp/quad_hmm_state.json")
    self._load_state()

def _save_state(self):
    import threading
    import json
    import time
    data = {
        "mu": self._mu.tolist(),
        "sigma": self._sigma.tolist(),
        "saved_at": time.time(),
    }

    def _write_firestore(payload: dict) -> None:
        try:
            import json
            fs_payload = payload.copy()
            fs_payload["mu"] = json.dumps(fs_payload.get("mu", []))
            fs_payload["sigma"] = json.dumps(fs_payload.get("sigma", []))
```

```
            from bot.heartbeat import get_db
            db = get_db()
            if db is not None:
                db.collection("botState").document("hmmEngine").set(fs_payload)
                logger.debug("[HMM] State saved to Firestore ■")
        except Exception as e:
            logger.warning(f"[HMM] Firestore state save failed: {e}")

    threading.Thread(
        target=_write_firestore,
        args=(dict(data),),
        daemon=True,
        name="HMMEngine-FSWrite",
    ).start()

    try:
        with open(self._persist_path, "w") as f:
            json.dump(data, f)
    except Exception as e:
        pass

def _load_state(self):
    import json
    import time
    try:
        from bot.heartbeat import get_db
        db = get_db()
        if db is not None:
            doc = db.collection("botState").document("hmmEngine").get()
            if doc.exists:
                data = doc.to_dict()
                mu_val = data.get("mu")
                sig_val = data.get("sigma")
                if isinstance(mu_val, str): mu_val = json.loads(mu_val)
                if isinstance(sig_val, str): sig_val = json.loads(sig_val)
                self._mu = np.array(mu_val if mu_val is not None else self._MU.tolist())
                self._sigma = np.array(sig_val if sig_val is not None else self._SIGMA.tolist())
                logger.info("[HMM] ■ State loaded from Firestore")
                return
    except Exception as e:
        logger.warning(f"[HMM] Firestore state load failed: {e} — falling back to /tmp/")

    try:
        import os
        if os.path.exists(self._persist_path):
            with open(self._persist_path) as f:
                data = json.load(f)
            self._mu = np.array(data.get("mu", self._MU.tolist()))
            self._sigma = np.array(data.get("sigma", self._SIGMA.tolist()))
            logger.info("[HMM] ■■ State loaded from /tmp/")
            return
    except Exception as e:
        pass

# ------------------------------------------------------------------
def _gaussian_log_prob(self, obs: np.ndarray) -> np.ndarray:
    """Log P(obs | state) for all 3 states. obs shape = (F,)."""
    log_probs = np.zeros(3)
    for s in range(3):
        diff    = obs - self._mu[s]
        log_p   = -0.5 * np.sum((diff / np.maximum(self._sigma[s], 1e-9)) ** 2)
        log_p  -= np.sum(np.log(np.maximum(self._sigma[s], 1e-9)))
        log_probs[s] = log_p
    return log_probs

# ------------------------------------------------------------------
@staticmethod
def _logsumexp(a: np.ndarray) -> float:
    """Numerically stable log-sum-exp."""
    a_max = np.max(a)
    if not np.isfinite(a_max):
        return a_max
    return a_max + np.log(np.sum(np.exp(a - a_max)))

# ------------------------------------------------------------------
def _forward(self, obs_seq: np.ndarray) -> np.ndarray:
    """
    Scaled forward algorithm — returns posterior P(state_T | obs_1...T).
```

```
    Unlike Viterbi (which gives the single most-likely STATE SEQUENCE),
    the forward algorithm gives the marginal probability of each state
    at the final timestep, integrating over all possible state paths.
    This is mathematically correct for regime uncertainty quantification.
    """
    T    = len(obs_seq)
    n_s  = 3
    log_A  = np.log(np.maximum(self._A, 1e-300))
    log_pi = np.log(np.maximum(self._PI, 1e-300))

    # log_alpha[t, s] = log P(o_1...o_t, state_t = s)
    log_alpha = np.full((T, n_s), -np.inf)
    log_alpha[0] = log_pi + self._gaussian_log_prob(obs_seq[0])

    for t in range(1, T):
        log_emit = self._gaussian_log_prob(obs_seq[t])
        for j in range(n_s):
            # Sum over all possible previous states (marginalise)
            log_alpha[t, j] = self._logsumexp(
                log_alpha[t - 1] + log_A[:, j]
            ) + log_emit[j]

    # Normalise to get posterior at time T
    log_evidence = self._logsumexp(log_alpha[-1])
    log_posterior = log_alpha[-1] - log_evidence
    posterior = np.exp(log_posterior)

    # Safety: ensure sums to 1.0 (numerical precision)
    posterior = np.nan_to_num(posterior, nan=0.0, posinf=0.0, neginf=0.0)
    posterior = np.maximum(posterior, 0.0)
    total = posterior.sum()
    if total > 0:
        posterior /= total
    else:
        posterior = np.array([0.50, 0.35, 0.15])  # fallback to prior

    return posterior

# ------------------------------------------------------------------
def _viterbi(self, obs_seq: np.ndarray) -> np.ndarray:
    """Pure-numpy Viterbi — returns most-likely state sequence."""
    T, _   = obs_seq.shape
    n_s    = 3
    log_A  = np.log(np.maximum(self._A, 1e-300))
    log_pi = np.log(np.maximum(self._PI, 1e-300))

    delta     = np.full((T, n_s), -np.inf)
    psi       = np.zeros((T, n_s), dtype=int)

    delta[0]  = log_pi + self._gaussian_log_prob(obs_seq[0])

    for t in range(1, T):
        log_emit = self._gaussian_log_prob(obs_seq[t])
        for j in range(n_s):
            trans    = delta[t - 1] + log_A[:, j]
            psi[t, j]   = int(np.argmax(trans))
            delta[t, j] = np.max(trans) + log_emit[j]

    # Back-track
    states      = np.zeros(T, dtype=int)
    states[-1]  = int(np.argmax(delta[-1]))
    for t in range(T - 2, -1, -1):
        states[t] = psi[t + 1, states[t + 1]]
    return states

# ------------------------------------------------------------------
def _online_update(self):
    """
    Lite Baum-Welch: re-estimate μ and σ from recent observations
    using soft-assignment (posterior from Viterbi hard assignment).
    Only runs every `update_every` cycles to avoid overhead.
    """
    if len(self._obs_buf) < 20:
        return
    obs = np.array(list(self._obs_buf)[-self._window:], dtype=float)
    states = self._viterbi(obs)
    for s in range(3):
        mask = states == s
```

```
        if mask.sum() >= 3:
            self._mu[s]    = obs[mask].mean(axis=0)
            self._sigma[s] = np.maximum(obs[mask].std(axis=0), 1e-4)

    self._save_state()

# ---------------------------------------------------------------------
def classify(self, atr_pct: float, z_score: float, tape: str,
             atr_pct_rank: float = 0.5) -> dict:
    """
    Main entry: add one observation and return regime probability vector.

    Args:
        atr_pct:      ATR as fraction of price (e.g. 0.007 = 0.7%)
        z_score:      VWAP Z-score
        tape:         'SCREAMING' | 'NORMAL'
        atr_pct_rank: ATR percentile rank in 30-day rolling window [0,1] (P2)

    Returns dict with:
        regime:     str   — committed regime label (with hysteresis)
        confidence: float — posterior probability of the committed regime
        p_range:    float — P(RANGE | observations)
        p_trend:    float — P(TREND | observations)
        p_volatile: float — P(VOLATILE | observations)
        raw_regime: str   — instantaneous HMM output (before hysteresis)
    """
    obs = np.array([
        float(np.clip(atr_pct,      0.0,  0.03)),
        float(np.clip(abs(z_score), 0.0,  4.0)),
        1.0 if tape == "SCREAMING" else 0.0,
        float(np.clip(atr_pct_rank, 0.0,  1.0)),  # P2: 4th feature
    ], dtype=float)

    self._obs_buf.append(obs)
    self._cycle += 1

    if self._cycle % self._update_n == 0:
        self._online_update()

    # Need at least 3 observations for a meaningful forward pass
    n_obs = min(len(self._obs_buf), self._window)
    seq   = np.array(list(self._obs_buf)[-n_obs:], dtype=float)

    if len(seq) < 3:
        # Fall back to simple thresholds while warming up
        if atr_pct > 0.01 and tape == "SCREAMING":
            raw = "TREND"
        elif abs(z_score) > 2.5:
            raw = "NEUTRAL"
        else:
            raw = "RANGE"
        return {
            "regime": raw, "confidence": 0.50,
            "p_range": 0.33, "p_trend": 0.33, "p_volatile": 0.33,
            "raw_regime": raw,
        }

    # ■■ Forward algorithm: posterior probability vector ■■■■■■■■■■■■
    posterior = self._forward(seq)
    best_state = int(np.argmax(posterior))
    raw_label  = self._LABELS[best_state]
    raw_conf   = float(posterior[best_state])

    # ■■ Hysteresis: 3-candle debounce on regime transitions ■■■■■■■
    # Prevents flickering at regime boundaries (e.g. TREND→RANGE→TREND
    # on consecutive cycles when the posterior is near 50/50).
    if raw_label == self._candidate_regime:
        self._candidate_streak += 1
    else:
        self._candidate_regime = raw_label
        self._candidate_streak = 1

    # Commit the new regime only if it's been consistent for N candles
    # AND meets the confidence threshold
    _fast = (raw_conf >= self.FAST_TRACK_CONFIDENCE
             and self._candidate_regime != self._committed_regime)
    _normal = (self._candidate_streak >= self.HYSTERESIS_CANDLES
               and raw_conf >= self.MIN_CONFIDENCE
```

```
                    and self._candidate_regime != self._committed_regime)

        if _fast or _normal:
            old = self._committed_regime
            self._committed_regime = self._candidate_regime
            logger.info(
                f"[HMM] Regime TRANSITION ({'FAST' if _fast else 'normal'}): "
                f"{old} → {self._committed_regime} "
                f"(conf={raw_conf:.1%}, streak={self._candidate_streak})"
            )

        # The committed regime's confidence is its actual posterior probability
        committed_idx = self._LABELS.index(self._committed_regime)
        committed_conf = float(posterior[committed_idx])

        logger.debug(
            f"[HMM] raw={raw_label}({raw_conf:.0%}) committed={self._committed_regime}"
            f"({committed_conf:.0%}) streak={self._candidate_streak} | "
            f"P=[R:{posterior[0]:.0%} T:{posterior[1]:.0%} V:{posterior[2]:.0%}]"
        )

        return {
            "regime":      self._committed_regime,
            "confidence":  committed_conf,
            "p_range":     float(posterior[0]),
            "p_trend":     float(posterior[1]),
            "p_volatile":  float(posterior[2]),
            "raw_regime":  raw_label,
        }


# Module-level singleton — persists observations across cycles
_hmm_classifier = _HMMRegimeClassifier(window=60, update_every=200)

# Derivatives context for institutional signals
_derivatives_ctx = DerivativesContext(symbol=FEED_SYMBOL)



def _detect_regime(
    metrics: Dict[str, Any],
    buy_walls: List[float],
    sell_walls: List[float],
    quant,  # PHASE-1.3: Reference to QuantEngine for stateful tracking
    sweep: Optional[str] = None,
) -> str:
    """
    HMM-based regime classifier with probabilistic output.

    Upgrades:
        1. Forward algorithm → P(state | all observations) instead of Viterbi hard label
        2. 70% confidence gate  → below threshold defaults to NEUTRAL
        3. 3-candle hysteresis  → regime change requires 3 consecutive agreements

    Wall proximity is checked FIRST as a hard override — being within 0.3%
    of a significant liquidity wall is always a LIQUIDITY regime regardless
    of ATR/Z/tape state (the HMM doesn't model wall proximity).

    PHASE-1.3: LIQUIDITY override now requires:
        - Nearest wall is within WALL_PROXIMITY of price
        - Wall size is ≥ 3× the median order book level (significant wall)
        - LIQUIDITY regime capped at 3 consecutive cycles without sweep confirmation

    Side-effect: injects regime_confidence, regime_probs into `metrics` dict
    so downstream consumers (Bayesian fusion, signal attribution) can use them.
    """

    price   = metrics["price"]
    z       = metrics["zScore"]
    tape    = metrics["tapeSpeed"]
    atr_pct = metrics["atr_pct"]

    # PHASE-1.3: New proximity threshold and significance filter
    atr_val = metrics.get("atr", price * 0.001)
    WALL_PROXIMITY = max(0.3 * atr_val / price, 0.003)
    near_wall = any(
        abs(price - w) / price <= WALL_PROXIMITY
        for w in (buy_walls[:1] + sell_walls[:1])
```

```
    )

    if near_wall:
        # Wall significance filter: only override if wall is ≥ 3× median level
        all_levels = list(metrics.get("bid_depths", {}).values()) + list(metrics.get("ask_depths", {}).values())
        median_level = float(np.median(all_levels)) if all_levels else 0.0

        # Check which wall is near
        nearest_buy = buy_walls[0] if buy_walls else None
        nearest_sell = sell_walls[0] if sell_walls else None
        near_wall_size = 0.0
        near_wall_is_buy = False
        if nearest_buy and abs(price - nearest_buy) / price <= WALL_PROXIMITY:
            near_wall_size = metrics.get("bid_depths", {}).get(nearest_buy, 0.0)
            near_wall_is_buy = True
        elif nearest_sell and abs(price - nearest_sell) / price <= WALL_PROXIMITY:
            near_wall_size = metrics.get("ask_depths", {}).get(nearest_sell, 0.0)

        wall_is_significant = (median_level > 0 and near_wall_size >= median_level * 3.0)

        if wall_is_significant:
            metrics["regime_confidence"] = 1.0
            metrics["regime_probs"] = {"RANGE": 0.0, "TREND": 0.0, "NEUTRAL": 0.0, "LIQUIDITY": 1.0}
            quant._liquidity_consecutive += 1
            if quant._liquidity_consecutive > 3 and sweep is None:
                logger.info("[Regime] LIQUIDITY cap reached — falling through to HMM")
                quant._liquidity_consecutive = 0
            else:
                return "LIQUIDITY"
        else:
            # Wall is near but not significant — reset counter and fall through
            quant._liquidity_consecutive = 0
    else:
        quant._liquidity_consecutive = 0

    # HMM classification → probability vector (P2: pass 4th feature)
    atr_pct_rank = metrics.get("atr_pct_rank", 0.5)
    hmm_result = _hmm_classifier.classify(atr_pct, z, tape, atr_pct_rank)

    # HIGH-2 FIX: REGIME_REMAP removed — it was dead code (HMM state 2 is "NEUTRAL"
    # in _LABELS, never "VOLATILE"). Keeping it was a hazard: renaming the label
    # would silently reroute all high-vol periods to TREND strategy.
    regime = hmm_result["regime"]
    confidence = hmm_result["confidence"]

    # Inject probabilities into metrics for downstream observability
    metrics["regime_confidence"] = confidence
    metrics["regime_probs"] = {
        "RANGE":    hmm_result["p_range"],
        "TREND":    hmm_result["p_trend"],
        "VOLATILE": hmm_result["p_volatile"],
    }

    # ■■ 70% Confidence Gate ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    # If the HMM isn't confident enough, fall back to NEUTRAL.
    # This prevents the bot from committing to TREND or MEAN_REVERSION
    # when the regime is genuinely ambiguous (e.g. P(TREND)=0.52).
    if confidence < _HMMRegimeClassifier.MIN_CONFIDENCE:
        logger.info(
            f"[HMM] Regime={hmm_result['raw_regime']}→NEUTRAL (conf={confidence:.0%} "
            f"< {_HMMRegimeClassifier.MIN_CONFIDENCE:.0%} gate) | "
            f"P=[R:{hmm_result['p_range']:.0%} T:{hmm_result['p_trend']:.0%} "
            f"V:{hmm_result['p_volatile']:.0%}]"
        )
        return "NEUTRAL"

    logger.info(
        f"[HMM] Regime={regime} conf={confidence:.0%} | "
        f"P=[R:{hmm_result['p_range']:.0%} T:{hmm_result['p_trend']:.0%} "
        f"V:{hmm_result['p_volatile']:.0%}] | "
        f"atr={atr_pct:.3%} z={z:.2f} tape={tape}"
    )
    return regime



# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
# ■■ STAGE 3 — LIQUIDITY SWEEP DETECTION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

def _detect_liquidity_sweep(metrics: Dict[str, Any],
                            buy_walls: List[float],
                            sell_walls: List[float],
                            candle_history: list) -> Optional[str]:
    if len(candle_history) < 2 or not sell_walls or not buy_walls:
        return None, None, None

    price = metrics["price"]
    nearest_sell = sell_walls[0]
    nearest_buy  = buy_walls[0]

    # FIX (CandleGate root cause): Check LIVE candle (-1) FIRST, then prev (-2).
    # The original order [(-2,"prev"),(-1,"live")] always returned the prev candle's
    # stale open-time (25+ min old) even when the live candle also had the wick above
    # the wall. CandleGate then permanently blocked every entry as >945s stale.
    #
    # NEW RULE:
    #   Live candle sweep  -> sweep_ts = time.time()        (detected RIGHT NOW, age=0s)
    #   Prev candle sweep  -> sweep_ts = open_time + 900s   (approx when candle closed)
    # A live-candle sweep will ALWAYS pass the CandleGate freshness check.
    # FIX-P5: Per-candle sweep deduplication.
    # The same wick on the same candle was re-firing every 15s cycle (audit Problem 5).
    # Only allow one sweep signal per unique candle open-timestamp.
    global _LAST_FIRED_SWEEP_CANDLE_TS
    for idx, label in [(-1, "live"), (-2, "prev")]:
        candle = candle_history[idx]
        candle_open_ts = float(candle.get("time", 0.0))

        if candle_open_ts > 0 and candle_open_ts == _LAST_FIRED_SWEEP_CANDLE_TS:
            logger.debug(
                f"[Sweep] Dedup: candle ts={candle_open_ts:.0f} already fired. "
                "Suppressing repeat sweep from same candle."
            )
            continue

        if candle["high"] > nearest_sell and price < nearest_sell:
            logger.info(f"[Sweep] ABOVE_HIGHS at {nearest_sell:.2f} (wick={candle['high']:.2f})")
            sweep_ts = time.time() if label == "live" else float(candle["time"]) + 900.0
            _LAST_FIRED_SWEEP_CANDLE_TS = candle_open_ts
            return "ABOVE_HIGHS", sweep_ts, float(candle["high"])

        if candle["low"] < nearest_buy and price > nearest_buy:
            logger.info(f"[Sweep] BELOW_LOWS at {nearest_buy:.2f} (wick={candle['low']:.2f})")
            sweep_ts = time.time() if label == "live" else float(candle["time"]) + 900.0
            _LAST_FIRED_SWEEP_CANDLE_TS = candle_open_ts
            return "BELOW_LOWS", sweep_ts, float(candle["low"])

    return None, None, None


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# ■■ STAGE 4 — STRATEGY LAYER ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

def _strategy_trend(metrics: Dict[str, Any]) -> Optional[str]:
    bayes      = metrics["bayesianPosterior"]
    ofi        = metrics["ofi"]
    cvd        = metrics["cvd"]
    dominant   = metrics["tapeDominant"]
    tape_speed = metrics.get("tapeSpeed", "NORMAL")  # Added: differentiate SCREAMING vs NORMAL tape
    # Z-06 Log-Return Z (P2): velocity-based momentum confirmation
    z_ret = metrics.get("zScore_ret", 0.0)

    score = 0.0
    if bayes > 0.65:    score += 2.0
    elif bayes > 0.55:  score += 1.0
    elif bayes < 0.35:  score -= 2.0
    elif bayes < 0.45:  score -= 1.0

    # SIG-2 FIX: OFI thresholds updated from old ±100 scale to new tanh ±1 scale.
    # Old: ofi > 20 / ofi > 8 were unreachable (max OFI after pipeline = 1.0).
    if ofi > 0.3:        score += 1.5
    elif ofi > 0.15:     score += 0.75
    elif ofi < -0.3:     score -= 1.5
    elif ofi < -0.15:    score -= 0.75
```

```
    if cvd > 0:          score += 1.0
    elif cvd < 0:        score -= 1.0

    # Require SCREAMING tape for full +1.0 boost; NORMAL tape only earns +0.5
    # Prevents entering trend trades on weak marginal tape flow (Patch #8)
    if "BUY" in dominant:
        score += 1.0 if tape_speed == "SCREAMING" else 0.5
    elif "SELL" in dominant:
        score -= 1.0 if tape_speed == "SCREAMING" else 0.5

    # P2: Z-06 momentum velocity score modifier (max +/-0.75)
    if z_ret > 1.5:    score += 0.75
    elif z_ret > 0.8:  score += 0.30
    elif z_ret < -1.5: score -= 0.75
    elif z_ret < -0.8: score -= 0.30

    ofi_bull = ofi > 0.15
    cvd_bull = cvd > 0
    tape_bull = "BUY" in dominant
    micro_confirms_bull = sum([ofi_bull, cvd_bull, tape_bull])

    ofi_bear = ofi < -0.15
    cvd_bear = cvd < 0
    tape_bear = "SELL" in dominant
    micro_confirms_bear = sum([ofi_bear, cvd_bear, tape_bear])

    if score >= 1.5:
        # Intended LONG
        ofi_cvd_aligned = (ofi > 0.15 and cvd > 0)
        if micro_confirms_bull < 2 and score < 3.0 and not ofi_cvd_aligned:
            BOT_STATS["gate_stats"]["micro_confirms_failed"] = BOT_STATS["gate_stats"].get("micro_confirms_failed", 0)
            logger.info(f"[TrendStrategy] score={score:+.2f} rejected LONG (micro_confirms_bull={micro_confirms_bull}/3
            return None
        return "BUY"

    if score <= -1.5:
        # Intended SHORT
        ofi_cvd_aligned = (ofi < -0.15 and cvd < 0)
        if micro_confirms_bear < 2 and score > -3.0 and not ofi_cvd_aligned:
            BOT_STATS["gate_stats"]["micro_confirms_failed"] = BOT_STATS["gate_stats"].get("micro_confirms_failed", 0)
            logger.info(f"[TrendStrategy] score={score:+.2f} rejected SHORT (micro_confirms_bear={micro_confirms_bear}/
            return None
        return "SELL"

    return None



def _strategy_mean_reversion(metrics: Dict[str, Any], z_threshold: float = 1.5) -> Optional[str]:
    """FIX NEW-C3: Dynamic RSI gates — adaptive to ATR rank."""
    z = metrics.get("zScore", 0.0)
    rsi = metrics.get("rsi", 50.0)
    atr_rank = metrics.get("atr_pct_rank", 0.5)
    if atr_rank < 0.5:
        rsi_long_gate = 55.0
        rsi_short_gate = 45.0
    else:
        rsi_long_gate = 42.0
        rsi_short_gate = 58.0
    if z >= z_threshold and rsi > rsi_short_gate:
        return "MEAN_REVERSAL_SHORT"
    if z <= -z_threshold and rsi < rsi_long_gate:
        return "MEAN_REVERSAL_LONG"
    return None


def _strategy_liquidity_sweep(sweep: str, metrics: Dict[str, Any]) -> Optional[str]:
    ofi      = metrics["ofi"]
    cvd      = metrics["cvd"]
    dominant = metrics["tapeDominant"]
    z        = metrics.get("zScore", 0.0)
    rsi      = metrics.get("rsi", 50.0)

    # P1-1 FIX: Z+RSI overbought/oversold gate on sweep entries.
    # The Apr-23 audit caught Z=+2.67, RSI=70 entering a BELOW_LOWS BUY sweep —
    # price was already statistically overbought; the sweep was a continuation,
    # not a reversal. Block these momentum-chase entries at the strategy level.
```

```
    if sweep == "BELOW_LOWS" and z > 2.0 and rsi > 65:
        logger.info(
            f"[SweepGate] BUY sweep BLOCKED — Z={z:.2f} RSI={rsi:.1f} "
            f"(price already overbought, sweep likely continuation not reversal)"
        )
        return None
    if sweep == "ABOVE_HIGHS" and z < -2.0 and rsi < 35:
        logger.info(
            f"[SweepGate] SELL sweep BLOCKED — Z={z:.2f} RSI={rsi:.1f} "
            f"(price already oversold, sweep likely continuation not reversal)"
        )
        return None

    # PHASE-2.1 FIX: Lower confirms threshold 3/3 → 2/3 for sweep entries.
    # 3/3 was too restrictive — all three micro-confirmations rarely align simultaneously
    # in live markets. 2/3 requires OFI+CVD+tape to agree, still enforcing quality.
    if sweep == "ABOVE_HIGHS":
        confirms = sum([
            ofi < -0.15,
            cvd < 0,
            "SELL" in dominant,
        ])
        if confirms >= 2:
            logger.info(f"[SweepStrat] SELL after ABOVE_HIGHS (confirms={confirms}/3)")
            return "SELL"

    if sweep == "BELOW_LOWS":
        confirms = sum([
            ofi > 0.15,
            cvd > 0,
            "BUY" in dominant,
        ])
        if confirms >= 2:
            logger.info(f"[SweepStrat] BUY after BELOW_LOWS (confirms={confirms}/3)")
            return "BUY"

    BOT_STATS["gate_stats"]["sweep_confirms_failed"] = BOT_STATS["gate_stats"].get("sweep_confirms_failed", 0) + 1
    return None


# ═══════════════════════════════════════════════════════════════════════════
# ══ STAGE 5 — BAYESIAN SIGNAL FUSION ═══════════════════════════════
# ═══════════════════════════════════════════════════════════════════════════

def _bayesian_fusion(metrics: Dict[str, Any], direction: str, regime: str, is_sweep: bool = False) -> float:
    """
    Stage 5: Macro Bayesian Logic
    Updates the prior confidence (from QuantEngine) with current directional evidence.
    Returns: Confidence score (0-1) in the *target signal direction*.

    Improvements over original:
    - Sweep Neutralizer is context-aware: higher floor when OFI/CVD confirm direction
    - Metrics extracted before Neutralizer (needed for context)
    - Mid-range OFI tier (5-10) now gets a partial boost (was binary ≥10 only)
    - CVD delta (rate-of-change) used: recovering CVD = bullish even when absolute CVD < 0
    - Scaled skew boost: gradual 0→15% as |skew| 0.05→0.30 (was never triggered at > 0.5)
    """
    # 1. Start with the prior confidence from Quant Engine Stage 4 (which is P-Bull)
    p_bull_prior = metrics.get("bayesianPosterior", 0.5)
    is_long = direction in ("BUY", "MEAN_REVERSAL_LONG")

    # 2. Base Signal Confidence
    p_signal_prior = p_bull_prior if is_long else (1.0 - p_bull_prior)

    # --- Extract all market metrics before Sweep Neutralizer (needed for context check) ---
    ofi       = metrics.get("ofi", 0.0)
    cvd       = metrics.get("cvd", 0.0)
    cvd_delta = metrics.get("cvd_delta", 0.0)  # positive = buy pressure returning
    skew      = metrics.get("skewness", 0.0)
    dominant  = metrics.get("tapeDominant", "BALANCED")
    tape      = metrics.get("tapeSpeed", "NORMAL")
    rsi       = metrics.get("rsi", 50.0)

    # [FIX-P2] SWEEP NEUTRALIZER: Only apply confidence floor when Bayesian SUPPORTS direction.
    # The old unconditional floor of 0.55 overrode a Bayes=35% (bearish) BUY signal,
    # making the Bayesian posterior entirely decorative. The floor now only activates
    # when p_signal_prior >= 0.45 (i.e. Bayes is at least neutral on the direction).
    # A sub-0.45 Bayesian reading means the core signal says no — let it fail the threshold.
```

```
if is_sweep:
    bayes_supports_direction = p_signal_prior >= 0.45
    if bayes_supports_direction:
        strong_confirm = (
            (is_long  and ofi >  0.3 and cvd_delta > 0) or
            (not is_long and ofi < -0.3 and cvd_delta < 0)
        )
        mild_confirm = (
            (is_long  and (ofi > 0.15 or cvd_delta > 0)) or
            (not is_long and (ofi < -0.15 or cvd_delta < 0))
        )
        if strong_confirm:
            p_signal_prior = max(0.65, p_signal_prior)   # strong OFI+CVD: floor at 0.65
        elif mild_confirm:
            p_signal_prior = max(0.58, p_signal_prior)   # mild confirm: floor at 0.58
        else:
            p_signal_prior = max(0.55, p_signal_prior)   # no extra confirm: floor at 0.55
    # If Bayesian < 0.45 (against direction), no floor — let it die at threshold

# 3. Transform to Odds
# Clip to avoid division by zero/infinity during transformation
p_signal_prior = max(0.01, min(0.99, p_signal_prior))
odds = p_signal_prior / (1.0 - p_signal_prior)

# 4. Flow Multipliers (OFI/CVD/CVD-delta) — direction-support check
# FIX: OFI is now in (-1, +1) range from tanh pipeline
# Tiered OFI: >0.3 = strong, >0.15 = moderate. CVD delta captures recovering buy pressure
flow_factor = 1.0
if is_long:
    if ofi > 0.3:           flow_factor *= 1.25
    elif ofi > 0.15:         flow_factor *= 1.10   # Mid-range OFI: partial boost
    if cvd > 0:              flow_factor *= 1.15
    elif cvd_delta > 100:    flow_factor *= 1.08   # CVD recovering → mild bullish tailwind
    elif cvd_delta < -100:   flow_factor *= 0.90   # CVD accelerating down → headwind
else:  # SHORT signal
    if ofi < -0.3:           flow_factor *= 1.25
    elif ofi < -0.15:         flow_factor *= 1.10   # Mid-range OFI: partial boost
    if cvd < 0:               flow_factor *= 1.15
    elif cvd_delta < -100:   flow_factor *= 1.08   # CVD dropping → mild bearish tailwind
    elif cvd_delta > 100:    flow_factor *= 0.90   # CVD recovering → headwind for shorts

odds *= flow_factor

# 5. Contextual Oscillator check — Regime-aware
osc_factor = 1.0
if regime == "TREND":
    # Trend continuation: RSI in signal direction confirms momentum
    if is_long and rsi > 65:       osc_factor *= 1.15
    elif not is_long and rsi < 35: osc_factor *= 1.15
elif regime == "LIQUIDITY":
    # Sweep reversal: Overextended RSI supports a bounce/rejection
    if not is_long and rsi > 60:   osc_factor *= 1.60  # Overbought strongly helps SELL
    elif is_long and rsi < 40:     osc_factor *= 1.60  # Oversold strongly helps BUY

    # Momentum Chase Penalty: avoid entry if RSI already buried too deep.
    # SWEEP EXCEPTION: a BELOW_LOWS BUY sweep on overbought RSI is a REVERSAL,
    # not a momentum chase. The RSI overextension actually CONFIRMS the sweeping
    # condition. Skip this penalty entirely for sweep strategies.
    if not is_long and rsi < 32 and not is_sweep:   osc_factor *= 0.75
    if is_long and rsi > 68 and not is_sweep:       osc_factor *= 0.75

odds *= osc_factor

# 6. Tape Check
if tape == "SCREAMING":
    if is_long and "BUY" in dominant:        odds *= 1.20
    elif not is_long and "SELL" in dominant: odds *= 1.20

# 7. [IMPROVED] Scaled Skew boost — proportional to skew magnitude.
# Old: binary cutoff at ±0.5 (never reached; max observed was 0.142).
# New: scales from 1.0→1.15 as |skew| goes from 0.05→0.30.
if abs(skew) >= 0.05:
    skew_magnitude = min(abs(skew), 0.30) / 0.30   # normalise 0→1, cap at |0.30|
    skew_boost     = 1.0 + skew_magnitude * 0.15   # 1.00 → 1.15
    if is_long and skew > 0:
        odds *= skew_boost
    elif not is_long and skew < 0:
        odds *= skew_boost
```

```
    # 8. Convert back to probability — MUST happen before regime blending (Fix #1: UnboundLocalError)
    p_final = odds / (1.0 + odds)

    # 9. [P1] Regime-Weighted Posterior Blending (HMM Spec Apr 2026)
    # P_adjusted = hmm_conf * regime_prior + (1 - hmm_conf) * base_posterior
    # High HMM confidence → trust per-regime historical win rate more.
    # Low HMM confidence  → trust raw Bayesian signal posterior more.
    regime_priors = metrics.get("_regime_priors", {})
    regime_samples = metrics.get("_regime_samples", {})
    if regime_priors and regime in regime_priors:
        n_regime_trades = int(regime_samples.get(regime, 0))
        MIN_REGIME_HISTORY = 10
        if n_regime_trades >= MIN_REGIME_HISTORY:
            hmm_conf = metrics.get("regime_confidence", 0.5)
            regime_prior = regime_priors[regime]
            p_final = float(np.clip(
                hmm_conf * regime_prior + (1.0 - hmm_conf) * p_final,
                0.0, 1.0
            ))

    return float(p_final)


# ═══════════════════════════════════════════════════════════════════════════
# ═══ STAGE 6 ★ NEW — ULIS/ALDE CONFIRMATION GATE ═══════════════════════
# ═══════════════════════════════════════════════════════════════════════════

def _apply_ulis_gate(
    metrics: Dict[str, Any],
    candle_history: list,
    feed_state,
    raw_direction: str,
    confidence: float,
) -> Tuple[bool, float, str]:
    """
    Run the ALDE+ULIS hybrid verdict engine as Stage 6.

    Returns:
        (should_trade, adjusted_confidence, ulis_verdict_str)
    """
    if not ULIS_GATE_ENABLED:
        return True, confidence, "GATE_DISABLED"

    ulis = compute_ulis_verdict(
        metrics=metrics,
        candles=candle_history,
        bids_dict=feed_state.bids,
        asks_dict=feed_state.asks,
    )

    verdict_str = ulis["verdict"]
    BOT_STATS["last_ulis"] = verdict_str

    # Veto conditions
    if not ulis["should_trade"]:
        logger.warning(f"[ULIS] VETO — {verdict_str} | {ulis['regime_label']}")
        return False, 0.0, verdict_str

    # Direction alignment check
    is_long = raw_direction in ("BUY", "MEAN_REVERSAL_LONG")
    ulis_bullish = verdict_str in ("STRONG_LONG", "LONG")
    ulis_bearish = verdict_str in ("STRONG_SHORT", "SHORT")

    BAYES_OVERRIDE_THRESHOLD = 0.78
    if is_long and ulis_bearish:
        if confidence >= BAYES_OVERRIDE_THRESHOLD:
            logger.info(f"[ULIS] Bayesian override ({confidence:.2%} >= {BAYES_OVERRIDE_THRESHOLD:.0%}) - proceeding de
            confidence *= 0.92
        else:
            logger.warning(f"[ULIS] Direction conflict - bot=LONG, ULIS={verdict_str}. Skipping.")
            return False, 0.0, verdict_str

    if not is_long and ulis_bullish:
        if confidence >= BAYES_OVERRIDE_THRESHOLD:
            logger.info(f"[ULIS] Bayesian override ({confidence:.2%} >= {BAYES_OVERRIDE_THRESHOLD:.0%}) - proceeding de
            confidence *= 0.92
        else:
            logger.warning(f"[ULIS] Direction conflict - bot=SHORT, ULIS={verdict_str}. Skipping.")
```

```
            return False, 0.0, verdict_str


    # --- PROPER ALIGNMENT FRAMEWORK: Triple Alignment Gate ---
    rsi = metrics.get("rsi", 50.0)
    ofi = metrics.get("ofi", 0.0)
    atr_pct = metrics.get("atr_pct", 0.005)

    is_atr_extended = atr_pct > 0.0035  # Typically >0.35% in 15m is massively extended

    # ULIS Triple Alignment Gate — directionally aware RSI check.
    #
    # FIX: Red light fires only when RSI contradicts the signal direction:
    #   BUY  → red light if RSI > 75  (overbought — don't chase)
    #   SELL → red light if RSI < 25  (oversold  — don't chase)
    # RSI in the middle zone or confirming direction = no red light.
    # FIX: OFI is now in (-1, +1) range from tanh pipeline
    if is_long:
        rsi_valid = rsi <= 75      # Block overbought BUY-chasing only
        ofi_valid = ofi > -0.15   # Block only if OFI strongly bearish
    else:
        rsi_valid = rsi >= 25     # Block oversold SELL-chasing only
        ofi_valid = ofi < 0.15     # Block only if OFI strongly bullish

    red_lights = 0
    if not rsi_valid: red_lights += 1
    if not ofi_valid: red_lights += 1
    if is_atr_extended: red_lights += 1

    if red_lights >= 2:
        BOT_STATS["gate_stats"]["ulis_alignment_fail"] = BOT_STATS["gate_stats"].get("ulis_alignment_fail", 0) + 1
        logger.warning(f"[Alignment] GATE FAILED: {red_lights} Red Lights (RSI={rsi:.1f}, OFI={ofi:.1f}, ATR Extended={
        return False, 0.0, f"{verdict_str} + Weak Alignment (R={red_lights})"
    elif red_lights == 1:
        confidence *= 0.90  # Reduced penalty from 20% to 10% (Patch #3)
        logger.info(f"[Alignment] 1 Red Light. Penalising confidence to {confidence:.2%}.")

    # Confidence adjustment
    # BREAKOUT_WATCH FIX: This verdict means momentum is building. If the
    # liquidity_vector aligns with the trade direction, it is corroborating
    # evidence → apply a small boost. Only penalise when it conflicts.
    boost = ulis["confidence_boost"]
    if verdict_str == "BREAKOUT_WATCH":
        vector_bullish = ulis["liquidity_vector"] > 0
        if (is_long and vector_bullish) or (not is_long and not vector_bullish):
            boost = +0.01   # Aligned: momentum building in our direction
        else:
            boost = -0.02   # Conflicting: momentum building against us
    adjusted = min(1.0, confidence + boost)
    logger.info(
        f"[ULIS] PASS — {verdict_str} | "
        f"vector={ulis['liquidity_vector']:.3f} | "
        f"cascade={ulis['cascade_risk']:.2f} | "
        f"conf: {confidence:.2%} → {adjusted:.2%}"
    )
    return True, adjusted, verdict_str


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# ■■ STAGE 4.5 — CVD DIVERGENCE GATE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

def _apply_cvd_divergence_gate(
    raw_direction: str,
    metrics: Dict[str, Any],
    current_confidence: float,
) -> Tuple[float, Optional[str]]:
    """
    Stage 4.5: CVD Divergence Gate — Institutional Alpha v2.0
    Sits between Strategy (Stage 4) and Bayesian Fusion (Stage 5).

    ■■ ROLE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    The ONLY gate in the pipeline that reads ACROSS candle boundaries.
    All other layers (OFI, tape, RSI) are single-snapshot.  This gate
    reads institutional footprint across 1–8 candles of history.

    ■■ OUTCOMES ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    1. CONFIRMING DIVERGENCE  — divergence direction agrees with trade
```

```
   → Confidence BOOST scaled by strength + volume confirmation
   → swing_extrema method receives additional +0.01 structural bonus

2. OPPOSING DIVERGENCE    — divergence opposes trade direction
   → Confidence PENALTY scaled by strength
   → STRONG opposing + volume spike → HARD VETO (blocks trade)
     Veto threshold: strength ≥ 0.62 AND confirms ≥ 1 AND vol ≥ 1.4×

3. NO DIVERGENCE / INSUFFICIENT DATA
   → Uncertainty-scaled penalty instead of flat -0.03
     penalty = 0.10 × (1 - n_snaps / MIN_VALID_SNAPS)
     MIN_VALID_SNAPS = 4  (1 hour of 15m candles)

     At n_snaps=0: penalty = -0.10  (completely blind)
     At n_snaps=2: penalty = -0.05  (partial data)
     At n_snaps=4: penalty =  0.00  (sufficient data, no penalty)
     At n_snaps>4: penalty = +0.00  (no reward for more data alone)

■■ VETO THRESHOLD JUSTIFICATION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Veto fires when ALL THREE conditions hold:
    (a) strength ≥ 0.62  — EMA-Z normalised, age-weighted, vol-confirmed
    (b) confirms ≥ 1     — at least one confirming candle pair
    (c) vol_spike ≥ 1.4  — elevated volume at the divergence extreme

Returns:
    (adjusted_confidence: float,  veto_reason: Optional[str])
    veto_reason is None when no veto is issued.
"""
MIN_VALID_SNAPS = 4
VETO_STRENGTH   = 0.62
VETO_VOL_SPIKE  = 1.40

div = metrics.get("cvd_divergence", {})
div_type    = div.get("type",              "NONE")
div_str     = div.get("strength",           0.0)
confirms    = div.get("candles_confirmed",   0)
vol_spike   = div.get("vol_spike",           1.0)
method      = div.get("detection_method",   "none")
n_snaps     = div.get("n_snaps",             0)

is_long  = raw_direction in ("BUY",  "MEAN_REVERSAL_LONG")
is_short = raw_direction in ("SELL", "MEAN_REVERSAL_SHORT")

if div_type == "NONE" or div_str < 0.28:
    if n_snaps >= MIN_VALID_SNAPS:
        penalty = 0.0
        reason  = f"No divergence (sufficient data n={n_snaps})"
    else:
        completeness = n_snaps / MIN_VALID_SNAPS
        penalty      = round(0.10 * (1.0 - completeness), 4)
        reason       = (
            f"Insufficient CVD history "
            f"(n={n_snaps}/{MIN_VALID_SNAPS}) — "
            f"uncertainty penalty={penalty:.2%}"
        )
    adjusted = max(0.0, current_confidence - penalty)
    logger.debug(f"[CVDGate] {reason} | conf {current_confidence:.2%} → {adjusted:.2%}")
    return adjusted, None

confirming = (
    (is_long  and div_type == "BULLISH") or
    (is_short and div_type == "BEARISH")
)
opposing = (
    (is_long  and div_type == "BEARISH") or
    (is_short and div_type == "BULLISH")
)

if confirming:
    if div_str >= 0.65:
        boost = 0.07
    elif div_str >= 0.45:
        boost = 0.05
    else:
        boost = 0.03

    vol_bonus = 0.01 if vol_spike >= 1.40 else 0.0
    method_bonus = 0.01 if method == "swing_extrema" else 0.0
```

```
        total_boost = boost + vol_bonus + method_bonus
        adjusted    = min(1.0, current_confidence + total_boost)

        logger.info(
            f"[CVDGate] ■ CONFIRMING {div_type} | "
            f"strength={div_str:.3f} method={method} "
            f"confirms={confirms} vol={vol_spike:.2f}× | "
            f"boost={total_boost:.2%} → conf {current_confidence:.2%} → {adjusted:.2%}"
        )
        return adjusted, None

    if opposing:
        if (div_str    >= VETO_STRENGTH and
                confirms   >= 1            and
                vol_spike  >= VETO_VOL_SPIKE):

            veto_msg = (
                f"CVD DIVERGENCE VETO: {div_type} divergence opposes "
                f"{raw_direction} | "
                f"strength={div_str:.3f} method={method} "
                f"confirms={confirms} vol={vol_spike:.2f}× | "
                f"Institutional footprint contradicts entry direction."
            )
            logger.warning(f"[CVDGate] ■ {veto_msg}")
            return 0.0, veto_msg

        if div_str >= 0.50:
            penalty = 0.09
        elif div_str >= 0.40:
            penalty = 0.06
        else:
            penalty = 0.03

        if vol_spike >= 1.40:
            penalty = min(penalty + 0.02, 0.12)

        adjusted = max(0.0, current_confidence - penalty)
        logger.info(
            f"[CVDGate] ■■  OPPOSING {div_type} | "
            f"strength={div_str:.3f} method={method} "
            f"confirms={confirms} vol={vol_spike:.2f}× | "
            f"penalty={penalty:.2%} → conf {current_confidence:.2%} → {adjusted:.2%}"
        )
        return adjusted, None

    return current_confidence, None


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# ■■ STAGE 7 — RISK ENGINE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

def _drawdown_adjusted_risk(equity: float, peak_equity: float, base_risk_pct: float) -> float:
    """
    Reduces risk percentage during drawdown. f_adj = base × (1 - DD)^1.5
    Floor: 0.4% (prevents sizing below exchange notional minimum).
    No adjustment when DD < 5% (below noise threshold).
    """
    if peak_equity <= 0:
        return base_risk_pct
    dd = max(0.0, (peak_equity - equity) / peak_equity)
    if dd < 0.05:
        return base_risk_pct
    f_adj = max(base_risk_pct * ((1.0 - dd) ** 1.5), 0.4)
    logger.info(
        f"[RiskEngine] DD={dd:.1%} → risk {base_risk_pct:.1%}→{f_adj:.1%} "
        f"(equity=${equity:.0f} peak=${peak_equity:.0f})"
    )
    return f_adj


def _risk_engine(
    direction: str,
    strategy_type: str,
    price: float,
    atr: float,
    buy_walls: List[float],
    sell_walls: List[float],
```

```
    sweep: Optional[str],
    sl_mult: float = 1.5,        # P0: regime-adaptive (default = NEUTRAL)
    tp_mult_ratio: float = 2.0,  # P0: regime-adaptive RR target
    candle_history: Optional[List[Dict[str, Any]]] = None,
    metrics: Optional[Dict[str, Any]] = None,  # BUG-4: needed for atr_pct_rank
    regime_p: Optional[Dict[str, Any]] = None,  # PHASE-3.2: needed for panic_threshold
    signal: Optional[Dict[str, Any]] = None,   # NEW: for sweep_wick access
) -> Tuple[float, float]:
    """
    ATR-based SL/TP calculator with regime-adaptive multipliers (P0).

    sl_mult and tp_mult_ratio are supplied from REGIME_PARAMS[regime]:
        RANGE:     SL=1.2×ATR, RR=1.8:1  → tight stops in quiet markets
        NEUTRAL:   SL=1.5×ATR, RR=2.0:1  → balanced default
        TREND:     SL=2.2×ATR, RR=2.5:1  → wide stops for trend volatility
        LIQUIDITY: SL=1.5×ATR, RR=2.0:1  → same as NEUTRAL
    """
    is_long = direction in ("BUY", "MEAN_REVERSAL_LONG")

    tp_from_wall = (sell_walls[0] * 0.9995 if sell_walls else None) if is_long \
                else (buy_walls[0] * 1.0005 if buy_walls else None)

    def _calc_atr_from_candles(candles: list, period: int = 14) -> float:
        if len(candles) < period + 1:
            return 0.0
        trs = []
        for i in range(1, len(candles)):
            h = candles[i].get("high", 0)
            l = candles[i].get("low", 0)
            pc = candles[i - 1].get("close", 0)
            tr = max(h - l, abs(h - pc), abs(l - pc))
            trs.append(tr)
        if len(trs) < period:
            return float(np.mean(trs)) if trs else 0.0
        atr = float(np.mean(trs[:period]))
        for j in range(period, len(trs)):
            atr = (atr * (period - 1) + trs[j]) / period
        return atr

    # PHASE-3.1: Dual vol scaling — candle-based vol_ratio AND atr_pct_rank
    # atr_pct_rank is the 30-day rolling rank (0.0=quietest, 1.0=loudest).
    # High rank (panic) = tighten stops; Low rank (quiet) = relax stops slightly.
    atr_pct_rank = metrics.get("atr_pct_rank", 0.5) if metrics else 0.5
    if candle_history and len(candle_history) >= 20:
        recent_candles = list(candle_history)[-5:]
        baseline_candles = list(candle_history)[-20:]
        # FIX-P8: recent_atr always returned 0.0 because length 5 is < period+1 (15).
        # We must explicitly set period=4 for the 5-candle recent slice.
        recent_atr = _calc_atr_from_candles(recent_candles, period=4)
        baseline_atr = _calc_atr_from_candles(baseline_candles, period=14)
        vol_ratio = recent_atr / max(baseline_atr, 1e-8)
    else:
        vol_ratio = 1.0

    vol_scale = float(np.clip(vol_ratio, 0.85, 1.50))
    rank_scale = 1.0 + (atr_pct_rank - 0.5) * 0.4
    rank_scale = float(np.clip(rank_scale, 0.75, 1.25))
    adaptive_mult = sl_mult * vol_scale * rank_scale

    if strategy_type == "TREND":
        adaptive_mult = min(adaptive_mult, 2.50)

    # PHASE-3.2: Panic threshold — emergency SL when ATR is in top 5% of 30-day range
    panic_threshold = regime_p.get("panic_threshold", 5.0) if regime_p else 5.0
    if atr_pct_rank >= (1.0 - panic_threshold / 100.0):
        emergency_mult = 1.0
        logger.warning(
            f"[RiskEngine] PANIC — ATR rank={atr_pct_rank:.0%} in top {panic_threshold:.0f}% of range. "
            f"Emergency SL: {emergency_mult:.2f}× ATR (normal adaptive={adaptive_mult:.2f})"
        )
        adaptive_mult = emergency_mult

    SL_MULT = adaptive_mult

    logger.info(
        f"[RiskEngine] sl_mult={sl_mult:.2f} vol_ratio={vol_ratio:.3f} "
        f"atr_rank={atr_pct_rank:.0%} rank_scale={rank_scale:.3f} "
        f"adaptive_mult={SL_MULT:.3f} strategy={strategy_type}"
```

```
    )

    TP_MULT = tp_mult_ratio  # P0: regime-adaptive (RR target from REGIME_PARAMS)


    def sl_tp(sl_dist: float) -> Tuple[float, float]:
        if is_long:
            sl = price - sl_dist
            tp_target = price + (sl_dist * TP_MULT)
            tp = tp_from_wall if (tp_from_wall and tp_from_wall > tp_target) else tp_target
        else:
            sl = price + sl_dist
            tp_target = price - (sl_dist * TP_MULT)
            tp = tp_from_wall if (tp_from_wall and tp_from_wall < tp_target) else tp_target
        return round(sl, 2), round(tp, 2)

    if strategy_type == "LIQUIDITY_SWEEP" and sweep:
        if sweep == "ABOVE_HIGHS" and sell_walls:
            sl_dist = abs(price - sell_walls[0]) + (atr * 0.5)
        elif sweep == "BELOW_LOWS" and buy_walls:
            sl_dist = abs(price - buy_walls[0]) + (atr * 0.5)
        else:
            sl_dist = atr * SL_MULT
        _sl_wick_override = None
        if signal is not None:
            _wick = signal.get("sweep_wick", 0.0)
            if signal.get("strategy_type") == "LIQUIDITY_SWEEP" and _wick > 0:
                _buf = 0.20 * atr
                if is_long:
                    _sl_wick_candidate = _wick - _buf
                    if _sl_wick_candidate < price - sl_dist:
                        _sl_wick_override = _sl_wick_candidate
                else:
                    _sl_wick_candidate = _wick + _buf
                    if _sl_wick_candidate > price + sl_dist:
                        _sl_wick_override = _sl_wick_candidate
                if _sl_wick_override is not None:
                    logger.info(f"[RiskEngine] Wick SL: {price - sl_dist:.2f}→{_sl_wick_override:.2f}")
                    sl_dist = abs(price - _sl_wick_override)
        return sl_tp(max(sl_dist, atr * SL_MULT))
    elif strategy_type == "TREND":
        return sl_tp(atr * SL_MULT)
    elif strategy_type == "MEAN_REVERSION":
        return sl_tp(atr * SL_MULT)
    else:
        return sl_tp(price * 0.008)


# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# ■■ FULL 7-STAGE SIGNAL ENGINE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

async def _compute_signal(
    metrics: Dict[str, Any],
    candle_history: list,
    feed_state,
    daily_loss_halt: bool,
    drawdown_halt: bool = False,
    quant=None,  # PHASE-5.1: QuantEngine ref for cold-start trade count
) -> Dict[str, Any]:
    WAIT = {
        "verdict": "WAIT", "confidence": 0.0,
        "stop_loss": 0.0, "take_profit": 0.0,
        "analysis": "", "ulis_verdict": "—"
    }

    if daily_loss_halt:
        logger.warning("[RiskEngine] Daily loss limit hit — all trading halted today.")
        _gate_stats_summary("daily_loss_halt")
        return {**WAIT, "analysis": "Daily loss limit reached. Halted."}

    if drawdown_halt:
        logger.warning("[RiskEngine] Max drawdown breached — all trading halted. Restart bot to resume.")
        _gate_stats_summary("drawdown_halt")
        return {**WAIT, "analysis": "Max drawdown breached. Restart bot to resume."}

    # ■■ Capital Hard Stop ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    # PATCH (2026-05-13): Lowered from $150 to $50.
```

```
# $150 was a conservative guideline. Fee math still works at $77 on 15×
# leverage — round-trip fee = 0.04% × 15 × 2 = 1.2% of equity, which is
# covered by any TP hit on a 1.8:1+ RR trade. Real risk at low equity is
# position sizing — handled by the small-account risk cap below.
_MIN_VIABLE_EQUITY = 50.0
if ACCOUNT_SIZE < _MIN_VIABLE_EQUITY:
    logger.warning(
        f"[RiskEngine] ■ Account ${ACCOUNT_SIZE:.2f} below absolute minimum "
        f"${_MIN_VIABLE_EQUITY:.0f}. All entries blocked. Deposit to resume."
    )
    _gate_stats_summary("signal_none")
    return {**WAIT, "analysis": f"Account ${ACCOUNT_SIZE:.2f} < ${_MIN_VIABLE_EQUITY:.0f} minimum. Deposit to resum

# ■■ Small-Account Risk Cap ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# When equity is below the recommended $150, cap risk per trade at 0.75%
# to prevent fee erosion from consecutive losses destroying the account.
# Win rate is the priority — protecting capital between signals is essential.
_RECOMMENDED_EQUITY = 150.0
_SMALL_ACCOUNT_RISK_CAP = 0.75   # % of equity — max loss ~$0.58 at $77
_small_account_mode = ACCOUNT_SIZE < _RECOMMENDED_EQUITY
if _small_account_mode:
    logger.info(
        f"[RiskEngine] Small-account mode active — risk capped at "
        f"{_SMALL_ACCOUNT_RISK_CAP}% (equity=${ACCOUNT_SIZE:.2f} < "
        f"${_RECOMMENDED_EQUITY:.0f} recommended). Win-rate focus."
    )

# PHASE-0.4: Z-Score session guard
# Z-score is statistically meaningless with fewer than 10 bars — it fits noise.
if not metrics.get("z_score_valid", True):
    _gate_stats_summary("zscore_warmup")
    return {**WAIT, "analysis": "Session warmup: Z-Score not yet valid (<10 bars)."}

# --- AUDIT FIX 1: ATR PANIC GATE ---
atr_pct_rank = metrics.get("atr_pct_rank", 0.5)
if atr_pct_rank >= 0.95:
    logger.warning(f"[RiskEngine] ■ ATR PANIC HALT — ATR rank={atr_pct_rank:.0%} >= 95%. Volatility too extreme to
    _gate_stats_summary("atr_panic_halt")
    return {**WAIT, "analysis": f"ATR PANIC HALT (rank={atr_pct_rank:.0%}). Volatility too extreme."}

global LAST_CASCADE_TIME
import time
# NOTE: Cascade cooldown moved AFTER regime detection (Strategy-B)
# so it can use regime-adaptive duration from REGIME_PARAMS.

# Universal post-trade cooldown: POST_TRADE_COOLDOWN_S after any exit (SL or TP)
global LAST_ANY_TRADE_CLOSE_TIME
time_since_last_trade = time.time() - LAST_ANY_TRADE_CLOSE_TIME
# FREQ-1: wire split cooldown — 45s after TP, 90s after SL
_last_was_sl = LAST_TRADE_WAS_SL
_cooldown = POST_TRADE_COOLDOWN_S if _last_was_sl else 120
if time_since_last_trade < _cooldown:
    _gate_stats_summary("post_trade_cooldown")
    return {**WAIT, "analysis": f"Post-trade cooldown ({_cooldown - int(time_since_last_trade)}s remain, {'SL' if _

global LAST_CANDLE_TS
import time as _time

price = metrics["price"]
atr   = metrics.get("atr", price * 0.005)
vpoc  = metrics.get("vpoc")

# Stage 1: Parse walls
buy_walls  = [p for p, _ in metrics.get("top_buy_walls", [])]
sell_walls = [p for p, _ in metrics.get("top_sell_walls", [])]

# Stage 1b: Pre-detect sweep (needed for LIQUIDITY cap in _detect_regime)
sweep, sweep_ts, sweep_wick = _detect_liquidity_sweep(metrics, buy_walls, sell_walls, candle_history)

# Stage 2: Regime
regime = _detect_regime(metrics, buy_walls, sell_walls, quant=quant, sweep=sweep)
logger.info(f"[Regime] {regime} | Z={metrics['zScore']:.2f} | "
            f"ATR%={metrics.get('atr_pct', 0):.3%} | Tape={metrics['tapeSpeed']}")

# Stage 2b: HTF (4H) Trend Filter — block counter-trend entries
htf = _htf_trend(candle_history)
if htf != "NEUTRAL":
    logger.info(f"[HTF] 4H trend = {htf}")
```

```
# ■■ P0: Load regime-conditional parameter matrix ■■■■■■■■■■■■■■■■■■■■■■■■■
# All downstream stages read thresholds from regime_p instead of hard-coded constants.
regime_p = REGIME_PARAMS.get(regime, REGIME_PARAMS["NEUTRAL"])

# STRATEGY-B: Regime-adaptive cascade cooldown.
# RANGE/LIQUIDITY = 180s (quiet markets recover fast, sweeps repeat).
# TREND = 240s (trend may still be valid). NEUTRAL = 300s (default).
cascade_cd = regime_p.get("cascade_cooldown_s", 300)

# --- AUDIT FIX 4: ESCALATING COOLDOWNS & CONSECUTIVE LOSS HALT ---
if quant:
    consecutive_losses = getattr(quant, "_consecutive_losses", 0)
    if consecutive_losses >= 3:
        logger.warning(f"[RiskEngine] ■ 3 CONSECUTIVE LOSSES — Session Halted. Taking a break to prevent further dr
        _gate_stats_summary("consecutive_loss_halt")
        return {**WAIT, "analysis": "3 consecutive losses halt. Session paused."}

    if consecutive_losses > 0:
        cascade_cd *= (1 + consecutive_losses)
        logger.info(f"[RiskEngine] Escalating cooldown active. {consecutive_losses} losses -> cascade_cd extended t

time_since_cascade = time.time() - LAST_CASCADE_TIME
if time_since_cascade < cascade_cd:
    remaining = cascade_cd - int(time_since_cascade)
    _gate_stats_summary("cascade_cooldown")
    return {**WAIT, "analysis": f"WAIT (Cascade Cooldown: {remaining}s remain, regime={regime})"}

logger.info(
    f"[RegimeParams] z_thr={regime_p['z_threshold']} "
    f"sl_mult={regime_p['atr_multiplier_sl']} "
    f"min_conf={regime_p['min_confidence']:.0%} "
    f"rr={regime_p['rr_target']} "
    f"gate={regime_p['candle_gate_sec']}s "
    f"htf_block={regime_p['htf_block']}"
)

# Stage 3: Sweep detection (already computed in Stage 1b — reuse)
# Stage 3b: Candle-Close Freshness Gate (LIQUIDITY_SWEEP only)
# New logic: we use the actual timestamp of the candle that triggered the sweep
# to measure its age. This correctly handles sweeps detected from the live candle.
if sweep:
    sweep_candle_age_s = _time.time() - sweep_ts
    # A 15m candle = 900s. We allow entry at ANY POINT within the CURRENT candle
    # after the previous candle produced the sweep — i.e. up to 2 full candle lengths.
    # Old value (900+gate_sec=945s) gave only a 45s reaction window which was far
    # too tight: mid-session starts and any latency caused instant staleness.
    gate_sec = regime_p["candle_gate_sec"]
    max_sweep_age = 900 + gate_sec  # PHASE-5.3: was 1800+gate_sec (~30min), now 900+gate_sec (~15min)
    if sweep_candle_age_s > max_sweep_age:
        logger.info(
            f"[CandleGate] Sweep signal stale — sweep candle closed "
            f"{sweep_candle_age_s:.0f}s ago (>{max_sweep_age}s). Blocking."
        )
        BOT_STATS["gate_stats"]["candle_gate_expired"] = BOT_STATS["gate_stats"].get("candle_gate_expired", 0) + 1
        sweep = None

# Stage 4: Strategy
raw_direction: Optional[str] = None
strategy_type: str

if sweep:
    strategy_type = "LIQUIDITY_SWEEP"
    raw_direction = _strategy_liquidity_sweep(sweep, metrics)
    logger.info(f"[MetaModel] → LIQUIDITY_SWEEP (sweep={sweep})")
elif regime == "TREND":
    strategy_type = "TREND"
    raw_direction = _strategy_trend(metrics)
    logger.info("[MetaModel] → TREND strategy")
elif regime == "RANGE":
    strategy_type = "MEAN_REVERSION"
    raw_direction = _strategy_mean_reversion(
        metrics,
        z_threshold=regime_p["z_threshold"],
    )
    logger.info(f"[MetaModel] → MEAN_REVERSION strategy (z_thr={regime_p['z_threshold']})")
elif regime == "NEUTRAL":
    strategy_type = "MEAN_REVERSION"
    z_current = metrics.get("zScore", 0.0)
```

```
    z_prev = metrics.get("zScore_prev", z_current)
    z_slope = z_current - z_prev
    rsi = metrics.get("rsi", 50.0)
    rsi_prev = metrics.get("rsi_prev", rsi)
    rsi_prev2 = metrics.get("rsi_prev2", rsi_prev)
    rsi_trough = (rsi > rsi_prev) and (rsi_prev < rsi_prev2)
    rsi_peak = (rsi < rsi_prev) and (rsi_prev > rsi_prev2)
    z_thr = regime_p["z_threshold"]
    candidate = _strategy_mean_reversion(metrics, z_threshold=z_thr)
    if candidate == "MEAN_REVERSAL_LONG" and (z_slope <= 0 or not rsi_trough):
        candidate = None
    if candidate == "MEAN_REVERSAL_SHORT" and (z_slope >= 0 or not rsi_peak):
        candidate = None
    raw_direction = candidate
    if raw_direction:
        logger.info(f"[MetaModel] NEUTRAL -> {strategy_type} ({raw_direction}) "
                    f"z={z_current:.2f} slope={z_slope:+.3f}")
    else:
        logger.debug(f"[MetaModel] NEUTRAL: no setup (z={z_current:.2f}, "
                     f"slope={z_slope:+.3f}")
else:
    strategy_type = "NEUTRAL"
    raw_direction = None
    logger.info(f"[MetaModel] → WAIT (regime={regime}, no sweep)")

if raw_direction is None:
    _gate_stats_summary("signal_none")
    return {**WAIT, "analysis": f"Regime={regime} strategy={strategy_type} — no edge."}

# --- AUDIT FIX 3: RSI EXTREMES GATE ---
rsi = metrics.get("rsi", 50.0)
is_long_dir = raw_direction in ("BUY", "MEAN_REVERSAL_LONG")
if is_long_dir and rsi > 70.0:
    logger.warning(f"[RiskEngine] ■ RSI OVERBOUGHT HALT — Blocking {raw_direction} at RSI={rsi:.1f} > 70.0")
    _gate_stats_summary("rsi_overbought")
    return {**WAIT, "analysis": f"RSI={rsi:.1f} > 70.0 blocks {raw_direction} entry."}
if not is_long_dir and rsi < 30.0:
    logger.warning(f"[RiskEngine] ■ RSI OVERSOLD HALT — Blocking {raw_direction} at RSI={rsi:.1f} < 30.0")
    _gate_stats_summary("rsi_oversold")
    return {**WAIT, "analysis": f"RSI={rsi:.1f} < 30.0 blocks {raw_direction} entry."}

# Stage 4b: HTF Counter-Trend Block
# P0: htf_block is regime-conditional. In RANGE, mean-reversion against HTF is the strategy.
is_long_dir = raw_direction in ("BUY", "MEAN_REVERSAL_LONG")
is_trend_strat = strategy_type == "TREND"

if regime_p["htf_block"]:   # P0: Only apply HTF block when regime says to
    if htf == "BEAR" and is_long_dir and is_trend_strat:
        logger.warning(f"[HTF] Blocking LONG trend trade — 4H trend is BEAR.")
        _gate_stats_summary("htf_counter_trend")
        return {**WAIT, "analysis": f"HTF=BEAR blocks {raw_direction} trend entry."}
    if htf == "BULL" and not is_long_dir and is_trend_strat:
        logger.warning(f"[HTF] Blocking SHORT trend trade — 4H trend is BULL.")
        _gate_stats_summary("htf_counter_trend")
        return {**WAIT, "analysis": f"HTF=BULL blocks {raw_direction} trend entry."}

    # Funding Rate Anti-Squeeze Protection (TREND only, since htf_block is now regime-gated)
    if is_trend_strat:
        try:
            fr = float(metrics.get("funding_rate", 0.0))
            # P1-6 FIX: Old threshold (0.015%) was 5-10x below normal BTC contango
            # (typically 0.03-0.10% per 8h). It silently blocked ~70% of long entries
            # in any bullish session. New thresholds reflect genuinely crowded positions.
            FUNDING_LONG_BLOCK  =  0.0008   # 0.08% per 8h — clearly crowded longs
            FUNDING_SHORT_BLOCK = -0.0005   # -0.05% per 8h — clearly crowded shorts
            if is_long_dir and fr > FUNDING_LONG_BLOCK:
                logger.warning(f"[Anti-Squeeze] Blocking LONG: extreme funding rate {fr:.4%} > {FUNDING_LONG_BLOCK:
                _gate_stats_summary("funding_blocks_long")
                return {**WAIT, "analysis": f"Funding Rate {fr:.4%} > {FUNDING_LONG_BLOCK:.4%}. Blocked long."}
            if not is_long_dir and fr < FUNDING_SHORT_BLOCK:
                logger.warning(f"[Anti-Squeeze] Blocking SHORT: extreme funding rate {fr:.4%} < {FUNDING_SHORT_BLOC
                _gate_stats_summary("funding_blocks_short")
                return {**WAIT, "analysis": f"Funding Rate {fr:.4%} < {FUNDING_SHORT_BLOCK:.4%}. Blocked short."}
        except (ValueError, TypeError):
            pass

# FIX-P4: HTF Directional Filter for LIQUIDITY Sweep entries.
# htf_block=False in LIQUIDITY correctly allows the regime — sweeps ARE reversal setups.
```

```
# BUT: a BELOW_LOWS BUY sweep into a 4H BEAR trend is a continuation, not a reversal.
# The audit found 13 trades, many of which were BUY sweeps into a sustained BEAR 4H.
# Fix: sweep entries must align with the reversal implied by the sweep direction vs HTF.
# HTF=NEUTRAL → no bias, allow both directions.
is_sweep_strat = strategy_type == "LIQUIDITY_SWEEP" and sweep is not None
if is_sweep_strat and htf != "NEUTRAL":
    if htf == "BEAR" and is_long_dir:
        logger.warning(
            "[HTF-Sweep] BUY sweep BLOCKED — 4H BEAR trend. "
            "BELOW_LOWS into a bear trend is a continuation, not a reversal."
        )
        _gate_stats_summary("htf_counter_trend")
        return {**WAIT, "analysis": "HTF=BEAR blocks BUY sweep (not a genuine reversal setup)."}
    if htf == "BULL" and not is_long_dir:
        logger.warning(
            "[HTF-Sweep] SELL sweep BLOCKED — 4H BULL trend. "
            "ABOVE_HIGHS into a bull trend is a continuation, not a reversal."
        )
        _gate_stats_summary("htf_counter_trend")
        return {**WAIT, "analysis": "HTF=BULL blocks SELL sweep (not a genuine reversal setup)."}

# === DERIVATIVES GATE: Institutional signal check ===
# Fetch context (cached, so this is near-instant on most calls)
_deriv_ctx = await _derivatives_ctx.get_full_context()
_deriv_ok, _deriv_reason = await _derivatives_gate(raw_direction, _deriv_ctx)
if not _deriv_ok:
    logger.warning(f"[DerivGate] VETO: {_deriv_reason}")
    _gate_stats_summary("derivatives_veto")
    return {**WAIT, "analysis": f"Derivatives gate veto: {_deriv_reason}"}
if _deriv_reason:
    logger.info(f"[DerivGate] PASS ({_deriv_reason})")

# FIXED: CVD gate BEFORE Bayes fusion (was after)
_pre_div_conf = metrics.get("bayesianPosterior", 0.5)
_div_conf, _div_veto = _apply_cvd_divergence_gate(raw_direction, metrics, _pre_div_conf)
if _div_veto:
    _gate_stats_summary("cvd_divergence_veto")
    return {**WAIT, "analysis": f"CVD divergence veto: {_div_veto}"}
if abs(_div_conf - _pre_div_conf) > 0.001:
    metrics = {**metrics, "bayesianPosterior": _div_conf}

# Stage 5: Bayesian fusion (P1: regime_priors already injected into metrics upstream)
confidence = _bayesian_fusion(metrics, raw_direction, regime, is_sweep=bool(sweep))

# --- AUDIT FIX 2: BAYES FLOOR GATE ---
if confidence < 0.50:
    logger.warning(f"[RiskEngine] ■ BAYES FLOOR HALT — Bayesian posterior {confidence:.2%} < 50%. Edge is worse tha
    _gate_stats_summary("bayes_floor_veto")
    return {**WAIT, "analysis": f"Bayesian Posterior {confidence:.2%} < 50% blocks entry."}

# Stage 5b: VPOC Proximity Confidence Boost
vpoc_boost = _vpoc_confidence_boost(price, vpoc, raw_direction)
if vpoc_boost > 0.0:
    confidence = min(1.0, confidence + vpoc_boost)
    logger.info(f"[VPOC] Near VPOC ({vpoc:.0f}). Confidence boosted by +{vpoc_boost:.2%} → {confidence:.2%}")

logger.info(f"[BayesFusion] direction={raw_direction} | P={confidence:.2%}")

# Stage 6 ★ ULIS/ALDE gate FIRST (applies confidence boost)
should_trade, confidence, ulis_verdict_str = _apply_ulis_gate(
    metrics, candle_history, feed_state, raw_direction, confidence
)

if not should_trade:
    _gate_stats_summary("ulis_veto")
    return {**WAIT, "analysis": f"ULIS veto: {ulis_verdict_str}", "ulis_verdict": ulis_verdict_str}

# FIX: Check threshold AFTER ULIS gate using boosted confidence
# P0: Use regime-adaptive min_confidence instead of global MIN_CONFIDENCE
# PHASE-5.1: Apply cold-start discount (few trades = slightly lower bar)
# PHASE-5.2: Wire MIN_CONFIDENCE as a floor under regime threshold
total_trades = sum(quant.get_regime_trade_counts().values()) if quant else 0
cold_start_discount = COLD_START_CONFIDENCE_DISCOUNT if total_trades < COLD_START_TRADE_COUNT else 0.0
# FIX: Ensure global environment MIN_CONFIDENCE does not force an impossibly high hurdle.
# Use 0.50 as an absolute baseline safety floor to avoid negative expectation entries.
regime_min_conf = max(
    regime_p["min_confidence"] - cold_start_discount,
    0.50
```

```
)
if confidence < regime_min_conf:
    _gate_stats_summary("confidence_below_threshold")
    return {**WAIT, "analysis": (
        f"Regime={regime} strategy={strategy_type} signal={raw_direction} "
        f"but P={confidence:.2%} < regime threshold={regime_min_conf:.0%}"
    )}

# Stage 7: Risk engine — P0: adaptive ATR multipliers from regime params
stop_loss, take_profit = _risk_engine(
    raw_direction, strategy_type,
    price,
    atr, buy_walls, sell_walls, sweep,
    sl_mult=regime_p["atr_multiplier_sl"],
    tp_mult_ratio=regime_p["rr_target"],
    candle_history=candle_history,
    metrics=metrics,  # BUG-4: pass for atr_pct_rank vol scaling
    regime_p=regime_p,  # PHASE-3.2: pass for panic_threshold
    signal={"sweep_wick": sweep_wick if sweep else 0.0, "strategy_type": strategy_type},  # NEW: pass for sweep_wic
)

# Sanity check — geometry must be valid
is_long = raw_direction in ("BUY", "MEAN_REVERSAL_LONG")
if is_long and (stop_loss >= price or take_profit <= price):
    logger.warning("[RiskEngine] Invalid geometry for LONG — skipped.")
    _gate_stats_summary("fee_geometry")
    return {**WAIT, "analysis": "Invalid SL/TP geometry — skipped."}
if not is_long and (stop_loss <= price or take_profit >= price):
    logger.warning("[RiskEngine] Invalid geometry for SHORT — skipped.")
    _gate_stats_summary("fee_geometry")
    return {**WAIT, "analysis": "Invalid SL/TP geometry — skipped."}

# ■■ Pre-trade fee profitability check ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Reject any trade where the expected TP gain (as % of entry) is less
# than the round-trip exchange fee cost. Without this check the bot will
# consistently lose money even on winning trades.
# LEVERAGE FIX: Binance charges fee on NOTIONAL (position × price).
# Since tp_gain_pct is measured as a raw asset price movement (not ROI on margin),
# the required break-even price movement is simply the round-trip fee rate on the notional.
# We do NOT multiply by LEVERAGE here, because leverage magnifies both profit and fees equally.
tp_gain_pct = abs(take_profit - price) / price          # % gain if TP hit
round_trip_fee_rate = _EXCHANGE_FEE_RATE * 2            # entry + exit on notional
min_viable_tp_pct = round_trip_fee_rate * 1.2           # TP must cover 1.2× fees
if tp_gain_pct < min_viable_tp_pct:
    logger.warning(
        f"[FeeCheck] TP gain {tp_gain_pct:.3%} < min viable {min_viable_tp_pct:.3%} "
        f"(round-trip fee rate={round_trip_fee_rate:.2%}). "
        f"Trade not profitable after fees. Skipping."
    )
    _gate_stats_summary("fee_geometry")
    return {**WAIT, "analysis": (
        f"TP gain {tp_gain_pct:.3%} below fee break-even {min_viable_tp_pct:.3%}. Skipped."
    ), "ulis_verdict": ulis_verdict_str}

verdict_map = {
    "BUY":               "BUY",
    "SELL":              "SELL",
    "MEAN_REVERSAL_LONG":  "MEAN_REVERSAL_LONG",
    "MEAN_REVERSAL_SHORT": "MEAN_REVERSAL_SHORT",
}
verdict = verdict_map.get(raw_direction, raw_direction)

analysis = (
    f"Regime={regime} Strategy={strategy_type} | {verdict} @ {price:.2f} "
    f"| P={confidence:.2%} | SL={stop_loss} TP={take_profit} "
    f"| Z={metrics['zScore']:.2f} RSI={metrics['rsi']:.1f} "
    f"OFI={metrics['ofi']:.1f} CVD={metrics['cvd']:.0f} "
    f"ATR={atr:.2f} Tape={metrics['tapeSpeed']}/{metrics['tapeDominant']} "
    f"ULIS={ulis_verdict_str}"
)
logger.info(f"[Signal] >> {verdict} | conf={confidence:.0%} | SL={stop_loss} TP={take_profit} | ULIS={ulis_verdict_

BOT_STATS["gate_stats"]["total_passed"] += 1

return {
    "verdict":      verdict,
    "confidence":   round(confidence, 4),
    "stop_loss":    stop_loss,
```

```
        "take_profit":  take_profit,
        "analysis":     analysis,
        "ulis_verdict": ulis_verdict_str,
        "regime":       regime,   # P1: stored in position for per-regime Beta update on exit
        "be_lock_trigger": regime_p.get("be_lock_trigger", 1.0),
        "atr_at_entry": atr,      # stored in position for any future trailing logic
        "sweep_wick":   sweep_wick if sweep else 0.0,
        "strategy_type": strategy_type,
    }


# ═══════════════════════════════════════════════════════════════════════════
# ■■ CORE EXECUTION LOOP ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# ═══════════════════════════════════════════════════════════════════════════

async def _process_exit(
    pnl: float,
    regime: str,
    stats: dict,
    quant,
    executor,
):
    """Single source of truth for all post-trade state updates.
    Called from both the main exit path and the 30s heartbeat poll path."""
    global LAST_CASCADE_TIME, LAST_ANY_TRADE_CLOSE_TIME, LAST_TRADE_WAS_SL

    LAST_ANY_TRADE_CLOSE_TIME = time.time()
    LAST_TRADE_WAS_SL = (pnl < 0)

    stats["daily_pnl"]   = stats.get("daily_pnl",   0.0) + pnl
    stats["session_pnl"] = stats.get("session_pnl", 0.0) + pnl

    quant.update_win_rate(won=(pnl >= 0), regime=regime)

    if pnl < 0:
        LAST_CASCADE_TIME = time.time()
        stats["consecutive_losses"] = stats.get("consecutive_losses", 0) + 1
        now_t = time.time()
        loss_times = [t for t in stats.get("loss_times", []) if now_t - t < 1800]
        loss_times.append(now_t)
        stats["loss_times"] = loss_times
        if len(loss_times) >= 3:
            stats["cooldown_until"] = now_t + 1800
            stats["loss_times"] = []
            logger.error("[RiskManager] 3 SL exits within 30 min — 30-min cooldown active.")
            if executor.notifier:
                await executor.notifier.send_message(
                    f"■ Quad-Desk CONSECUTIVE LOSS HALT\n"
                    f"3 SL exits within 30 minutes. Paused 30 min."
                )
    else:
        stats["consecutive_losses"] = 0

    # P0-2 FIX: Use live equity (not static boot ACCOUNT_SIZE) for risk calcs.
    # After gains/losses, the absolute dollar limits must scale with real equity.
    current_equity = ACCOUNT_SIZE + stats.get("session_pnl", 0.0)

    max_loss_usd = current_equity * MAX_DAILY_LOSS_PCT / 100.0
    if stats["daily_pnl"] < -max_loss_usd and not stats.get("daily_loss_halt"):
        stats["daily_loss_halt"] = True
        logger.warning(
            f"[RiskEngine] ■ Daily loss limit: ${stats['daily_pnl']:.2f} "
            f"(limit=-${max_loss_usd:.2f} on equity=${current_equity:.2f}). Halted until tomorrow."
        )

    max_drawdown_usd = current_equity * MAX_DRAWDOWN_PCT / 100.0
    if stats["session_pnl"] < -max_drawdown_usd and not stats.get("drawdown_halt"):
        stats["drawdown_halt"] = True
        logger.critical(
            f"[RiskEngine] ■ MAX DRAWDOWN BREACHED: "
            f"${stats['session_pnl']:.2f} (limit=-${max_drawdown_usd:.2f} on equity=${current_equity:.2f}). HALTED."
        )
        if executor.notifier:
            await executor.notifier.send_message(
                f"■ Quad-Desk MAX DRAWDOWN HIT\n"
                f"Session PnL: ${stats['session_pnl']:.2f} / Limit: -${max_drawdown_usd:.2f}\n"
                f"Bot halted. Restart to resume."
            )
```

```
async def execution_loop(
    feed,
    quant: QuantEngine,
    executor: TradingExecutor,
    stats: Dict[str, Any],
):
    await executor.initialize()

    # CRIT-2 FIX: Set leverage AFTER markets are loaded (initialize() calls load_markets).
    # Pre-initialize leverage set could hit Binance with an unvalidated symbol and
    # silently fail, leaving the account at whatever leverage was last manually set.
    if executor.is_futures and not executor.dry_run:
        try:
            ccxt_symbol = executor._get_ccxt_symbol(FEED_SYMBOL)
            await executor.exchange.set_leverage(LEVERAGE, ccxt_symbol)
            logger.info(f"[Main] Futures leverage confirmed: {LEVERAGE}× on {ccxt_symbol} ✓")
        except Exception as e:
            logger.warning(f"[Main] Could not set leverage (will continue with account default): {e}")

    global ACCOUNT_SIZE, LAST_CVD, LAST_CASCADE_TIME, LAST_ANY_TRADE_CLOSE_TIME, LAST_CANDLE_TS, LAST_TRADE_WAS_SL
    global _CYCLE_ERROR_COUNT, _LAST_CYCLE_ERROR

    if not executor.dry_run:
        try:
            fetched_bal = await executor.get_usdt_balance(ACCOUNT_SIZE)
            if fetched_bal != ACCOUNT_SIZE and fetched_bal > 0:
                logger.info(f"[Main] Overriding BOT_ACCOUNT_SIZE with dynamically fetched live balance: ${fetched_bal:.
                ACCOUNT_SIZE = fetched_bal
        except Exception as e:
            logger.warning(f"[Main] Could not dynamically fetch account balance at startup: {e}")

    stats["account_equity"] = ACCOUNT_SIZE + stats.get("session_pnl", 0.0)

    EQUITY_MIN_TRADEABLE = 200.0
    if ACCOUNT_SIZE < EQUITY_MIN_TRADEABLE:
        logger.warning(
            f"[RiskEngine] ■■ Account ${ACCOUNT_SIZE:.2f} below recommended minimum "
            f"(${EQUITY_MIN_TRADEABLE:.0f}). Trade sizing may fail or be suboptimal."
        )
        if executor.notifier:
            await executor.notifier.send_message(
                f"■■ Low Equity: ${ACCOUNT_SIZE:.2f} (min recommended: ${EQUITY_MIN_TRADEABLE:.0f})"
            )

    mode = "DRY-RUN" if executor.dry_run else "LIVE"
    logger.info(
        f"■■■ Quad-Desk Bot ■■■■■■■■■■■■■■■■■■■■■■■\n"
        f"■  Exchange     : {EXCHANGE.upper()}\n"
        f"■  Symbol       : {SYMBOL}\n"
        f"■  Feed Symbol  : {FEED_SYMBOL} (Binance WS)\n"
        f"■  Mode         : {mode}\n"
        f"■  Risk/trade   : {MAX_RISK_PCT}%  (${ACCOUNT_SIZE * MAX_RISK_PCT / 100:.2f} max loss)\n"
        f"■  Max daily ↓  : {MAX_DAILY_LOSS_PCT}%\n"
        f"■  Min P(edge)  : {MIN_CONFIDENCE:.0%}\n"
        f"■  ULIS Gate    : {'ON ✓' if ULIS_GATE_ENABLED else 'OFF'}\n"
        f"■  Interval     : every {ANALYSIS_INTERVAL}s\n"
        f"■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■"
    )

    _last_recon_ts = 0.0
    _orphan_flatten_attempts: int = 0          # how many times we've tried to flatten an orphan
    _last_orphan_flatten_ts: float = 0.0       # timestamp of last flatten attempt

    # Startup lockout: prevent any entry for the first 90s after boot.
    # The REST prefetch loads 100 historical candles, but live CVD, OFI and
    # order-book readings need a few cycles to stabilise before we act on them.
    _STARTUP_LOCKOUT_SECS = 90
    _boot_time = time.time()
    logger.info(
        f"[Main] ■ Startup lockout active — no new entries for {_STARTUP_LOCKOUT_SECS}s "
        f"while live feed settles."
    )

    while True:
        try:
            # ■■ Event-driven execution (Phase 6) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            try:
                # Wait for candle close OR timeout (whichever comes first)
```

```
                await asyncio.wait_for(
                    asyncio.shield(feed.state.candle_close_event.wait()),
                    timeout=float(ANALYSIS_INTERVAL)
                )
                feed.state.candle_close_event.clear()
            except (asyncio.TimeoutError, TimeoutError):
                pass  # Fallback to polling if websocket hasn't fired yet

            # ■■ DIAGNOSTIC: confirm loop is alive every cycle ■■■■■■■■■■■■■■■
            n_candles = len(feed.state.candles)
            logger.debug(f"[Main] Cycle tick | candles={n_candles} | "
                         f"trades={len(feed.state.recent_trades)} | cvd={feed.state.cvd:.0f}")

            if n_candles < QuantEngine.MIN_CANDLES:
                logger.info(f"[Main] Warming up… {n_candles}/{QuantEngine.MIN_CANDLES} candles")
                continue

            # ■■ Startup lockout guard ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            # Allow exit monitoring immediately, but block new entries until
            # the live feed has had time to build real CVD/OFI/order-book data.
            _startup_elapsed = time.time() - _boot_time
            _in_startup_lockout = _startup_elapsed < _STARTUP_LOCKOUT_SECS

            # ■■ Daily loss reset at midnight ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            today = date.today()
            if stats.get("_last_trade_day") != today:
                stats["_last_trade_day"] = today
                stats["daily_pnl"]       = 0.0


... [FILE TRUNCATED - TOO LARGE] ...
```

■ quant_engine.py

```
import time
import logging
import math
import json
import os
import numpy as np
import pandas as pd
from collections import deque
from typing import Dict, Any, Optional

logger = logging.getLogger(__name__)



class QuantEngine:
    """
    Computes the 7 core Macro Strategy metrics from a live MarketState.
    All calculations mirror the audited React store/index.ts logic.

    Enhancements (v2):
      - Beta(α,β) conjugate prior for self-calibrating Bayesian win rate
      - Student's t-distribution Z-score (fat-tail robust, replaces Gaussian)
      - funding_rate passed through from MarketState for ULIS signal
    """

    MIN_CANDLES = 51  # Need 51 to produce 50 log-returns

    def __init__(self, state):
        self.state = state

        # ■■ Per-Regime Beta Priors (P1 — HMM Spec Apr 2026) ■■■■■■■■■■■■■■■■■■■■
        # Separate Beta(α,β) conjugate prior per HMM regime.
        # Tracks win rate independently for RANGE (mean-reversion),
        # TREND (momentum), and NEUTRAL/LIQUIDITY (mixed) regimes.
        # Mean = α/(α+β) = 0.5 at startup (uniform prior, no assumptions).
        self._regime_alpha: dict = {"RANGE": 5.0, "NEUTRAL": 5.0, "TREND": 5.0, "LIQUIDITY": 5.0}
        self._regime_beta:  dict = {"RANGE": 5.0, "NEUTRAL": 5.0, "TREND": 5.0, "LIQUIDITY": 5.0}
        self._regime_trade_count: dict = {"RANGE": 0, "NEUTRAL": 0, "TREND": 0, "LIQUIDITY": 0}
        # Keep global fallback for backwards compatibility with _bayesian()
        self._alpha: float = 5.0
        self._beta:  float = 5.0


        # ■■ OFI State (Three-Stage Pipeline) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        # Stage 1: True OFI = delta of bid/ask depth between snapshots
        self._prev_bid_depth: float = 0.0
        self._prev_ask_depth: float = 0.0
        # Stage 2: MAD Persistence Filter
        self._ofi_history: deque = deque(maxlen=100)
        self._ofi_outlier_count: int = 0
        # Stage 3: EWMA normalisation + tanh soft saturation
        self._ofi_ewma_mu:  float = 0.0
        self._ofi_ewma_var: float = 1.0
        self._ofi_smooth:   float = 0.0

        # ■■ ATR Percentile Rank (P2 — HMM Spec Apr 2026) ■■■■■■■■■■■■■■■■■■■■
        # Rolling 30-day window (4 candles/hr × 24hr × 30d = 2880 candles)
        # Rank of current ATR within this window gives a normalised [0,1]
        # measure of how extreme current volatility is vs recent history.
        # Much more robust than raw ATR% which varies by price level.
        self._atr_history: deque = deque(maxlen=2880)

        # ■■ State variables extracted from main.py ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        self._liquidity_consecutive: int = 0

        # ■■ RSI History Cache (MED-1 fix) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        # Caches the last 3 computed RSI values so rsi_prev/rsi_prev2 reflect
        # true historically-observed values rather than recomputed ones with
        # different EMA seeds.
        self._rsi_history: deque = deque(maxlen=3)

        # ■■ CVD Divergence Snapshot History ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        # Per-candle snapshots of {candle_ts, close, low, high, cvd} taken at
        # the moment each new 15m candle is detected.  maxlen=12 (3 hours).
        # Needed because self.state.cvd is a running session total with no memory
        # of prior candle closes — divergence detection requires comparing CVD
```

```
    # at candle N vs candle N-k.
    self._cvd_candle_snapshots: deque = deque(maxlen=12)
    self._last_snapped_candle_ts: float = 0.0

    # ■■ CVD Delta EMA-Variance Normalization (Fix #1) ■■■■■■■■■■■■■■■■■■■■■■■■
    # Welford's online EMA for CVD delta normalization (replaces fragile
    # session-range denominator). λ=0.92 → half-life ≈ 8 observations (~2h).
    self._cvd_delta_ewma_mu:  float = 0.0
    self._cvd_delta_ewma_var: float = 1.0
    self._cvd_delta_ewma_n:   int   = 0

    # ■■ CVD Divergence Streak Tracker (Age Cap Fix) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    # If the SAME divergence direction fires for >8 consecutive candle snapshots
    # (~2 hours), it is measuring a macro TREND not a divergence.  Decay the
    # strength exponentially so the signal expires naturally.
    # exp(-0.15 × (12-8)) ≈ 0.55 — well below MIN_STRENGTH=0.28 at 12 candles.
    self._cvd_div_streak_dir:   str = "NONE"
    self._cvd_div_streak_count: int = 0

    # ■■ Consecutive-Loss Tracker (Prior Softening) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    self._consecutive_losses: int = 0

    # ■■ CVD Post-Reset Cooldown (Ghost Signal Fix 2026-05-13) ■■■■■■■■■■■■■■■
    # Suppresses divergence detection for N cycles after the daily CVD reset.
    # Without this the reset artifact triggers a spurious BEARISH divergence
    # that persists for 100+ candles before expiring naturally (~22 min of
    # false bearish bias). Set to 0 at boot; reset_cvd_divergence_state()
    # sets it to 5 whenever the daily hook fires.
    self._cvd_post_reset_cooldown: int = 0

    # ■■ Session VWAP Anchor (Phase 2) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    self._session_vwap_num: float = 0.0
    self._session_vwap_den: float = 0.0
    self._session_date = None

    # Persistence path — survives Railway restarts if /tmp is mounted
    self._persist_path = os.environ.get("BOT_STATE_PATH", "/tmp/quad_bot_state.json")
    self._load_state()


# ------------------------------------------------------------------
# Public: update win-rate tracker after each trade
# ------------------------------------------------------------------
def update_win_rate(self, won: bool, regime: str = "NEUTRAL"):
    if won:
        self._alpha += 1.0
        self._regime_alpha[regime] = self._regime_alpha.get(regime, 1.0) + 1.0
        self._consecutive_losses = 0   # reset streak on any win
    else:
        self._beta += 1.0
        self._regime_beta[regime] = self._regime_beta.get(regime, 1.0) + 1.0
        self._consecutive_losses = getattr(self, "_consecutive_losses", 0) + 1
        # Prior-softening: after 3+ consecutive losses the Beta prior has drifted
        # too bearish to recover naturally. Pull it 15% toward Beta(5,5) each loss
        # so the bot can re-engage after a bad streak without a full cold-start reset.
        if self._consecutive_losses >= 3:
            _target = 5.0
            self._alpha = self._alpha * 0.85 + _target * 0.15
            self._beta  = self._beta  * 0.85 + _target * 0.15
            logger.warning(
                f"[QuantEngine] Consecutive loss #{self._consecutive_losses} — "
                f"softening prior toward Beta(5,5): "
                f"α={self._alpha:.1f} β={self._beta:.1f} "
                f"P(bull)={self._alpha/(self._alpha+self._beta):.1%}"
            )
    self._regime_trade_count[regime] = self._regime_trade_count.get(regime, 0) + 1
    p_bull = self._alpha / (self._alpha + self._beta)
    n = self._alpha + self._beta - 2
    logger.info(
        f"[QuantEngine] Win-rate updated: α={self._alpha:.0f} β={self._beta:.0f} "
        f"→ P(bull)={p_bull:.2%} | regime={regime}  (n={n:.0f} trades)"
    )
    self._save_state()

def get_regime_trade_counts(self) -> dict:
    return dict(self._regime_trade_count)

def reset_cvd_divergence_state(self) -> None:
```

```
    """
    Call this whenever the daily CVD accumulator is reset to zero.
    Prevents the divergence detector from comparing fresh post-reset CVD
    values against stale pre-reset snapshots, which would produce a false
    massive BULLISH spike.
    """
    self._cvd_candle_snapshots.clear()
    self._last_snapped_candle_ts = 0.0
    self._cvd_delta_ewma_mu  = 0.0
    self._cvd_delta_ewma_var = 1.0
    self._cvd_delta_ewma_n   = 0
    self._cvd_div_streak_dir   = "NONE"
    self._cvd_div_streak_count = 0
    # Suppress divergence for 5 candles post-reset to prevent ghost bearish signal
    self._cvd_post_reset_cooldown = 5
    logger.info("[QuantEngine] CVD divergence state cleared (daily reset hook). "
                "Divergence suppressed for 5 candles to prevent ghost signal.")

# ---------------------------------------------------------------------
# P1: Per-regime win rate accessor
# ---------------------------------------------------------------------
def get_all_regime_priors(self) -> dict:
    """
    Returns dict of regime → win_rate_prior for all 4 regimes.
    Injected into metrics before _compute_signal() so _bayesian_fusion
    can blend per-regime prior with the base Bayesian posterior.
    """
    result = {}
    for regime in ("RANGE", "NEUTRAL", "TREND", "LIQUIDITY"):
        a = self._regime_alpha.get(regime, 1.0)
        b = self._regime_beta.get(regime, 1.0)
        result[regime] = a / (a + b)   # Beta distribution mean
    return result

# ---------------------------------------------------------------------
# State Persistence: Firestore primary, /tmp/ file fallback
# Issue #3 FIX: /tmp/ is wiped on every Railway container restart.
# Firestore survives indefinitely. Load order on boot:
#   1. Firestore (botState/quantEngine document)
#   2. /tmp/ local file (fast path for same-container warm restarts)
#   3. Fresh Beta(5,5) prior (true cold start)
# ---------------------------------------------------------------------
def _save_state(self):
    """
    Persist Bayesian priors and OFI EWMA state.
    Writes to Firestore in a background thread (non-blocking, won't
    slow down the main trading loop) and also writes to /tmp/ as a
    local cache for rapid same-container restarts.
    """
    import threading
    data = {
        "alpha":          self._alpha,
        "beta":           self._beta,
        "regime_alpha":   self._regime_alpha,
        "regime_beta":    self._regime_beta,
        "regime_count":   self._regime_trade_count,
        "ofi_ewma_mu":    self._ofi_ewma_mu,
        "ofi_ewma_var":   self._ofi_ewma_var,
        "ofi_smooth":     self._ofi_smooth,
        "session_vwap_num": self._session_vwap_num,
        "session_vwap_den": self._session_vwap_den,
        "session_date":     str(self._session_date) if self._session_date else None,
        "saved_at":       time.time(),
    }

    # ■■ 1. Firestore write (background thread, non-blocking) ■■■■■■■■
    def _write_firestore(payload: dict) -> None:
        try:
            from bot.heartbeat import get_db
            db = get_db()
            if db is not None:
                db.collection("botState").document("quantEngine").set(payload)
                logger.debug("[QuantEngine] State saved to Firestore ✓")
        except Exception as e:
            logger.warning(f"[QuantEngine] Firestore state save failed: {e}")

    threading.Thread(
        target=_write_firestore,
```

```
        args=(dict(data),),
        daemon=True,
        name="QuantEngine-FSWrite",
    ).start()

    # ■■ 2. /tmp/ file as local cache (synchronous) ■■■■■■■■■■■■■■■■■■■
    try:
        with open(self._persist_path, "w") as f:
            json.dump(data, f)
    except Exception as e:
        logger.warning(f"[QuantEngine] Local state cache write failed: {e}")

def _load_state(self):
    """
    Load Bayesian priors on startup.
    Tries Firestore first (survives Railway deploys), then falls back
    to the /tmp/ local file (useful for same-container warm restarts),
    then initialises a fresh Beta(5,5) prior.
    """
    import os
    if os.getenv("DEV_RESET", "True").lower() in ("true", "1", "yes"):
        import logging
        logger = logging.getLogger(__name__)
        logger.info("[QuantEngine] DEV_RESET=True - skipping Firestore/local cache. Starting with fresh Beta(5,5) p
        return
    # ■■ 1. Try Firestore ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    try:
        from bot.heartbeat import get_db
        db = get_db()
        if db is not None:
            doc = db.collection("botState").document("quantEngine").get()
            if doc.exists:
                data = doc.to_dict()
                self._alpha = float(data.get("alpha", 5.0))
                self._beta  = float(data.get("beta",  5.0))
                saved_ra = data.get("regime_alpha", {})
                saved_rb = data.get("regime_beta",  {})
                saved_rc = data.get("regime_count", {})
                for r in ("RANGE", "NEUTRAL", "TREND", "LIQUIDITY"):
                    self._regime_alpha[r]       = float(saved_ra.get(r, 5.0))
                    self._regime_beta[r]        = float(saved_rb.get(r, 5.0))
                    self._regime_trade_count[r] = int(saved_rc.get(r, 0))

                self._session_vwap_num = float(data.get("session_vwap_num", 0.0))
                self._session_vwap_den = float(data.get("session_vwap_den", 0.0))
                s_date = data.get("session_date")
                from datetime import datetime
                if s_date and s_date != "None":
                    try:
                        self._session_date = datetime.strptime(s_date, "%Y-%m-%d").date()
                    except:
                        self._session_date = None

                n_trades = sum(self._regime_trade_count.values())
                p_bull   = self._alpha / (self._alpha + self._beta)
                logger.info(
                    f"[QuantEngine] ■ State loaded from Firestore — "
                    f"α={self._alpha:.0f} β={self._beta:.0f} "
                    f"P(bull)={p_bull:.1%} | "
                    f"n_trades={n_trades} "
                    f"(saved {(time.time() - float(data.get('saved_at', time.time()))):.0f}s ago)"
                )
                return  # ← success: stop here
            else:
                logger.info("[QuantEngine] Firestore botState/quantEngine: no document yet — checking /tmp/")
    except Exception as e:
        logger.warning(f"[QuantEngine] Firestore state load failed: {e} — falling back to /tmp/")

    # ■■ 2. Try /tmp/ local file ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    try:
        if os.path.exists(self._persist_path):
            with open(self._persist_path) as f:
                data = json.load(f)
            self._alpha = float(data.get("alpha", 5.0))
            self._beta  = float(data.get("beta",  5.0))
            saved_ra = data.get("regime_alpha", {})
            saved_rb = data.get("regime_beta",  {})
            saved_rc = data.get("regime_count", {})
```

```
            for r in ("RANGE", "NEUTRAL", "TREND", "LIQUIDITY"):
                self._regime_alpha[r]       = float(saved_ra.get(r, 5.0))
                self._regime_beta[r]        = float(saved_rb.get(r, 5.0))
                self._regime_trade_count[r] = int(saved_rc.get(r, 0))

            self._session_vwap_num = float(data.get("session_vwap_num", 0.0))
            self._session_vwap_den = float(data.get("session_vwap_den", 0.0))
            s_date = data.get("session_date")
            from datetime import datetime
            if s_date and s_date != "None":
                try:
                    self._session_date = datetime.strptime(s_date, "%Y-%m-%d").date()
                except:
                    self._session_date = None
            logger.info(
                "[QuantEngine] ■■ State loaded from /tmp/ (Firestore unavailable). "
                "Prior will be lost on next Railway deploy."
            )
            return  # ← success
        except Exception as e:
            logger.warning(f"[QuantEngine] /tmp/ state load error: {e}")

        # ■■ 3. Cold start — fresh Beta(5,5) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
        for r in ("RANGE", "NEUTRAL", "TREND", "LIQUIDITY"):
            self._regime_alpha[r]       = 5.0
            self._regime_beta[r]        = 5.0
            self._regime_trade_count[r] = 0
        self._alpha = 5.0
        self._beta  = 5.0
        logger.info("[QuantEngine] ■ Cold start: Beta(5,5) uninformed prior.")


    # ------------------------------------------------------------------
    # Public API
    # ------------------------------------------------------------------
    def compute_metrics(self) -> Optional[Dict[str, Any]]:
        """
        Returns a fully-populated metrics dict, or None if not enough data.
        """
        if len(self.state.candles) < self.MIN_CANDLES:
            return None

        c_list = self.state.candles
        closes = np.array([c['close'] for c in c_list], dtype=float)
        highs  = np.array([c['high']  for c in c_list], dtype=float)
        lows   = np.array([c['low']   for c in c_list], dtype=float)
        vols   = np.array([c['volume']for c in c_list], dtype=float)

        current_price = closes[-1]

        # Calculate ATR and rank early for session VWAP and fix BUG-V3-03
        atr = self._atr(highs, lows, closes)
        self._atr_history.append(atr)
        arr = np.array(list(self._atr_history))
        atr_pct_rank = float(np.mean(arr <= atr))

        skewness       = self._skewness(closes)
        z_score        = self._session_vwap_z(highs, lows, closes, vols, current_price, atr_pct_rank)
        zScore_prev    = getattr(self, "_last_z_score", z_score)
        self._last_z_score = z_score

        # MED-1 FIX: Use cached RSI history for rsi_prev/rsi_prev2.
        # Re-slicing closes[] and recomputing _rsi() produces a different Wilder EMA
        # seed each time — not the value that was observed last cycle. Cache instead.
        rsi = self._rsi(closes)
        self._rsi_history.append(rsi)
        rsi_prev  = self._rsi_history[-2] if len(self._rsi_history) >= 2 else rsi
        rsi_prev2 = self._rsi_history[-3] if len(self._rsi_history) >= 3 else rsi_prev

        tape_speed, dominant_side = self._tape_metrics()
        ofi, wall_context, all_walls_str, execution_price, valid_bids, valid_asks, nearest_bid, nearest_ask, top_bids,
        z_ret = self._log_return_z_score(closes)
        # Lightweight regime proxy for z_vel gating: |Z_t| > 1.5 ≈ TREND-like.
        # Full HMM regime is determined later in main.py; this gives quant engine
        # a useful signal without a circular dependency on the regime classifier.
        _regime_proxy = "TREND" if abs(z_score) > 1.5 else "NEUTRAL"
        bayesian_posterior = self._bayesian(rsi, z_score, skewness, ofi,
                                            z_ret=z_ret, regime=_regime_proxy)
```

```
cvd = self.state.cvd

vpoc = self._volume_poc(c_list)
funding_rate = getattr(self.state, 'funding_rate', 0.0)

last_trade_ts = getattr(self.state, '_last_trade_ts', 0.0)
trade_buffer_healthy = (
    last_trade_ts > 0 and
    (time.time() - last_trade_ts) < 30.0 and
    len(self.state.recent_trades) >= 5
)

# ■■ CVD Candle Snapshot (Divergence Layer) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Detect a new candle by comparing the latest candle's timestamp to the
# last one we snapped.  When a new candle has opened (previous one closed),
# record the closed candle's OHLC and the CVD value AT THAT MOMENT.
current_candle_ts = float(c_list[-1].get("time", 0.0)) if c_list else 0.0
if (current_candle_ts != self._last_snapped_candle_ts
        and len(c_list) >= 2):
    closed_candle = c_list[-2]
    snap = {
        "candle_ts": float(closed_candle.get("time",  0.0)),
        "close":     float(closed_candle.get("close", current_price)),
        "low":       float(closed_candle.get("low",   current_price)),
        "high":      float(closed_candle.get("high",  current_price)),
        "volume":    float(closed_candle.get("volume", 0.0)),
        "cvd":       float(cvd),
    }
    self._cvd_candle_snapshots.append(snap)
    self._last_snapped_candle_ts = current_candle_ts
    logger.debug(
        f"[CVDSnap] New candle — snapped CVD={cvd:.0f} "
        f"low={snap['low']:.2f} high={snap['high']:.2f} vol={snap['volume']:.1f}"
    )

# ■■ CVD Divergence Detection ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
cvd_divergence = self._cvd_divergence(current_price, atr)

return {
    "symbol":            self.state.symbol,
    "price":             current_price,
    "skewness":          skewness,
    "bayesianPosterior": bayesian_posterior,
    "zScore":            z_score,
    "zScore_prev":       zScore_prev,
    "rsi":               rsi,
    "rsi_prev":          rsi_prev,
    "rsi_prev2":         rsi_prev2,
    "ofi":               ofi,
    "cvd":               cvd,
    "tapeSpeed":         tape_speed,
    "tapeDominant":      dominant_side,
    "wallContext":       wall_context,
    "allWalls":          all_walls_str,
    "atr":               atr,
    "atr_pct":           atr / current_price if current_price > 0 else 0.0,
    "atr_pct_rank":      atr_pct_rank,   # P2: percentile rank in 30-day window
    "vpoc":              vpoc,
    "funding_rate":      funding_rate,
    # PHASE-0.4: Z-score validity guard.
    # Z-score is meaningless with fewer than 10 bars of data in the
    # current session — it's fitting noise from too-small a sample.
    "z_score_valid":     len(closes) >= 10,
    # Z-06: Log-Return Z-Score — reuse value computed for _bayesian() above
    "zScore_ret":        z_ret,
    # PHASE-0.2: aggTrade stream health flag.
    "trade_buffer_healthy": trade_buffer_healthy,
    # PHASE-0.4: Order book mid-price for execution (not candle close).
    "execution_price":   execution_price,
    # PHASE-0.3: Structured wall data for _detect_regime significance filter
    "bid_depths":        valid_bids,
    "ask_depths":        valid_asks,
    "nearest_buy_wall":  nearest_bid,
    "nearest_sell_wall": nearest_ask,
    "top_buy_walls":     top_bids,
    "top_sell_walls":    top_asks,
    # ■■ CVD Divergence ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    "cvd_divergence":    cvd_divergence,
```

```
    }

# ------------------------------------------------------------------
# 1. Log-Return Skewness (50 periods)
# ------------------------------------------------------------------
def _skewness(self, closes: np.ndarray) -> float:
    """FIX DIV-M1: 20-bar window matches Z-score temporal alignment."""
    closes_21 = closes[-21:]  # was -51 (50 bars) — now 20 bars
    if len(closes_21) < 21:
        return 0.0
    safe = np.where(closes_21[:-1] > 0, closes_21[:-1], np.nan)
    returns_20 = np.log(closes_21[1:] / safe)
    valid = returns_20[~np.isnan(returns_20)]
    if len(valid) < 3:
        return 0.0
    mean = np.mean(valid); var = np.var(valid)
    if var == 0: return 0.0
    return float(np.mean(((valid - mean) / np.sqrt(var)) ** 3))

# ------------------------------------------------------------------
# 1b. Log-Return Z-Score (Z-06) — momentum / breakout detection
# ------------------------------------------------------------------
def _log_return_z_score(self, closes: np.ndarray, window: int = 20) -> float:
    """
    Z-06: Measures whether the current price VELOCITY (log-return) is
    statistically unusual compared to recent history.  Unlike the VWAP
    Z-score which measures price LEVEL deviation (mean-reversion), this
    measures price SPEED deviation — the correct signal for TREND momentum.

    Formula (PDF §5 / Z-06):
        r_t   = ln(P_t / P_{t-1})
        r■     = mean of last `window` log-returns
        σ_r   = std  of last `window` log-returns
        Z_ret = (r_t - r■) / σ_r

    Output clipped to [-4, +4].  Returns 0.0 if insufficient data.
    """
    if len(closes) < window + 2:
        return 0.0
    log_rets = np.log(closes[-(window + 1):][1:] / closes[-(window + 1):][:-1])
    valid = log_rets[np.isfinite(log_rets)]
    if len(valid) < 5:
        return 0.0
    mu    = float(np.mean(valid))
    sigma = float(np.std(valid))
    if sigma < 1e-10:
        return 0.0
    z_ret = (float(valid[-1]) - mu) / sigma
    return float(np.clip(z_ret, -4.0, 4.0))

# ------------------------------------------------------------------
# 2. VWAP-Anchored Z-Score — True Session VWAP (Phase 2)
# ------------------------------------------------------------------
def _session_vwap_z(
    self,
    highs: np.ndarray,
    lows: np.ndarray,
    closes: np.ndarray,
    vols: np.ndarray,
    current_price: float,
    atr_pct_rank: float = 0.5,
) -> float:
    from datetime import datetime, timezone
    today = datetime.now(timezone.utc).date()
    if self._session_date != today:
        self._session_vwap_num = 0.0
        self._session_vwap_den = 0.0
        self._session_date = today

    if len(closes) == 0:
        return 0.0

    # Update running VWAP with latest candle
    tp = float((highs[-1] + lows[-1] + closes[-1]) / 3.0)
    v = float(vols[-1])
    self._session_vwap_num += tp * v
    self._session_vwap_den += v
    if self._session_vwap_den <= 0:
```

```
        return 0.0
    vwap_session = self._session_vwap_num / self._session_vwap_den

    # Adaptive sigma window
    n_sig = max(10, int(atr_pct_rank * 40))
    if len(closes) < n_sig:
        n_sig = len(closes)

    if n_sig <= 0:
        return 0.0

    h = highs[-n_sig:]
    l = lows[-n_sig:]
    c = closes[-n_sig:]
    vv = vols[-n_sig:]

    tp_w = (h + l + c) / 3.0
    vol_sum = np.sum(vv)
    if vol_sum <= 0:
        return 0.0

    vw_var = np.sum(vv * (tp_w - vwap_session)**2) / vol_sum
    std = np.sqrt(vw_var)

    # P1-7 FIX: Guard against near-zero std during zero-volatility periods.
    MIN_STD = current_price * 0.00005  # 0.005% of current price
    if std < MIN_STD:
        return 0.0

    # t-scale correction factor
    NU_FIXED = 6.0
    t_scale = math.sqrt((NU_FIXED - 2.0) / NU_FIXED)  # = sqrt(4/6) = 0.8165
    z = (current_price - vwap_session) / std
    return float(np.clip(z * t_scale, -4.0, 4.0))

# ------------------------------------------------------------------
# 3. RSI (14 period) — Wilder EMA smoothing (Fix #7 from audit)
# ------------------------------------------------------------------
def _rsi(self, closes: np.ndarray) -> float:
    """
    Wilder's RSI: uses exponential smoothing with α = 1/period.
    This matches TradingView, TA-Lib, and the frontend dashboard.
    SMA-based RSI under-estimates extremes — Wilder's EMA gives
    accurate 70/30 signals for overbought/oversold gate logic.
    """
    period = 14
    if len(closes) < period + 1:
        return 50.0
    delta  = np.diff(closes)
    gains  = np.where(delta > 0, delta, 0.0)
    losses = np.where(delta < 0, -delta, 0.0)

    # Seed with simple average over first `period` bars
    avg_gain = float(np.mean(gains[:period]))
    avg_loss = float(np.mean(losses[:period]))

    # Wilder EMA over remaining bars
    for i in range(period, len(gains)):
        avg_gain = (avg_gain * (period - 1) + gains[i]) / period
        avg_loss = (avg_loss * (period - 1) + losses[i]) / period

    if avg_loss < 1e-10:
        return 100.0 if avg_gain > 0 else 50.0

    rs = avg_gain / avg_loss
    return float(100.0 - (100.0 / (1.0 + rs)))

# ------------------------------------------------------------------
# 4. Tape Speed & Dominant Side (USD-normalised)
# ------------------------------------------------------------------
def _tape_metrics(self):
    trades = list(self.state.recent_trades)
    now_ms = time.time() * 1000

    buy_10s = sell_10s = total_60s = 0.0
    for t in trades:
        total_60s += t['usd_volume']
        age_ms     = now_ms - t['time']
```

```
        if age_ms < 10_000:
            if t['side'] == 'BUY':
                buy_10s += t['usd_volume']
            else:
                sell_10s += t['usd_volume']

    total_10s    = buy_10s + sell_10s
    baseline_10s = (total_60s / 60.0) * 10 if total_60s > 0 else 0.0

    is_screaming = (
        (baseline_10s > 0 and total_10s > baseline_10s * 3)
        or total_10s > 2_000_000
    )

    if len(trades) < 5:
        return "NORMAL", "NO DATA"

    tape_speed = "SCREAMING" if is_screaming else "NORMAL"
    if buy_10s > sell_10s * 1.5:
        dominant = "BUY (ASK HIT)"
    elif sell_10s > buy_10s * 1.5:
        dominant = "SELL (BID HIT)"
    else:
        dominant = "BALANCED"

    return tape_speed, dominant

# ------------------------------------------------------------------
# 5. LOB Walls & OFI
# ------------------------------------------------------------------
def _lob_metrics(self, current_price: float):
    bids = self.state.bids
    asks = self.state.asks

    MAX_DIST   = 0.001
    valid_bids = {p: s for p, s in bids.items()
                  if (current_price - p) / current_price <= MAX_DIST}
    valid_asks = {p: s for p, s in asks.items()
                  if (p - current_price) / current_price <= MAX_DIST}

    ofi = sum(valid_bids.values()) - sum(valid_asks.values())

    all_sizes      = list(valid_bids.values()) + list(valid_asks.values())
    median_size    = float(np.median(all_sizes)) if all_sizes else 1.0
    wall_threshold = median_size * 5.0

    # PHASE-1.1: Sort by DISTANCE to price (ascending = nearest first).
    # Previously sorted by SIZE (largest first) — wrong for sweep detection.
    # Sweep = price crossing a nearby wall, not a large distant wall.
    def bid_distance(p, s):
        return (current_price - p) / current_price  # fraction of price
    def ask_distance(p, s):
        return (p - current_price) / current_price

    top_bids = sorted(
        [(p, s) for p, s in valid_bids.items() if s > wall_threshold],
        key=lambda x: bid_distance(*x),
    )[:5]
    top_asks = sorted(
        [(p, s) for p, s in valid_asks.items() if s > wall_threshold],
        key=lambda x: ask_distance(*x),
    )[:5]

    nearest_bid = top_bids[0][0] if top_bids else None
    nearest_ask = top_asks[0][0] if top_asks else None

    bid_dist = (
        f"{((current_price - nearest_bid) / current_price * 100):.2f}"
        if nearest_bid else "N/A"
    )
    ask_dist = (
        f"{((nearest_ask - current_price) / current_price * 100):.2f}"
        if nearest_ask else "N/A"
    )

    # PHASE-1.1: Top-of-book bid/ask for execution price (not wall-filtered mid).
    # Wall prices can be $50+ off actual spread on BTC; use actual best bid/ask.
    best_bid = max(self.state.bids.keys()) if self.state.bids else current_price
```

```
best_ask = min(self.state.asks.keys()) if self.state.asks else current_price
execution_price = (best_bid + best_ask) / 2.0 if (self.state.bids and self.state.asks) else current_price

wall_context = (
    f"Nearest Buy Wall: {nearest_bid} (-{bid_dist}%), "
    f"Nearest Sell Wall: {nearest_ask} (+{ask_dist}%)"
)

wall_parts = []
for p, s in top_asks:
    if current_price > 0:
        dist = (p - current_price) / current_price * 100
        wall_parts.append(f"SELL@{p:.1f} (+{dist:.2f}%, sz:{s:.2f})")
for p, s in top_bids:
    if current_price > 0:
        dist = (current_price - p) / current_price * 100
        wall_parts.append(f"BUY@{p:.1f} (-{dist:.2f}%, sz:{s:.2f})")

# ■■ THREE-STAGE OFI INTEGRITY PIPELINE ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Replaces the static ±100 clamp (Links Investment Corps, Apr 2026)

# STAGE 1 — True OFI via depth-snapshot delta
# The bot uses depth20@100ms (full snapshots, not delta stream),
# so we measure the CHANGE in total bid/ask depth between frames.
# This is true Order Flow Imbalance, not just Order Book Imbalance.
curr_bid_depth = sum(valid_bids.values())
curr_ask_depth = sum(valid_asks.values())

if self._prev_bid_depth == 0.0 and self._prev_ask_depth == 0.0:
    ofi_raw = 0.0  # First cycle: no delta available yet, emit neutral
else:
    ofi_raw = (
        (curr_bid_depth - self._prev_bid_depth) -
        (curr_ask_depth - self._prev_ask_depth)
    )
self._prev_bid_depth = curr_bid_depth
self._prev_ask_depth = curr_ask_depth

# STAGE 2 — MAD Persistence Filter
# Classifies transient spikes (1-2 cycles) vs sustained extremes (3+ cycles).
# Only sustained extremes pass through (genuine institutional pressure).
self._ofi_history.append(ofi_raw)
ofi_filtered = ofi_raw  # default: pass through

if len(self._ofi_history) >= 20:
    arr = np.array(self._ofi_history)
    median_ofi = float(np.median(arr))
    mad = float(np.median(np.abs(arr - median_ofi)))
    mad_threshold = 3.5 * 1.4826 * mad  # 3.5σ robust boundary — NOT changed per plan

    if mad_threshold > 0 and abs(ofi_raw - median_ofi) > mad_threshold:
        self._ofi_outlier_count += 1
        if self._ofi_outlier_count < 3:
            # Transient spike (1-2 cycles) — replace with neutral median
            logger.debug(
                f"[OFI-MAD] Transient outlier ({ofi_raw:.2f}) — "
                f"replacing with median ({median_ofi:.2f})"
            )
            ofi_filtered = median_ofi
        else:
            # Sustained extreme (3+ cycles) — genuine, cap at MAD boundary
            sign = 1.0 if ofi_raw >= 0 else -1.0
            ofi_filtered = sign * (abs(median_ofi) + mad_threshold)
            self._ofi_outlier_count = 0
            logger.debug(
                f"[OFI-MAD] Sustained extreme ({ofi_raw:.2f}) — "
                f"capped at MAD boundary ({ofi_filtered:.2f})"
            )
    else:
        self._ofi_outlier_count = 0

# STAGE 3 — EWMA normalisation + tanh soft saturation
# Output is in (-1, +1). Replaces the static ±100 hard clamp.
# tanh is monotone — preserves direction, never hard-caps genuine signals.
LAMBDA_EWMA  = 0.94   # PHASE-4.1: Was 0.97. Half-life: ~11 obs (~2.8 min) instead of ~23 obs (~5.7 min)
ALPHA_SMOOTH = 0.35   # PHASE-4.1: Was 0.30. Slightly faster signal response
self._ofi_ewma_mu  = LAMBDA_EWMA * self._ofi_ewma_mu  + (1 - LAMBDA_EWMA) * ofi_filtered
self._ofi_ewma_var = LAMBDA_EWMA * self._ofi_ewma_var + (1 - LAMBDA_EWMA) * (ofi_filtered - self._ofi_ewma_mu)
```

```
    sigma = max(math.sqrt(self._ofi_ewma_var), 1e-6)

    ofi_norm   = (ofi_filtered - self._ofi_ewma_mu) / sigma
    self._ofi_smooth = ALPHA_SMOOTH * ofi_norm + (1 - ALPHA_SMOOTH) * self._ofi_smooth
    ofi = math.tanh(self._ofi_smooth / 2.0)  # output: (-1, +1)

    logger.debug(
        f"[OFI-Pipeline] raw={ofi_raw:.2f} filtered={ofi_filtered:.2f} "
        f"norm={ofi_norm:.3f} smooth={self._ofi_smooth:.3f} final_tanh={ofi:.4f}"
    )

    return ofi, wall_context, "; ".join(wall_parts), execution_price, valid_bids, valid_asks, nearest_bid, nearest_

# ------------------------------------------------------------------
# 6. Bayesian Posterior P(Bull | Evidence) — Beta conjugate prior
# ------------------------------------------------------------------
def _bayesian(self, rsi: float, z_score: float, skewness: float,
              ofi: float, z_ret: float = 0.0, regime: str = "NEUTRAL") -> float:
    """
    Beta(α,β) conjugate prior — self-calibrates to historical win rate.

    OFI is now in (-1, +1) from the Three-Stage Pipeline (tanh output).
    Thresholds updated from the old ±100 scale to the new ±1 scale.

    z_ret  : Log-Return Z-Score (Z-06) — velocity-based signal for TREND.
    regime : Current HMM regime — gates the z_ret likelihood to TREND only.
    """
    # ■■ Step 1: Beta prior ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    p_prior = self._alpha / (self._alpha + self._beta)
    p_prior = max(0.01, min(0.99, p_prior))
    prior_odds = p_prior / (1.0 - p_prior)

    # ■■ Step 2: RSI likelihood ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    L_rsi = 1.8 if rsi > 60 else 0.55 if rsi < 40 else 1.0

    # ■■ Step 3: OFI + Z-Score combined gate (OFI now in (-1,+1)) ■■■■■■■■■■
    # M-02 FIX: z_score is already Z_t (shrunk by t_scale=0.8165).
    # Thresholds updated 1.5 → 1.22 to match corrected signal_config.py.
    if z_score < -1.22 and ofi > 0.2:       # oversold + buying flow
        L_flow = 2.0
    elif z_score > 1.22 and ofi < -0.2:     # overbought + selling flow
        L_flow = 0.5
    else:
        L_z = 1.3 if z_score < -1.22 else 0.76 if z_score > 1.22 else 1.0
        L_o = 1.2 if ofi > 0.3 else 0.83 if ofi < -0.3 else 1.0
        L_flow = L_z * L_o

    # ■■ Step 4: Skewness likelihood ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    L_skew = 1.2 if skewness > 0.3 else 0.83 if skewness < -0.3 else 1.0

    # ■■ Step 4b: Z_vel likelihood — TREND regime only (C2 / tx.txt fix) ■■
    # VWAP Z measures price LEVEL vs anchor (mean-reversion signal).
    # Z_vel measures price SPEED vs recent history (momentum signal).
    # In TREND regime the VWAP signal is uninformative; Z_vel fills the gap.
    # Capped at ±25% odds multiplier — conservative during cold-start prior.
    L_zvel = 1.0
    if regime == "TREND" and abs(z_ret) > 0.5:
        if z_ret > 1.5:
            L_zvel = 1.25   # Strong upward velocity → strong bullish evidence
        elif z_ret < -1.5:
            L_zvel = 0.80   # Strong downward velocity → bearish drag on long odds
        elif z_ret > 0.5:
            L_zvel = 1.10   # Mild upward momentum → slight boost
        else:
            L_zvel = 0.93   # Mild downward pressure → slight penalty

    # ■■ Step 5: Update odds ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    posterior_odds = prior_odds * L_rsi * L_flow * L_skew * L_zvel

    # ■■ Step 6: Convert back to probability ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    p_bull = posterior_odds / (1.0 + posterior_odds)

    logger.debug(
        f"[QuantEngine] Bayesian — prior={p_prior:.2%} "
        f"L_rsi={L_rsi:.2f} L_flow={L_flow:.2f} L_skew={L_skew:.2f} "
        f"L_zvel={L_zvel:.2f}(regime={regime} z_ret={z_ret:.2f}) "
        f"ofi_tanh={ofi:.4f} → posterior={p_bull:.2%}"
    )
```

```
    return float(p_bull)


# ---------------------------------------------------------------------
# 7. ATR — Average True Range (14 periods)
# ---------------------------------------------------------------------
def _atr(self, highs: np.ndarray, lows: np.ndarray,
         closes: np.ndarray, period: int = 14) -> float:
    """Wilder ATR — 3x period warmup ensures EMA loop always executes.
    FIX NEW-C2: Previous period+1 lookback produced empty loop.
    """
    WARMUP = period * 3  # 42 bars for period=14
    lookback = WARMUP + 1  # 43 bars total
    if len(closes) < period + 1:
        return 0.0
    if len(closes) >= lookback:
        h = highs[-lookback:]; l = lows[-lookback:]; c = closes[-lookback:]
    else:
        h = highs; l = lows; c = closes
    prev_c = c[:-1]
    tr = np.maximum(h[1:]-l[1:], np.maximum(np.abs(h[1:]-prev_c), np.abs(l[1:]-prev_c)))
    if len(tr) < period:
        return float(np.mean(tr)) if len(tr) > 0 else 0.0
    atr_val = float(np.mean(tr[:period]))
    for i in range(period, len(tr)):
        atr_val = (atr_val * (period - 1) + float(tr[i])) / period
    return float(atr_val)

# ---------------------------------------------------------------------
# 8. Volume Point of Control (VPOC)
# ---------------------------------------------------------------------
def _volume_poc(self, candles, n: int = 24) -> Optional[float]:
    """
    Identify the $10 price bucket with the highest cumulative traded
    volume over the last N candles (default 24 = ~6 hours on 15m bars).
    """
    candle_list = list(candles)[-n:]
    if len(candle_list) < 2:
        return None
    price_vol: Dict[int, float] = {}
    for c in candle_list:
        bucket_size = max(10.0, round(float(c["close"]) * 0.001 / 10.0) * 10)
        bucket = int(round(float(c["close"]) / bucket_size)) * bucket_size
        price_vol[bucket] = price_vol.get(bucket, 0.0) + float(c.get("volume", 0.0))
    if not price_vol:
        return None
    return float(max(price_vol, key=price_vol.get))

# ---------------------------------------------------------------------
# CVD Divergence Detector — Institutional Alpha v2.0
# Fixes: #1 EMA-Z normalization | #2 True ATR | #3 Swing extrema
#        #4 Volume confirmation  | #5 Uncertainty penalty data
# ---------------------------------------------------------------------
def _cvd_divergence(self, current_price: float, true_atr: float = 0.0) -> Dict[str, Any]:
    """
    Institutional-grade CVD divergence detector operating on per-candle
    close snapshots.  All six head-office criticisms addressed.

    ■■ FIX #1: EMA-VARIANCE NORMALIZATION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    Replaces the fragile session-range denominator using Welford's online EMA.

    ■■ FIX #2: TRUE ATR ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    Receives the Wilder 14-period ATR already computed in compute_metrics().

    ■■ FIX #3: SWING-EXTREMA ADAPTIVE LOOKBACK ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    Finds swing low/high within full snapshot window instead of fixed k-back.

    ■■ FIX #4: VOLUME SPIKE CONFIRMATION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    Institutional absorption/distribution requires elevated volume.

    ■■ FIX #5: SNAPSHOT COUNT FOR UNCERTAINTY SCALING ■■■■■■■■■■■■■■■■■■■■■■■■■■
    Returns n_snaps so the gate can apply data-driven penalties.

    Returns:
        {
            "type":              "BULLISH" | "BEARISH" | "NONE",
            "strength":          float ∈ [0, 1],
            "candles_confirmed": int,
```

```
        "price_delta":       float,
        "cvd_delta":         float,
        "lookback_k":        int,
        "vol_spike":         float,
        "detection_method":  "swing_extrema" | "sequential" | "none",
        "n_snaps":           int,
    }
"""
λ = 0.92

NULL_RESULT = {
    "type": "NONE", "strength": 0.0,
    "candles_confirmed": 0, "price_delta": 0.0,
    "cvd_delta": 0.0, "lookback_k": 0,
    "vol_spike": 1.0, "detection_method": "none",
    "n_snaps": len(self._cvd_candle_snapshots),
}

snaps = list(self._cvd_candle_snapshots)
n_snaps = len(snaps)

# ■■ Post-reset cooldown ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Suppress divergence for N candles after daily CVD reset.
# Each call to _cvd_divergence() corresponds to one compute_metrics()
# cycle (~15 s). At 5 candles = ~75 s of suppression after reset.
if self._cvd_post_reset_cooldown > 0:
    self._cvd_post_reset_cooldown -= 1
    logger.debug(
        f"[CVDDiv] Post-reset cooldown: {self._cvd_post_reset_cooldown} cycles remain. "
        "Returning NONE to prevent ghost signal."
    )
    NULL_RESULT["n_snaps"] = n_snaps
    return NULL_RESULT

if n_snaps < 2:
    return NULL_RESULT

latest = snaps[-1]

atr_ref = max(true_atr * 1.5, current_price * 0.003, 1.0)

all_vols = [s.get("volume", 0.0) for s in snaps]
mean_vol = float(np.mean(all_vols)) if all_vols else 1.0
mean_vol = max(mean_vol, 1e-8)

if n_snaps >= 2:
    latest_delta_raw = snaps[-1]["cvd"] - snaps[-2]["cvd"]
    self._cvd_delta_ewma_mu  = (λ * self._cvd_delta_ewma_mu +
                                (1 - λ) * latest_delta_raw)
    self._cvd_delta_ewma_var = (λ * self._cvd_delta_ewma_var +
                                (1 - λ) * (latest_delta_raw - self._cvd_delta_ewma_mu) ** 2)
    self._cvd_delta_ewma_n  += 1

ewma_std = math.sqrt(max(self._cvd_delta_ewma_var, 1.0))

def _norm_cvd(cvd_delta_raw: float) -> float:
    z = (cvd_delta_raw - self._cvd_delta_ewma_mu) / ewma_std
    return float(np.clip(abs(math.tanh(z / 2.0)), 0.0, 1.0))

def _norm_price(price_delta: float) -> float:
    return float(min(abs(price_delta) / atr_ref, 1.0))

def _norm_vol(snap_vol: float) -> float:
    spike = snap_vol / mean_vol
    return float(np.clip(math.tanh(spike / 2.0), 0.0, 1.0))

def _age_weight(k: int) -> float:
    if k <= 4:
        return 1.0
    return math.exp(-0.30 * (k - 4))

def _score(price_delta: float, cvd_delta_raw: float, vol_at_extreme: float, k: int) -> float:
    nP   = _norm_price(price_delta)
    nC   = _norm_cvd(cvd_delta_raw)
    nV   = _norm_vol(vol_at_extreme)
    age  = _age_weight(k)
    return age * (0.35 * nP + 0.45 * nC + 0.20 * nV)
swing_bull_result = None
```

```
swing_bear_result = None

if n_snaps >= 3:
    swing_low_idx = int(np.argmin([s["low"] for s in snaps]))
    swing_low_snap = snaps[swing_low_idx]
    k_swing_bull = n_snaps - 1 - swing_low_idx

    if (swing_low_idx < n_snaps - 1 and
            latest["low"] > swing_low_snap["low"] and
            latest["cvd"] > swing_low_snap["cvd"]):
        price_delta_mag  = abs(latest["low"] - swing_low_snap["low"])
        cvd_delta_raw    = latest["cvd"] - swing_low_snap["cvd"]
        vol_at_low       = swing_low_snap.get("volume", mean_vol)
        s = _score(price_delta_mag, cvd_delta_raw, vol_at_low, k_swing_bull)
        swing_bull_result = (s, k_swing_bull, price_delta_mag,
                             cvd_delta_raw, vol_at_low / mean_vol)

    swing_high_idx = int(np.argmax([s["high"] for s in snaps]))
    swing_high_snap = snaps[swing_high_idx]
    k_swing_bear = n_snaps - 1 - swing_high_idx

    if (swing_high_idx < n_snaps - 1 and
            latest["high"] < swing_high_snap["high"] and
            latest["cvd"] < swing_high_snap["cvd"]):
        price_delta_mag  = abs(latest["high"] - swing_high_snap["high"])
        cvd_delta_raw    = swing_high_snap["cvd"] - latest["cvd"]
        vol_at_high      = swing_high_snap.get("volume", mean_vol)
        s = _score(price_delta_mag, cvd_delta_raw, vol_at_high, k_swing_bear)
        swing_bear_result = (s, k_swing_bear, price_delta_mag,
                             cvd_delta_raw, vol_at_high / mean_vol)

max_k = min(8, n_snaps - 1)
seq_bull_best  = (0.0, 0, 0.0, 0.0, 1.0)
seq_bear_best  = (0.0, 0, 0.0, 0.0, 1.0)
bull_confirms  = 0
bear_confirms  = 0

for k in range(1, max_k + 1):
    prior = snaps[-(k + 1)]

    if latest["low"] < prior["low"] and latest["cvd"] > prior["cvd"]:
        bull_confirms += 1
        pd   = prior["low"] - latest["low"]
        cd   = latest["cvd"] - prior["cvd"]
        vl   = prior.get("volume", mean_vol)
        s    = _score(pd, cd, vl, k)
        if s > seq_bull_best[0]:
            seq_bull_best = (s, k, pd, cd, vl / mean_vol)

    if latest["high"] > prior["high"] and latest["cvd"] < prior["cvd"]:
        bear_confirms += 1
        pd   = latest["high"] - prior["high"]
        cd   = prior["cvd"] - latest["cvd"]
        vl   = prior.get("volume", mean_vol)
        s    = _score(pd, cd, vl, k)
        if s > seq_bear_best[0]:
            seq_bear_best = (s, k, pd, cd, vl / mean_vol)

best_bull_s, best_bull_k, best_bull_pd, best_bull_cd, best_bull_vs = 0.0, 0, 0.0, 0.0, 1.0
best_bull_meth = "none"
if swing_bull_result and swing_bull_result[0] >= seq_bull_best[0]:
    best_bull_s, best_bull_k, best_bull_pd, best_bull_cd, best_bull_vs = swing_bull_result
    best_bull_meth = "swing_extrema"
elif seq_bull_best[0] > 0.0:
    best_bull_s, best_bull_k, best_bull_pd, best_bull_cd, best_bull_vs = seq_bull_best
    best_bull_meth = "sequential"

best_bear_s, best_bear_k, best_bear_pd, best_bear_cd, best_bear_vs = 0.0, 0, 0.0, 0.0, 1.0
best_bear_meth = "none"
if swing_bear_result and swing_bear_result[0] >= seq_bear_best[0]:
    best_bear_s, best_bear_k, best_bear_pd, best_bear_cd, best_bear_vs = swing_bear_result
    best_bear_meth = "swing_extrema"
elif seq_bear_best[0] > 0.0:
    best_bear_s, best_bear_k, best_bear_pd, best_bear_cd, best_bear_vs = seq_bear_best
    best_bear_meth = "sequential"

atr_rank = min(max(getattr(self, '_atr_pct_rank', 0.5), 0.1), 1.0)
MIN_STRENGTH = 0.10 + (0.10 * atr_rank)
```

```
AGE_CAP_CANDLES = 8   # beyond 8 candles (~2h) in same direction = TREND, not divergence
DECAY_RATE      = 0.15  # exp(-0.15*(streak-8)); at streak=12: factor≈0.55 → below threshold

if best_bull_s >= MIN_STRENGTH or best_bear_s >= MIN_STRENGTH:
    # Determine winning direction BEFORE applying age cap
    winning_dir = "BULLISH" if best_bull_s >= best_bear_s else "BEARISH"
    winning_s   = best_bull_s if winning_dir == "BULLISH" else best_bear_s

    # ■■ Streak / Age-Cap Logic ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    # Track how many consecutive candle snapshots this direction has fired.
    # If it exceeds AGE_CAP_CANDLES, the "divergence" is actually a trend —
    # decay strength so it naturally falls below MIN_STRENGTH and expires.
    if winning_dir == self._cvd_div_streak_dir:
        self._cvd_div_streak_count += 1
    else:
        self._cvd_div_streak_dir   = winning_dir
        self._cvd_div_streak_count = 1

    streak = self._cvd_div_streak_count
    if streak > AGE_CAP_CANDLES:
        decay = math.exp(-DECAY_RATE * (streak - AGE_CAP_CANDLES))
        winning_s_decayed = winning_s * decay
        logger.debug(
            f"[CVDDiv] Age-cap decay: dir={winning_dir} streak={streak} "
            f"strength {winning_s:.3f} → {winning_s_decayed:.3f} (decay={decay:.3f})"
        )
        if winning_s_decayed < MIN_STRENGTH:
            logger.info(
                f"[CVDDiv] ■ {winning_dir} signal expired (streak={streak} candles, "
                f"aged below MIN_STRENGTH={MIN_STRENGTH}). Treating as TREND, not divergence."
            )
            NULL_RESULT["n_snaps"] = n_snaps
            return NULL_RESULT
        winning_s = winning_s_decayed
        if winning_dir == "BULLISH":
            best_bull_s = winning_s
        else:
            best_bear_s = winning_s
    # ■■ End Age-Cap ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

    if best_bull_s >= best_bear_s:
        logger.info(
            f"[CVDDiv] ■ BULLISH | strength={best_bull_s:.3f} "
            f"method={best_bull_meth} k={best_bull_k} streak={self._cvd_div_streak_count} "
            f"confirms={bull_confirms} vol_spike={best_bull_vs:.2f}×"
        )
        return {
            "type":              "BULLISH",
            "strength":          round(best_bull_s, 4),
            "candles_confirmed": bull_confirms,
            "price_delta":       round(best_bull_pd, 4),
            "cvd_delta":         round(best_bull_cd, 2),
            "lookback_k":        best_bull_k,
            "vol_spike":         round(best_bull_vs, 3),
            "detection_method":  best_bull_meth,
            "n_snaps":           n_snaps,
        }
    else:
        logger.info(
            f"[CVDDiv] ■ BEARISH | strength={best_bear_s:.3f} "
            f"method={best_bear_meth} k={best_bear_k} streak={self._cvd_div_streak_count} "
            f"confirms={bear_confirms} vol_spike={best_bear_vs:.2f}×"
        )
        return {
            "type":              "BEARISH",
            "strength":          round(best_bear_s, 4),
            "candles_confirmed": bear_confirms,
            "price_delta":       round(best_bear_pd, 4),
            "cvd_delta":         round(best_bear_cd, 2),
            "lookback_k":        best_bear_k,
            "vol_spike":         round(best_bear_vs, 3),
            "detection_method":  best_bear_meth,
            "n_snaps":           n_snaps,
        }
else:
    # Direction changed — reset streak
    self._cvd_div_streak_dir   = "NONE"
    self._cvd_div_streak_count = 0
```

```
NULL_RESULT["n_snaps"] = n_snaps
return NULL_RESULT
```

■ backtest_hybrid.py

```
import os
import time
import numpy as np
import pandas as pd
from datetime import datetime, timedelta
import warnings

warnings.filterwarnings('ignore')

try:
    from colorama import Fore, Style, init
    init(autoreset=True)
except ImportError:
    class Fore: GREEN = RED = YELLOW = CYAN = WHITE = MAGENTA = ""
    class Style: RESET_ALL = BRIGHT = ""

# ================================================================================
# MAin CONFIGURATION and new changes for ATR Threshold for the Live Bot
# ATR threshold is the Minimum ATR required to enter the market.
# It makes sure that there is enough volatility to trade and make a profit.
# ================================================================================
CONFIG = {
    "symbols": ["BTCUSDT"],
    "timeframe_mins": 15,
    "initial_balance": 10000.0,
    "base_risk_pct": 1.0,
    "max_daily_loss_pct": 3.0,
    "min_confidence": 0.60,

    # Feature Engine
    "atr_period": 14,
    "rsi_period": 14,
    "vwap_period": 20,
    "skew_period": 20,  # was 50 — matches live bot Fix 2

    # Thresholds
    "trend_atr_threshold": 0.006,
    "range_z_threshold": 1.06,

    # ■■ Realistic execution friction ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    # slippage_bps: half-spread cost per side (3 bps on a $95k BTC ≈ $2.85 per side)
    # Set to 0 to see 'theoretical max' performance, 3-5 is realistic for liquid markets
    "slippage_bps": 3,
    # commission_pct: exchange fee per trade LEG (Coinbase Advanced taker = 0.10%)
    # Total round-trip cost at defaults: 3 bps slip ×2 + 0.10% comm ×2 = ~0.26% per trade
    "commission_pct": 0.0004,    # expressed as a fraction (0.001 = 0.1%)
}

# ================================================================================
# VECTORISED FEATURE ENGINE
# ================================================================================

def calc_atr(high, low, close, period=14):
    tr = np.maximum(high[1:] - low[1:],
                    np.maximum(np.abs(high[1:] - close[:-1]),
                               np.abs(low[1:] - close[:-1])))
    atr = np.full(len(close), np.nan)
    if len(tr) >= period:
        atr[period] = np.mean(tr[:period])
        for i in range(period + 1, len(close)):
            atr[i] = (atr[i-1] * (period - 1) + tr[i-1]) / period
    return atr

def calc_rsi(close, period=14):
    delta = np.diff(close)
    gains = np.where(delta > 0, delta, 0.0)
    losses = np.where(delta < 0, -delta, 0.0)
    rsi = np.full(len(close), 50.0)
    if len(gains) >= period:
        avg_g = np.mean(gains[:period])
        avg_l = np.mean(losses[:period])
        for i in range(period, len(gains)):
            avg_g = (avg_g * (period - 1) + gains[i]) / period
            avg_l = (avg_l * (period - 1) + losses[i]) / period
            rsi[i+1] = 100 - (100 / (1 + avg_g / avg_l)) if avg_l > 0 else 100
    return rsi
```

```
def calc_vwap_zscore(df, period=20):
    tp = (df['high'] + df['low'] + df['close']) / 3.0
    vol = df['volume']

    vwap = (tp * vol).rolling(period).sum() / vol.rolling(period).sum()
    std = tp.rolling(period).std(ddof=0)

    z = (df['close'] - vwap) / std
    return z.fillna(0).values

def calc_skewness(close, period=50):
    log_ret = np.zeros(len(close))
    log_ret[1:] = np.log(close[1:] / np.maximum(close[:-1], 1e-10))
    skew = pd.Series(log_ret).rolling(period).skew().fillna(0).values
    return skew

def calc_cvd_proxy(close, open_, volume, period=20, taker_buy_base=None):
    """
    Cumulative Volume Delta.
    If `taker_buy_base` is supplied (Binance kline field 9), uses the exact
    live formula matching App.tsx:  delta = 2 × takerBuyVol − totalVol
    Otherwise falls back to the candle-sign body-weighted proxy.
    """
    if taker_buy_base is not None:
        # Exact formula — mirrors the live frontend CVD calculation
        delta = (2.0 * taker_buy_base) - volume
    else:
        body  = np.abs(close - open_)
        conv  = np.clip(body / (body + 1e-10), 0.3, 1.0)
        dir_  = np.where(close >= open_, 1.0, -1.0)
        delta = dir_ * volume * conv
    cvd = pd.Series(delta).rolling(period).sum().fillna(0).values
    return cvd


# ═══════════════════════════════════════════════════════════════════════════
# REAL DATA FETCHER — Binance Public REST API
# ═══════════════════════════════════════════════════════════════════════════

def fetch_binance_candles(symbol: str, interval: str = "5m", years_back: int = 2) -> pd.DataFrame:
    """
    Fetch real OHLCV + taker-buy-volume klines from the Binance public API.
    Returns a DataFrame with columns: open, high, low, close, volume, taker_buy_base
    Falls back to an empty DataFrame on any network/API error.
    """
    try:
        import requests as _req
    except ImportError:
        print("  ■■  `requests` not installed. Run: pip install requests")
        return pd.DataFrame()

    end_dt   = datetime.now()
    start_dt = end_dt - timedelta(days=int(365 * years_back))
    start_ms = int(start_dt.timestamp() * 1000)
    end_ms   = int(end_dt.timestamp()   * 1000)

    print(f"  Fetching {symbol} ({interval}) from Binance API "
          f"[{start_dt.date()} → {end_dt.date()}]...")

    all_klines: list = []
    current_ms = start_ms

    while current_ms < end_ms:
        try:
            resp = _req.get(
                "https://api.binance.com/api/v3/klines",
                params={"symbol":    symbol,
                        "interval":  interval,
                        "startTime": current_ms,
                        "endTime":   end_ms,
                        "limit":     1000},
                timeout=20,
            )
            resp.raise_for_status()
            batch = resp.json()
        except Exception as exc:
            print(f"  ■■  Binance API error: {exc}. Falling back to CSV / synthetic data.")
            return pd.DataFrame()
```

```
        if not batch:
            break
        all_klines.extend(batch)
        current_ms = int(batch[-1][0]) + 1
        if len(batch) < 1000:
            break

    if not all_klines:
        return pd.DataFrame()

    cols = [
        'time', 'open', 'high', 'low', 'close', 'volume',
        'close_time', 'quote_vol', 'trades',
        'taker_buy_base', 'taker_buy_quote', 'ignore',
    ]
    df = pd.DataFrame(all_klines, columns=cols)
    df['time'] = pd.to_datetime(df['time'].astype(int), unit='ms')
    df.set_index('time', inplace=True)
    for col in ('open', 'high', 'low', 'close', 'volume', 'taker_buy_base'):
        df[col] = df[col].astype(float)

    print(f"  ✓ {len(df):,} real bars loaded  "
          f"({df.index[0].date()} → {df.index[-1].date()})")
    return df


# ═══════════════════════════════════════════════════════════════════════════
# STRATEGY & BAYESIAN FUSION
# ═══════════════════════════════════════════════════════════════════════════

def backtest_hybrid(df, config, symbol):
    close = df['close'].values
    high = df['high'].values
    low = df['low'].values
    open_ = df['open'].values
    vol = df['volume'].values
    ts = df.index

    print("  Computing features...")
    atr = calc_atr(high, low, close, config['atr_period'])
    rsi = calc_rsi(close, config['rsi_period'])
    zscore = calc_vwap_zscore(df, config['vwap_period'])
    skew = calc_skewness(close, config['skew_period'])
    # CVD: use real taker-buy volume when available (Binance kline field 'taker_buy_base')
    # This mirrors the live app formula exactly: delta = 2 * takerBuyVol - totalVol
    taker_buy_base = df['taker_buy_base'].values if 'taker_buy_base' in df.columns else None
    cvd = calc_cvd_proxy(close, open_, vol, 20, taker_buy_base=taker_buy_base)

    atr_pct = np.zeros_like(atr)
    valid_idx = close > 0
    atr_pct[valid_idx] = atr[valid_idx] / close[valid_idx]

    atr_pct_rank = np.zeros(len(close))
    for idx in range(50, len(close)):
        window = atr_pct[max(0, idx-2880):idx]
        if len(window) > 0:
            atr_pct_rank[idx] = np.mean(window <= atr_pct[idx])

    # Proxies for missing live data (LOB / Tape)
    # We use local extreme rollings to simulate walls / liquidity pools
    sw_high = pd.Series(high).rolling(20).max().shift(1).values
    sw_low  = pd.Series(low).rolling(20).min().shift(1).values

    # OFI proxy based on volume delta momentum
    ofi_proxy = pd.Series(np.where(close >= open_, vol, -vol)).rolling(5).sum().values
    ofi_normalised = np.tanh(ofi_proxy / 2.0)  # array, index with [i] in loop

    # Volume-surge proxy for the live bot's SCREAMING tape requirement (Stage 2 TREND gate)
    # Live:  10-second taker notional > 3× per-10s rolling baseline → SCREAMING
    # Proxy: current bar volume > 3× 60-bar rolling mean (same 3× multiplier, different window)
    vol_sma_60 = pd.Series(vol).rolling(60, min_periods=1).mean().shift(1).fillna(0).values

    print("  Simulating event-driven trades...")

    balance = config['initial_balance']
    trades = []
    open_trades = []
    daily_losses = 0
```

```
last_day = None
daily_halt = False

backtest_alpha = 7.0
backtest_beta = 5.0
backtest_regime_alpha = {"RANGE": 7.0, "NEUTRAL": 7.0, "TREND": 7.0, "LIQUIDITY": 7.0}
backtest_regime_beta = {"RANGE": 5.0, "NEUTRAL": 5.0, "TREND": 5.0, "LIQUIDITY": 5.0}
backtest_regime_count = {"RANGE": 0, "NEUTRAL": 0, "TREND": 0, "LIQUIDITY": 0}

for i in range(50, len(close)):
    cur_day = ts[i].date()
    if cur_day != last_day:
        daily_losses = 0
        daily_halt = False
        last_day = cur_day

    px = close[i]

    # 1. Manage Open Trades
    still_open = []
    for t in open_trades:
        hit_sl = (t['dir'] == 1 and low[i] <= t['sl']) or (t['dir'] == -1 and high[i] >= t['sl'])
        hit_tp = (t['dir'] == 1 and high[i] >= t['tp']) or (t['dir'] == -1 and low[i] <= t['tp'])

        if hit_sl or hit_tp:
            # Pessimistic Bias: if both hit in the same candle, assume Stop Loss occurred first
            if hit_sl and hit_tp:
                raw_exit = t['sl']
            else:
                raw_exit = t['sl'] if hit_sl else t['tp']

            is_long  = t['dir'] == 1

            # ■■ Apply realistic friction ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            slip = config.get('slippage_bps', 3) / 10_000.0
            comm = config.get('commission_pct', 0.0004)  # Binance USDM futures default

            # Slippage worsens both fill prices
            if is_long:
                eff_entry = t['entry'] * (1.0 + slip)
                eff_exit  = raw_exit  * (1.0 - slip)
            else:
                eff_entry = t['entry'] * (1.0 - slip)
                eff_exit  = raw_exit  * (1.0 + slip)

            # ■■ Correct Risk Position Sizing ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
            risk_usd = balance * (config['base_risk_pct'] / 100.0)
            sl_distance = abs(t['entry'] - t['sl'])

            # How much coin to buy so that sl_distance == risk_usd
            size_coin = risk_usd / max(sl_distance, 1e-9)
            notional_value = size_coin * t['entry']

            # Raw directional PnL based on nominal cost vs exit
            price_diff = (eff_exit - eff_entry) if is_long else (eff_entry - eff_exit)
            raw_pnl = size_coin * price_diff

            # Commission is paid on total notional volume traded (Entry + Exit)
            exit_notional = size_coin * eff_exit
            commission_cost = (notional_value + exit_notional) * comm

            pnl = raw_pnl - commission_cost
            r_mult = pnl / max(risk_usd, 1e-9)

            exit_px = raw_exit  # keep original for logging
            balance += pnl
            if pnl < 0:
                daily_losses += 1
                if daily_losses >= int(config['max_daily_loss_pct']):
                    daily_halt = True

            trades.append({
                "entry_time": t['entry_time'],
                "exit_time": ts[i],
                "dir": "BUY" if t['dir'] == 1 else "SELL",
                "entry": t['entry'],
                "exit": exit_px,
                "pnl": pnl,
```

```
            "r_mult": r_mult,
            "balance": balance,
            "regime": t['regime']
        })
        won = (pnl > 0)
        trade_regime = t['regime']
        if won:
            backtest_alpha += 1.0
            backtest_regime_alpha[trade_regime] = backtest_regime_alpha.get(trade_regime, 1.0) + 1.0
        else:
            backtest_beta += 1.0
            backtest_regime_beta[trade_regime] = backtest_regime_beta.get(trade_regime, 1.0) + 1.0
        backtest_regime_count[trade_regime] = backtest_regime_count.get(trade_regime, 0) + 1
    else:
        still_open.append(t)
open_trades = still_open

if daily_halt or len(open_trades) > 0:
    continue

# 2. Market Regime
regime = "NEUTRAL"
# Proximate to recent swing high/low (simulated walls)
near_wall = False
nav_h = sw_high[i]
nav_l = sw_low[i]

if not np.isnan(nav_h) and not np.isnan(nav_l):
    if abs(px - nav_h) / px <= 0.001 or abs(px - nav_l) / px <= 0.001:
        regime = "LIQUIDITY"
        near_wall = True

if not near_wall:
    # Mirror live bot Stage 2: TREND requires high ATR AND a volume surge
    # (proxy for the SCREAMING tape condition checked against Binance trade feed)
    sma_vol   = vol_sma_60[i] if i < len(vol_sma_60) else 0.0
    vol_surge = sma_vol > 0 and vol[i] > sma_vol * 3.0
    if atr_pct[i] > config['trend_atr_threshold'] and vol_surge:
        regime = "TREND"
    elif abs(zscore[i]) < config['range_z_threshold']:
        regime = "RANGE"

# 3. Liquidity Sweep Detection
sweep = None
if not np.isnan(nav_h) and not np.isnan(nav_l):
    prev_h, prev_l = high[i-1], low[i-1]
    prev_c = close[i-1]
    if prev_h > nav_h and px < nav_h:
        sweep = "ABOVE_HIGHS"
    elif prev_l < nav_l and px > nav_l:
        sweep = "BELOW_LOWS"

# 4. Synthese Pre-Bayes matching live `quant_engine.py`
curr_rsi = rsi[i]
curr_z = zscore[i]
curr_skew = skew[i]
curr_ofi = ofi_normalised[i]  # tanh scale (-1, +1), replaces raw proxy * 10.0

L_rsi  = 1.8 if curr_rsi > 60 else 0.55 if curr_rsi < 40 else 1.0
if curr_z < -1.22 and curr_ofi > 0.15:
    L_flow = 2.0
elif curr_z > 1.22 and curr_ofi < -0.15:
    L_flow = 0.5
else:
    L_z = 1.3 if curr_z < -1.22 else 0.76 if curr_z > 1.22 else 1.0
    L_o = 1.2 if curr_ofi > 0.30 else 0.83 if curr_ofi < -0.30 else 1.0
    L_flow = L_z * L_o

L_skew = 1.2 if curr_skew > 0.3 else 0.83 if curr_skew < -0.3 else 1.0
p_prior = backtest_alpha / (backtest_alpha + backtest_beta)
prior_odds = p_prior / (1.0 - p_prior)
bull_odds = prior_odds * L_rsi * L_flow * L_skew
bayes = bull_odds / (bull_odds + 1.0)

# 5. Strategy Layer
raw_direction = None
strategy = ""
if sweep:
```

```
    strategy = "SWEEP"
    if sweep == "ABOVE_HIGHS" and (curr_ofi < -0.15 or cvd[i] < 0):
        raw_direction = "SELL"
    elif sweep == "BELOW_LOWS" and (curr_ofi > 0.15 or cvd[i] > 0):
        raw_direction = "BUY"

elif regime == "TREND":
    strategy = "TREND"
    score = 0.0

    if bayes > 0.65:    score += 2.0
    elif bayes > 0.55:  score += 1.0
    elif bayes < 0.35:  score -= 2.0
    elif bayes < 0.45:  score -= 1.0

    if curr_ofi > 0.30:         score += 1.5
    elif curr_ofi > 0.15:        score += 0.75
    elif curr_ofi < -0.30:      score -= 1.5
    elif curr_ofi < -0.15:       score -= 0.75

    if cvd[i] > 0: score += 1.0
    elif cvd[i] < 0: score -= 1.0

    if curr_ofi > 0: score += 1.0
    elif curr_ofi < 0: score -= 1.0

    if score >= 1.5: raw_direction = "BUY"
    elif score <= -1.5: raw_direction = "SELL"

elif regime == "RANGE":
    strategy = "MEAN_REVERSION"
    atr_rank = atr_pct_rank[i]
    if atr_rank < 0.5:
        rsi_long_gate = 55.0
        rsi_short_gate = 45.0
    else:
        rsi_long_gate = 42.0
        rsi_short_gate = 58.0
    if curr_z >= 1.06 and curr_rsi > rsi_short_gate: raw_direction = "SELL"
    elif curr_z <= -1.06 and curr_rsi < rsi_long_gate: raw_direction = "BUY"

if not raw_direction: continue

# 6. Bayesian Fusion + ULIS proxy
odds = 1.0
if curr_ofi > 0.30:    odds *= 2.0
elif curr_ofi > 0.15:   odds *= 1.4
elif curr_ofi < -0.30: odds *= 0.5
elif curr_ofi < -0.15:  odds *= 0.7

if cvd[i] > 0: odds *= 1.25
elif cvd[i] < 0: odds *= 0.8

if curr_skew > 0.3: odds *= 1.12
elif curr_skew < -0.3: odds *= 0.88

if curr_rsi > 60: odds *= 1.15
elif curr_rsi < 40: odds *= 0.85

p_bull = odds / (odds + 1.0)
conf = p_bull if raw_direction == "BUY" else (1.0 - p_bull)

# CVD directional gate: penalise trades where CVD contradicts direction.
# Mirrors Stage 4.5 CVD divergence logic in the live bot.
cvd_confirms_long  = (raw_direction == "BUY"  and cvd[i] > 0)
cvd_confirms_short = (raw_direction == "SELL" and cvd[i] < 0)
if not (cvd_confirms_long or cvd_confirms_short):
    conf -= 0.04  # Mild penalty for CVD-contradicting direction

if (raw_direction == "BUY" and curr_ofi < 0 and curr_z < 0) or \
   (raw_direction == "SELL" and curr_ofi > 0 and curr_z > 0):
    conf -= 0.15 # ULIS cascade veto penalty simulation

if conf < config['min_confidence']:
    continue

# 7. Risk Engine
is_long = raw_direction == "BUY"
```

```
        cur_atr = atr[i]

        if strategy == "SWEEP" and sweep:
            sl_dist = abs(px - nav_h if sweep == "ABOVE_HIGHS" else nav_l) + cur_atr * 0.5
            sl_dist = max(sl_dist, cur_atr * 1.43)
        elif strategy == "TREND":
            sl_dist = cur_atr * 2.09  # matches REGIME_PARAMS TREND
        elif strategy == "MEAN_REVERSION":
            sl_dist = cur_atr * 1.43  # matches REGIME_PARAMS RANGE/NEUTRAL
        else:
            sl_dist = px * 0.008

        rr_map = {"SWEEP": 2.0, "TREND": 2.5, "MEAN_REVERSION": 1.8}
        rr_target = rr_map.get(strategy, 2.0)

        sl = px - sl_dist if is_long else px + sl_dist
        tp = px + (sl_dist * rr_target) if is_long else px - (sl_dist * rr_target)

        open_trades.append({
            "entry": px,
            "sl": sl,
            "tp": tp,
            "dir": 1 if is_long else -1,
            "entry_time": ts[i],
            "regime": regime
        })

    # Close remaining open trades at the last known price
    last_px = close[-1]
    slip = config.get('slippage_bps', 3) / 10_000.0
    comm = config.get('commission_pct', 0.001)
    for t in open_trades:
        is_long = t['dir'] == 1
        eff_entry = t['entry'] * (1.0 + slip) if is_long else t['entry'] * (1.0 - slip)
        eff_exit  = last_px  * (1.0 - slip) if is_long else last_px  * (1.0 + slip)

        # Correct position sizing (same as main loop)
        risk_usd = balance * (config['base_risk_pct'] / 100.0)
        sl_distance = abs(t['entry'] - t['sl'])
        size_coin = risk_usd / max(sl_distance, 1e-9)
        notional_value = size_coin * t['entry']

        price_diff = (eff_exit - eff_entry) if is_long else (eff_entry - eff_exit)
        raw_pnl = size_coin * price_diff

        exit_notional = size_coin * eff_exit
        commission_cost = (notional_value + exit_notional) * comm
        pnl = raw_pnl - commission_cost
        balance += pnl
        trades.append({
            "entry_time": t['entry_time'],
            "exit_time": ts[-1],
            "dir": "BUY" if t['dir'] == 1 else "SELL",
            "entry": t['entry'],
            "exit": last_px,
            "pnl": pnl,
            "r_mult": pnl / max(risk_usd, 1e-9),
            "balance": balance,
            "regime": t['regime']
        })

    return trades, balance

# ═══════════════════════════════════════════════════════════════════════════
# MAIN RUNNER
# ═══════════════════════════════════════════════════════════════════════════

def load_data(symbol: str, years_back: int = 2) -> pd.DataFrame:
    """
    Data loading priority (most to least realistic):
      1. Binance public REST API  — real OHLCV + taker-buy volume  [DEFAULT]
      2. Local CSV at data/{symbol}_M5.csv
      3. Synthetic random walk    — LAST RESORT, results are non-predictive
    """
    # ■■ 1. Real Binance data (preferred) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    df = fetch_binance_candles(symbol, interval="15m", years_back=years_back)
    if not df.empty:
        return df
```

```
    # ■■ 2. Local CSV fallback ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    csv_path = f"data/{symbol}_M5.csv"
    if os.path.exists(csv_path):
        print(f"  Loading {csv_path}...")
        df = pd.read_csv(csv_path, index_col=0, parse_dates=True)
        df.sort_index(inplace=True)
        return df

    # ■■ 3. Synthetic fallback (CI / offline only) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    print("\n  ■■  WARNING: No real data available. Using SYNTHETIC random walk.")
    print("  ■■  Results are for code validation ONLY — NOT predictive of live performance.\n")
    np.random.seed(42)
    end_date = datetime.now()
    try:
        start_date = end_date.replace(year=end_date.year - years_back)
    except ValueError:
        start_date = end_date.replace(year=end_date.year - years_back, day=end_date.day - 1)

    periods = int((end_date - start_date).total_seconds() / 300)  # 5-min bars
    dates   = pd.date_range(end=end_date, periods=periods, freq="5min")

    returns = np.random.normal(0, 0.001, periods) * np.abs(np.random.normal(1, 0.2, periods))
    close   = 30_000.0 * np.exp(np.cumsum(returns))   # BTC-realistic starting price
    high    = close * (1 + np.abs(np.random.normal(0, 0.0005, periods)))
    low     = close * (1 - np.abs(np.random.normal(0, 0.0005, periods)))
    open_   = np.roll(close, 1); open_[0] = close[0]
    vol     = np.random.lognormal(mean=10, sigma=1, size=periods)

    return pd.DataFrame(
        {'open': open_, 'high': high, 'low': low, 'close': close, 'volume': vol},
        index=dates,
    )

def run():
    print(f"\n{Fore.CYAN}{Style.BRIGHT}{'■'*62}")
    print(f"  Macro Hybrid Bot — Vectorised Backtester")
    print(f"  10-Year Test Engine")
    print(f"{'■'*62}{Style.RESET_ALL}\n")

    t_start = time.time()

    for symbol in CONFIG['symbols']:
        print(f"  {Fore.CYAN}■ {symbol}{Style.RESET_ALL}")
        # Try to load local CSV if it exists, otherwise use synthetic
        csv_path = f"data/{symbol}_M5.csv"
        if os.path.exists(csv_path):
            print(f"  Loading {csv_path}...")
            df = pd.read_csv(csv_path, index_col=0, parse_dates=True)
            df.sort_index(inplace=True)
        else:
            df = load_data(symbol)

        print(f"  Bars: {len(df):,} ({df.index[0].date()} -> {df.index[-1].date()})")

        trades, final_bal = backtest_hybrid(df, CONFIG, symbol)

        # Report
        df_t = pd.DataFrame(trades)
        if len(df_t) == 0:
            print(f"  {Fore.RED}No trades executed.{Style.RESET_ALL}")
            continue

        wins = (df_t['pnl'] > 0).sum()
        total = len(df_t)
        wr = wins / total * 100
        net = df_t['pnl'].sum()

        print(f"\n  Results for {symbol}:")
        print(f"  Initial Balance: ${CONFIG['initial_balance']:,.2f}")
        color = Fore.GREEN if net > 0 else Fore.RED
        print(f"  Final Balance:   {color}${final_bal:,.2f}{Style.RESET_ALL}")
        print(f"  Net Profit:      {color}${net:,.2f}{Style.RESET_ALL}")
        print(f"  Total Trades:    {total}")
        print(f"  Win Rate:        {wr:.1f}%")

        # Breakdown by regime
        print("\n  Performance by Regime:")
        by_regime = df_t.groupby('regime').agg(
```

```
            trades=('pnl', 'count'),
            win_rate=('pnl', lambda x: (x>0).mean() * 100),
            pnl=('pnl', 'sum')
        )
        print(by_regime.to_string())
        print(f"\n{Fore.CYAN}{Style.BRIGHT}{'■'*62}{Style.RESET_ALL}\n")

    print(f"  Total time: {time.time() - t_start:.2f}s")

if __name__ == "__main__":
    run()
```

■ verify_region.py

```
import ccxt
import sys
import logging

logging.basicConfig(level=logging.INFO, format="%(asctime)s | %(levelname)-8s | %(name)s | %(message)s")
logger = logging.getLogger("PreFlight")

def main():
    logger.info("Running pre-flight region and network check...")

    # Initialize the exchange strictly to check connectivity and region IP status
    # We do not need API keys just to load public markets.
    try:
        exchange = ccxt.binanceusdm({
            'enableRateLimit': True,
            'timeout': 15000,
        })

        logger.info("Attempting to load Binance USDM markets to verify region...")
        exchange.load_markets()

        logger.info("■ SUCCESS: Successfully connected to Binance USDM.")
        logger.info("■ SUCCESS: The current server region IP is ALLOWED by Binance.")
        sys.exit(0)

    except Exception as e:
        error_str = str(e).lower()
        if "451" in error_str or "restricted location" in error_str or "unavailable from a restricted location" in erro
            logger.error("■ CRITICAL ERROR: The server is located in a restricted region (e.g., USA).")
            logger.error("■ Binance responded with HTTP 451: Service unavailable from a restricted location.")
            logger.error("■ FIX: Go to your Railway Dashboard -> Service -> Settings -> Region, and change it to Europe
            # Exit with 1 to crash the deploy so it doesn't run silently and fail later.
            sys.exit(1)
        else:
            logger.error(f"■ CRITICAL ERROR: Failed to connect to Binance. Details: {str(e)}")
            sys.exit(1)

if __name__ == "__main__":
    main()
```